

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



HCMUTE

KHÓA LUẬN TỐT NGHIỆP

**THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG
ROBOT HƯỚNG DẪN VIÊN TỰ HÀNH TÍCH
HỢP TRÍ TUỆ NHÂN TẠO CHO MÔI
TRƯỜNG TRONG NHÀ**

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

Sinh viên: HUỲNH QUỐC THẮNG

MSSV: 22119230

BÙI QUANG TẤN

MSSV: 22119228

GVHD: TS. ĐỖ DUY TÂN

TP. HỒ CHÍ MINH, 06/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



HCMUTE

KHÓA LUẬN TỐT NGHIỆP

**THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG
ROBOT HƯỚNG DẪN VIÊN TỰ HÀNH TÍCH
HỢP TRÍ TUỆ NHÂN TẠO CHO MÔI
TRƯỜNG TRONG NHÀ**

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

Sinh viên: HUỲNH QUỐC THẮNG

MSSV: 22119230

BÙI QUANG TẤN

MSSV: 22119228

GVHD: TS. ĐỖ DUY TÂN

TP. HỒ CHÍ MINH, 06/2026

PHIẾU GIAO NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

1. Thông tin sinh viên

Họ và tên: Bùi Quang Tấn
Tel: 0334169310

MSSV: 22119228
Email:
buiquangtan.2004vta@gmail.com
MSSV: 22119230
Email:
thanghuynhdev2004@gmail.com

Họ và tên: Huỳnh Quốc Thắng
Tel: 0329370955

2. Thông tin đề tài

- Tên của đề tài: Thiết kế và triển khai hệ thống robot hướng dẫn viên tự hành tích hợp trí tuệ nhân tạo cho môi trường trong nhà.
- Mục đích của đề tài: Chế tạo Mobile Robot tự hành dùng Lidar nhằm tự động dẫn đường và kết hợp trí tuệ nhân tạo giúp giao tiếp, tương tác với con người trong môi trường trong nhà.

Đồ án tốt nghiệp được thực hiện tại: Bộ môn Kỹ Thuật Máy Tính, Khoa Điện – Điện Tử, Trường Đại học Công Nghệ Kỹ Thuật Thành phố Hồ Chí Minh.

Thời gian thực hiện: Từ ngày 01/03/2026 đến ngày 25/06/2026

3. Các nhiệm vụ cụ thể của đề tài

- Thiết kế xe tự hành, lập trình các chức năng cho phần cứng và phần mềm
- Triển khai các giải thuật xử lý dữ liệu cảm biến để robot tự động lập bản đồ môi trường, định vị vị trí và điều hướng di chuyển tránh vật cản.
- Tích hợp hệ thống truyền thông âm thanh kết nối các mô hình trí tuệ nhân tạo xử lý giọng nói, xây dựng giao diện quản lý từ xa.

4. Lời cam đoan của sinh viên

Tôi (Chúng tôi) cam đoan đồ án tốt nghiệp là công trình nghiên cứu của bản thân dưới sự hướng dẫn của giảng viên hướng dẫn.

Các kết quả trình bày trong đồ án là trung thực và không sao chép từ bất kỳ công trình nào khác.

	Tp.HCM, ngày tháng năm 20.... Sinh viên thực hiện đồ án (Ký và ghi rõ họ tên)
--	--

Giáo viên hướng dẫn xác nhận về mức độ hoàn thành và cho phép được bảo vệ:

Xác nhận của Bộ môn	Tp.HCM, ngày tháng năm 20.... Giáo viên hướng dẫn (Ký, ghi rõ họ tên và học hàm học vị)
---------------------	--

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: Thiết kế và trình khai hệ thống Robot hướng dẫn viên tự hành tích hợp trí tuệ nhân tạo cho môi trường tự nhiên

Sinh viên: + HUYNH QUOC THANG

MSSV: 22119230

+ BUI QUANG TAN

MSSV: 22119228

Hướng dẫn: PGS. Đỗ Duy Tân

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề: ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể, đề tài có tính mới, cấp thiết, đề tài có khả năng ứng dụng, tính sáng tạo.

Đặt vấn đề rõ ràng, hợp lý.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết, có phân tích và đánh giá phù hợp, có tính mới và tính sáng tạo.

Hợp lý.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài: phân tích và đánh giá/ kiểm thử thiết kế hợp lý, có tính sáng tạo/ kiểm định chất chẽ và đảm bảo độ tin cậy.

Kết quả thực hiện phù hợp với mục tiêu đề tài.

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

Phù hợp.

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu, có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

Bố cục hợp lý, cấu trúc rõ ràng.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

Kỹ năng tốt.

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo, tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

TIK phù hợp.

8. Đánh giá về sự trùng lặp của đề tài

Cần không định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

Có cái tên sa và các đề tài đã có. Tỷ lệ check Turnitin là 24%.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

Cải thiện trình bày các bảng biểu, chỉnh sửa các lỗi chính tả.

- Phân tích rõ hơn kết quả đánh giá.

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Tốt.

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Báo cáo đạt/không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/không được phép bảo vệ khóa luận tốt nghiệp"

Báo cáo đạt yêu cầu 1 KLTN kỹ sư. Được phép bảo vệ KLTN.

Tp. HCM, ngày 25 tháng 6 năm 2026

Người nhận xét
(Ký và ghi rõ họ tên)

Đỗ Duy Tân

* Giảng viên hướng dẫn có thể ghi tiếp vào mặt sau

LỜI CẢM ƠN

Trong suốt quá trình thực hiện đồ án tốt nghiệp, nhóm đã nhận được rất nhiều sự quan tâm, hỗ trợ và động viên từ thầy cô, gia đình và bạn bè.

Trước tiên, nhóm xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Thầy Đỗ Duy Tân – người đã trực tiếp hướng dẫn và đồng hành cùng nhóm trong suốt quá trình thực hiện đề tài. Những định hướng nghiên cứu, góp ý về kỹ thuật và kinh nghiệm thực tiễn mà Thầy chia sẻ đã giúp nhóm vượt qua nhiều khó khăn, từng bước hoàn thiện hệ thống và nâng cao chất lượng báo cáo.

Nhóm cũng xin gửi lời cảm ơn đến quý thầy cô Khoa Điện – Điện tử, đã tận tâm giảng dạy, truyền đạt kiến thức nền tảng trong suốt những năm học vừa qua. Đó là hành trang quan trọng giúp nhóm có đủ năng lực tiếp cận và giải quyết các bài toán kỹ thuật trong đề tài này.

Mặc dù còn nhiều thiếu sót. Nhóm rất mong nhận được sự góp ý của quý thầy cô và các bạn để hệ thống có thể được cải thiện và hoàn thiện hơn trong tương lai.

Nhóm thực hiện đề tài
(Ký và ghi rõ họ tên)

LỜI CAM ĐOAN

Chúng tôi, Huỳnh Quốc Thắng và Bùi Quang Tấn, xin cam đoan những nội dung sau đây về đồ án tốt nghiệp với đề tài “*Thiết kế và triển khai hệ thống robot hướng dẫn viên tự hành tích hợp trí tuệ nhân tạo cho môi trường trong nhà*”, được thực hiện dưới sự hướng dẫn của Thầy Đỗ Duy Tân.

Đây là công trình do chính nhóm độc lập nghiên cứu, thiết kế và triển khai. Các kết quả thực nghiệm, số liệu đo đạc và nhận xét đánh giá được trình bày trong báo cáo phản ánh trung thực quá trình làm việc thực tế của nhóm, không bị chỉnh sửa hay làm sai lệch vì bất kỳ mục đích nào. Những nội dung kế thừa từ tài liệu, công trình hay nguồn thông tin bên ngoài đều được nhận diện rõ ràng và trích dẫn đúng theo quy định học thuật hiện hành.

Nhóm xin chịu hoàn toàn trách nhiệm trước Nhà trường về tính trung thực, tính chính xác và sự liêm chính trong toàn bộ nội dung báo cáo. Trong trường hợp phát hiện bất kỳ vi phạm nào liên quan đến sao chép hay gian lận học thuật, nhóm sẵn sàng chịu mọi hình thức xử lý theo quy định.

Nhóm thực hiện đề tài
(Ký và ghi rõ họ tên)

Mục lục

1	TỔNG QUAN	1
1.1	GIỚI THIỆU	1
1.1.1	Các nghiên cứu trước đó	1
1.1.2	Khoảng trống nghiên cứu và hướng tiếp cận của đề tài	2
1.2	LÝ DO CHỌN ĐỀ TÀI	3
1.3	MỤC TIÊU NGHIÊN CỨU	3
1.4	NHIỆM VỤ NGHIÊN CỨU	4
1.5	PHƯƠNG PHÁP NGHIÊN CỨU	4
1.6	ĐÓNG GÓP CHÍNH CỦA ĐỀ TÀI	5
1.7	PHẠM VI VÀ GIỚI HẠN NGHIÊN CỨU	5
1.7.1	Phạm vi nghiên cứu	5
1.7.2	Giới hạn nghiên cứu	5
1.8	BỔ CỤC ĐỀ TÀI	6
2	CƠ SỞ LÝ THUYẾT	7
2.1	TỔNG QUAN VỀ PHẦN CỨNG	7
2.1.1	Giới thiệu về LiDAR	7
2.1.2	Giới thiệu về ORANGE PI 5	8
2.1.3	Giới thiệu về vi điều khiển STM32F103	9
2.1.4	Động cơ giảm tốc TT có Encoder	11
2.1.5	Mạch điều khiển động cơ L298N	13
2.1.6	Giới thiệu về mạch giảm áp buck DC-DC XY-3606	14
2.2	ĐỘNG HỌC ROBOT DI ĐỘNG BÁNH VI SAI VÀ ĐIỀU KHIỂN VẬN TỐC	15
2.2.1	Mô hình hình học và Ràng buộc động học toán học	16
2.2.2	Hệ phương trình động học thuận và ngược	17
2.2.3	Mô hình tính toán định vị động học (Odometry)	19
2.2.4	Phương pháp đo lường và tính toán vận tốc từ Encoder	20
2.2.5	Thuật toán điều khiển PID rời rạc cho động cơ DC	22
2.2.6	Hệ thống phản hồi vòng kín và mối quan hệ giữa PID - Odometry	24
2.3	FRAMEWORK ROS 2 VÀ CÔNG CỤ PHÁT TRIỂN HỆ THỐNG	26
2.3.1	Tổng quan về ROS 2 và Mô hình truyền thông	26
2.3.2	Cơ chế Quality of Service trong ROS 2	27
2.3.3	Công cụ trực quan hóa RViz2	28
2.4	SLAM, ĐỊNH VỊ VÀ ĐIỀU HƯỚNG TỰ HÀNH	29
2.4.1	Bài toán SLAM và mô hình bản đồ lưới chiếm dụng	29
2.4.2	Dữ liệu 2D LiDAR và kỹ thuật Scan Matching	32
2.4.3	Gói SLAM Toolbox và phương pháp SLAM dựa trên đồ thị tư thế	33

2.4.4	Định vị thời gian thực bằng AMCL	35
2.4.5	Khung điều hướng Navigation2 và bản đồ chi phí	36
2.4.6	Quy hoạch đường đi toàn cục bằng thuật toán A*	38
2.4.7	Điều khiển bám quỹ đạo cục bộ bằng Regulated Pure Pursuit	41
2.4.8	Thiết kế cây tọa độ TF trong ROS 2	44
2.5	TRUYỀN THÔNG THỜI GIAN THỰC GIỮA ROBOT VÀ MÁY CHỦ	46
2.5.1	Giao thức WebRTC và nền tảng LiveKit	46
2.5.2	Audio track, dữ liệu ứng dụng và vai trò của SFU	47
2.5.3	Thiết lập phiên, NAT traversal và khả năng phục hồi kết nối	47
2.6	XỬ LÝ TIẾNG NÓI VÀ MÔ HÌNH AI HỘI THOẠI	48
2.6.1	Phát hiện hoạt động giọng nói và xác định lượt hội thoại	48
2.6.2	Chuyển đổi tiếng nói thành văn bản	49
2.6.3	Mô hình ngôn ngữ lớn và hội thoại theo ngữ cảnh vị trí	49
2.6.4	Chuyển đổi văn bản thành tiếng nói	50
2.6.5	Pipeline hội thoại đầu cuối và các thành phần độ trễ	51
2.7	CƠ SỞ CẤU HÌNH TOUR VÀ GIAO DIỆN QUẢN LÝ HỆ THỐNG	51
2.7.1	Mô hình dữ liệu Map–Area–Point–Tour	51
2.7.2	Chuyển đổi tọa độ ảnh bản đồ và tọa độ ROS 2	52
2.7.3	Giao diện quản lý bản đồ	52
2.7.4	Giao diện thiết kế khu vực, điểm tham quan và lộ trình	53
2.7.5	Giao diện giám sát và điều khiển thời gian thực	54
3	THIẾT KẾ HỆ THỐNG	56
3.1	YÊU CẦU THIẾT KẾ VÀ SƠ ĐỒ KHỐI TỔNG THỂ	56
3.1.1	Yêu cầu thiết kế	56
3.1.2	Sơ đồ khối tổng thể hệ thống	57
3.2	THIẾT KẾ TỪNG KHỐI TRONG HỆ THỐNG	58
3.2.1	Sơ đồ nguyên lý hệ thống	58
3.2.2	Khối điều khiển động cơ	59
3.2.3	Khối cảm biến và khối nhập xuất dữ liệu	60
3.2.4	Khối xử lý trung tâm	61
3.2.5	Khối Cloud	61
3.3	LƯU ĐỒ GIẢI THUẬT	63
3.3.1	Lưu đồ khởi động hệ thống	63
3.3.2	Lưu đồ định vị AMCL	65
3.3.3	Lưu đồ lập kế hoạch và điều hướng Nav2	67
3.3.4	Lưu đồ giao tiếp âm thanh và hướng dẫn người dùng	68
4	THI CÔNG VÀ ĐÁNH GIÁ HỆ THỐNG	70
4.1	THI CÔNG PHẦN CỨNG	70

4.1.1	LẮP RÁP ROBOT	70
4.2	KẾT QUẢ THỰC NGHIỆM	72
4.2.1	Thí nghiệm ở các tình huống	72
4.2.2	Đánh giá tổng quan	85
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	86
5.1	KẾT LUẬN	86
5.2	HẠN CHẾ	86
5.3	HƯỚNG PHÁT TRIỂN	87

Danh sách bảng

1.1.1 So sánh các hệ thống robot hướng dẫn liên quan	2
2.1.1 Thông số kỹ thuật của LiDAR RPLIDAR A1M8-R6	8
2.1.2 Thông số kỹ thuật của Orange Pi 5	9
2.1.3 Thông số kỹ thuật của vi điều khiển STM32F103	11
2.1.4 Thông số kỹ thuật của động cơ giảm tốc TT có encoder	12
2.1.5 Thông số kỹ thuật của mạch điều khiển động cơ L298N	14
2.1.6 Thông số kỹ thuật của mạch giảm áp Buck DC-DC XY-3606 5V 5A	15
4.2.1 Kết quả thực nghiệm các chức năng của hệ thống	72
4.2.2 Cấu hình sử dụng trong quá trình xây dựng bản đồ	74
4.2.3 Các tham số ảnh hưởng đến độ chính xác điều hướng	78

Danh sách hình vẽ

2.1.1 Hình ảnh thực tế của LiDAR RPLIDAR A1M8-R6 [1]	7
2.1.2 Hình ảnh thực tế của Orange Pi 5 [2]	9
2.1.3 Hình ảnh thực tế của vi điều khiển STM32F103 [3]	10
2.1.4 Hình ảnh thực tế của động cơ giảm tốc TT có encoder 2 pha AB [4]	12
2.1.5 Hình ảnh thực tế của mạch điều khiển động cơ L298N [5]	13
2.1.6 Hình ảnh thực tế của mạch giảm áp Buck DC-DC XY-3606 5V 5A [6]	15
2.2.1 Mô hình động học robot bánh vi sai và tâm quay tức thời ICC.	17
2.2.2 Tín hiệu quadrature encoder và nguyên lý giải mã X4.	21
2.2.3 Đáp ứng vận tốc hai động cơ khi chưa có bộ điều khiển PID	23
2.2.4 Đáp ứng vận tốc hai động cơ khi sử dụng điều khiển PID	24
2.2.5 Vòng điều khiển vận tốc bánh xe dùng PID và encoder.	25
2.3.1 Kiến trúc phân tầng tổng quan của hệ điều hành robot ROS 2[7]	27
2.3.2 Công cụ trực quan hóa RViz2	29
2.4.1 Mô tả bài toán SLAM	30
2.4.2 Kỹ thuật scan matching	32
2.4.3 Slam Toolbox trong Ros2	35
2.4.4 Đường đi trong bản đồ Global có vật cản cố định	37
2.4.5 Bản đồ chi phí trong Ros2	38
2.4.6 Ví dụ minh họa thuật toán A* trên đồ thị có trọng số	39
2.4.7 Nguyên lý lựa chọn điểm nhìn trước trong bộ điều khiển Pure Pursuit [8]	41
2.4.8 Ảnh hưởng của khoảng nhìn trước đến hành vi bám quỹ đạo [8]	43
2.4.9 Cây tọa độ TF của robot Wall-E	44
2.7.1 Giao diện quản lý bản đồ trên nền tảng Web Admin	53
2.7.2 Giao diện thiết kế khu vực, điểm tham quan và ngữ cảnh tại Studio	53
2.7.3 Giao diện vận hành và giám sát robot thời gian thực	54
3.1.1 Sơ đồ khối tổng thể của hệ thống Robot	57
3.2.1 Sơ đồ nguyên lý của hệ thống Robot	59
3.2.2 Khối điều khiển động cơ	59
3.2.3 Khối cảm biến	60
3.2.4 Khối nhập xuất dữ liệu	60
3.2.5 Khối xử lý trung tâm	61
3.2.6 Khối Cloud	62
3.3.1 Lưu đồ khởi động hệ thống ROS 2 Navigation	63
3.3.2 Lưu đồ thuật toán định vị AMCL	65
3.3.3 Lưu đồ quá trình lập kế hoạch và điều hướng của hệ thống Navigation2	67
3.3.4 Lưu đồ giao tiếp âm thanh và tương tác giọng nói của robot	68

4.1.1	Bố trí các thiết bị trên mặt trên của robot	70
4.1.2	Robot hoàn thiện sau khi lắp ráp	71
4.2.1	Luồng kiểm thử quá trình xây dựng bản đồ bằng SLAM	73
4.2.2	Sơ đồ bố trí thực tế của phòng Lab C406B	75
4.2.3	Bản đồ môi trường được xây dựng bởi SLAM Toolbox	75
4.2.4	Chú thích các thành phần trên bản đồ SLAM	76
4.2.5	luồng kiểm thử quá trình điều hướng tự động	77
4.2.6	Ví dụ một lần thử nghiệm điều hướng khoảng cách 1.8m	79
4.2.7	Giá trị khoảng cách đo được sau các lần kiểm thử tại các vị trí khác nhau	80
4.2.8	Câu hỏi và câu trả lời trong các ngữ cảnh tham quan	82
4.2.9	Kết quả đo độ trễ đối với nhóm câu lệnh bỏ qua để tiếp tục tham quan	83
4.2.10	Kết quả đo độ trễ đối với nhóm câu hỏi đúng ngữ cảnh	84
4.2.11	Kết quả đo độ trễ đối với nhóm câu hỏi không thuộc ngữ cảnh	84

Danh mục các từ viết tắt

Dưới đây là danh mục các từ viết tắt được sử dụng trong báo cáo.

Các từ viết tắt	Định nghĩa
AI	Artificial Intelligence
ANN	Artificial Neural Network
AMCL	Adaptive Monte Carlo Localization
CNN	Convolutional Neural Network
DL	Deep Learning
LiDAR	Light Detection and Ranging
LLM	Large Language Model
ML	Machine Learning
RNN	Recurrent Neural Network
ROS	Robot Operating System
SE	Squeeze-and-Excitation
SLAM	Simultaneous Localization and Mapping
STT	Speech-to-Text
TTS	Text-to-Speech

Chương 1

TỔNG QUAN

1.1 GIỚI THIỆU

Sự phát triển của robot di động tự hành và trí tuệ nhân tạo đã mở ra khả năng xây dựng các hệ thống robot dịch vụ có thể vừa di chuyển trong môi trường thực, vừa giao tiếp với con người bằng ngôn ngữ tự nhiên. Trong các không gian trong nhà như phòng thí nghiệm, trường học, bảo tàng và khu trưng bày, robot hướng dẫn có thể hỗ trợ giới thiệu thông tin, dẫn người dùng đến các vị trí cần thiết và cung cấp nội dung phù hợp với khu vực đang tham quan.

Một hệ thống robot hướng dẫn hoàn chỉnh không chỉ yêu cầu khả năng định vị và điều hướng an toàn, mà còn cần kết hợp nhiều thành phần khác nhau, bao gồm điều khiển chuyển động, xây dựng bản đồ, quản lý điểm tham quan, truyền thông thời gian thực, nhận dạng tiếng nói, mô hình ngôn ngữ lớn, tổng hợp tiếng nói và giao diện quản trị. Đặc biệt, nội dung phản hồi của robot cần được gắn với vị trí vật lý hiện tại để hạn chế câu trả lời không liên quan và tạo ra trải nghiệm tương tác phù hợp với từng khu vực.

Trên cơ sở đó, đề tài tập trung thiết kế và triển khai một robot trợ lý tự hành hoạt động trong phòng thí nghiệm. Hệ thống sử dụng ROS 2 cho các chức năng điều khiển, xây dựng bản đồ, định vị và điều hướng; đồng thời tích hợp hệ thống hội thoại tiếng Việt và nền tảng Web để cấu hình bản đồ, khu vực, điểm tham quan, nội dung hướng dẫn và giám sát trạng thái robot trong quá trình vận hành.

1.1.1 Các nghiên cứu trước đó

Trong những năm gần đây, nhiều nghiên cứu đã tập trung phát triển robot hướng dẫn tự hành nhằm hỗ trợ khách tham quan trong các môi trường trong nhà. Các nghiên cứu liên quan có thể được xem xét theo hai hướng chính: nâng cao khả năng tự hành của robot và tăng cường khả năng tương tác giữa người và robot.

Nghiên cứu **DORIS – A Tour-Guide Robot: Moving Towards Interaction with Humans (2020)** [9] tập trung cải thiện khả năng tương tác người–robot thông qua giao tiếp bằng giọng nói, biểu cảm khuôn mặt và cơ chế biểu đạt cảm xúc, nhằm nâng cao trải nghiệm của người dùng trong không gian công cộng. Tuy nhiên, cơ chế hội thoại vẫn phụ thuộc nhiều vào tập luật và cơ sở dữ liệu được xây dựng trước, do đó khả năng xử lý các câu hỏi mở và duy trì hội thoại tự nhiên còn hạn chế.

Nghiên cứu **Design and Simulation of Indoor Tour Guide Robot Based on ROS (2021)** [10] xây dựng một robot hướng dẫn trong nhà với các chức năng điều hướng tự hành, tránh vật cản và tương tác giọng nói cơ bản dựa trên nền tảng ROS 1. Tuy nhiên, hệ thống chủ yếu được đánh giá trong môi trường mô phỏng và nội dung tương tác được tổ chức theo các kịch bản định trước, chưa hỗ trợ hỏi đáp mở dựa trên ngữ cảnh vị trí của robot.

Nghiên cứu **Tour Guide Robot: A 5G-Enabled Robot Museum Guide (2024)** [11] của Rosa và cộng sự triển khai thực tế một robot hướng dẫn bảo tàng sử dụng nền tảng robot humanoid có sẵn, kết hợp điều hướng tự hành trên ROS 1 với hệ thống nhận dạng giọng nói qua đám mây và quản lý hội thoại thông qua Dialogflow, trong đó phần lớn tính toán được chuyển lên máy chủ từ xa qua mạng 5G. Hệ thống được kiểm thử trực tiếp trong môi trường bảo tàng và cho thấy tính khả thi của kiến trúc xử lý phân tán. Tuy nhiên, cơ chế hội thoại vẫn dựa trên máy trạng thái hữu hạn theo intent định sẵn, chưa ứng dụng mô hình ngôn ngữ lớn để sinh câu trả lời linh hoạt theo ngữ cảnh vị trí hiện tại của robot.

1.1.2 Khoảng trống nghiên cứu và hướng tiếp cận của đề tài

Các công trình trên đã chứng minh tính khả thi của robot hướng dẫn tự hành và khả năng tích hợp giao tiếp giọng nói. Tuy nhiên, phần lớn các hệ thống hiện có còn giới hạn, chưa tận dụng được khả năng sinh ngôn ngữ linh hoạt của các mô hình ngôn ngữ lớn để phản hồi theo đúng ngữ cảnh của robot. Bên cạnh đó, các nền tảng tự hành vẫn chủ yếu dựa trên ROS 1 với kiến trúc tập trung, chưa khai thác ưu điểm của ROS 2 về khả năng phân tán, ổn định thời gian thực và hỗ trợ triển khai trên phần cứng nhúng chi phí thấp. Đề tài này hướng đến lấp đầy khoảng trống đó bằng cách kết hợp nền tảng tự hành ROS 2 với mô hình ngôn ngữ lớn có khả năng hội thoại theo ngữ cảnh, triển khai trên phần cứng tự thiết kế từ linh kiện phổ thông và tích hợp giao diện Web để quản lý lộ trình và giám sát robot theo thời gian thực.

Bảng 1.1.1 tóm tắt sự khác biệt giữa các hệ thống liên quan và hệ thống được đề xuất trong đề tài.

Bảng 1.1.1: So sánh các hệ thống robot hướng dẫn liên quan

Hệ thống	Tự thiết kế phần cứng	Nền tảng tự hành	Hội thoại dựa trên LLM	Ngữ cảnh theo vị trí	Hạ tầng mạng
DORIS [9]	Có	Có	Không	Kịch bản định trước	Không đề cập
Zhang và cộng sự [10]	Có	ROS 1 (mô phỏng)	Không	Hạn chế	Không đề cập
Rosa và cộng sự [11]	Không	ROS 1 (thực tế)	Không	Hạn chế	5G chuyên dụng
Hệ thống đề xuất	Có	ROS 2 (thực tế)	Có	Có	Wi-Fi thông thường

Từ khoảng trống trên, đề tài lựa chọn hướng tiếp cận tích hợp theo kiến trúc nhiều tầng. Tầng điều khiển thấp sử dụng vi điều khiển để đọc encoder và điều khiển vận tốc hai động cơ. Tầng robot sử dụng ROS 2, SLAM Toolbox, AMCL và Navigation2 để xây dựng bản đồ, định vị và điều hướng. Tầng tương tác sử dụng truyền thông âm thanh thời gian thực kết hợp các mô hình nhận dạng tiếng nói, mô hình ngôn ngữ lớn và tổng hợp tiếng nói. Tầng quản trị Web cho phép tổ chức dữ liệu theo cấu trúc bản đồ, khu vực, điểm tham quan và lộ trình, từ đó cung cấp đúng ngữ cảnh cho hệ thống hội thoại tại từng vị trí.

1.2 LÝ DO CHỌN ĐỀ TÀI

Trong phòng thí nghiệm nghiên cứu, khách tham quan và sinh viên thường cần được giới thiệu về các khu vực làm việc, thiết bị, hướng nghiên cứu và các nguyên mẫu đang phát triển. Việc giới thiệu trực tiếp bởi người phụ trách có ưu điểm về tính linh hoạt nhưng phụ thuộc vào thời gian của nhân sự, khó duy trì liên tục và khó mở rộng khi có nhiều lượt tham quan.

Các hình thức cung cấp thông tin cố định như bảng giới thiệu, mã QR hoặc màn hình hiển thị có chi phí thấp nhưng thiếu khả năng dẫn đường và tương tác hai chiều. Người dùng vẫn phải tự tìm vị trí, chủ động đọc nội dung và không thể đặt các câu hỏi mở dựa trên nhu cầu thực tế. Vì vậy, một robot có khả năng tự di chuyển, phát nội dung giới thiệu và trả lời câu hỏi tại đúng khu vực có thể tạo ra trải nghiệm trực quan và chủ động hơn.

Về mặt kỹ thuật, sự phát triển của cảm biến LiDAR, máy tính nhúng, ROS 2, các thuật toán SLAM, định vị xác suất, điều hướng tự hành và mô hình ngôn ngữ lớn giúp việc xây dựng một nguyên mẫu robot hướng dẫn trở nên khả thi. Việc kết hợp các công nghệ này trong cùng một hệ thống cũng tạo điều kiện để nghiên cứu mối liên hệ giữa phần cứng điều khiển, robot tự hành, truyền thông thời gian thực và trí tuệ nhân tạo hội thoại.

Do đó, đề tài được lựa chọn nhằm xây dựng một nền tảng robot trợ lý có khả năng hỗ trợ giới thiệu phòng thí nghiệm, đồng thời tạo ra mô hình thử nghiệm cho các bài toán robot dịch vụ trong trường học, bảo tàng, khu triển lãm và các không gian công cộng trong nhà. Đề tài chỉ tập trung ở mức nguyên mẫu nghiên cứu, không khẳng định hệ thống đã sẵn sàng triển khai thương mại hoặc vận hành đại trà.

1.3 MỤC TIÊU NGHIÊN CỨU

Đề tài hướng đến xây dựng một hệ thống robot hướng dẫn viên tự hành trong nhà hoàn chỉnh với các mục tiêu cụ thể như sau.

- Về phần cứng, thiết kế và lắp ráp nền tảng robot di động tích hợp đầy đủ các thiết bị cảm biến, truyền động, xử lý và giao tiếp âm thanh. Về điều khiển chuyển động, xây dựng vòng điều khiển vận tốc cho từng động cơ và cung cấp thông tin phản hồi phục vụ định vị.
- Về tự hành, robot cần xây dựng được bản đồ môi trường, định vị và điều hướng tự động đến các điểm tham quan.
- Về hướng dẫn, xây dựng cơ chế tour cho phép robot di chuyển tuần tự, tự động phát nội dung giới thiệu và chuyển trạng thái giữa các giai đoạn.
- Về tương tác, phát triển chức năng hỏi đáp bằng giọng nói tiếng Việt dựa trên ngữ cảnh khu vực hiện tại, với thời gian phản hồi trung bình không vượt quá 10 s.
- Xây dựng giao diện Web để quản lý bản đồ, cấu hình lộ trình và giám sát trạng thái robot theo thời gian thực.

1.4 NHIỆM VỤ NGHIÊN CỨU

Các nhiệm vụ chính được thực hiện trong đề tài bao gồm:

- Khảo sát các nghiên cứu, kiến trúc và công nghệ liên quan đến robot di động, điều hướng tự hành và tương tác giọng nói.
- Triển khai xây dựng bản đồ, định vị và lập kế hoạch đường đi, bám quỹ đạo và xử lý các tình huống điều hướng trong phạm vi thử nghiệm.
- Xây dựng nền tảng Web quản lý bản đồ, khu vực, điểm tham quan, ngữ cảnh, lộ trình và trạng thái vận hành của robot.
- Tích hợp nền tảng truyền thông thời gian thực và các mô hình phát hiện giọng nói, mô hình ngôn ngữ lớn để hỗ trợ hội thoại.

1.5 PHƯƠNG PHÁP NGHIÊN CỨU

Đề tài kết hợp các phương pháp sau. **Nghiên cứu tài liệu** nhằm khảo sát các công trình, sách kỹ thuật và tài liệu chính thức liên quan đến các thành phần được sử dụng. **Phân tích và thiết kế hệ thống** nhằm phân rã hệ thống thành các khối chức năng độc lập và xác định giao diện giữa chúng. **Thiết kế thực nghiệm** bao gồm lắp ráp nguyên mẫu, cấu hình thuật toán và thử nghiệm lặp lại trong môi trường thực. **Đo lường và đánh giá** dựa trên các số liệu như sai số vị trí, độ trễ phản hồi và mức độ phù hợp của câu trả lời. **Cải tiến lặp** thông qua phân tích lỗi và điều chỉnh tham số để nâng cao độ ổn định toàn hệ thống.

1.6 ĐÓNG GÓP CHÍNH CỦA ĐỀ TÀI

Các kết quả và đóng góp triển khai chính của đề tài bao gồm:

- Xây dựng nguyên mẫu robot di động bánh vi sai, trong đó tầng xử lý cấp cao và tầng điều khiển thời gian thực được tách biệt rõ ràng.
- Xây dựng mô hình dữ liệu bản đồ khu vực điểm tham quan lộ trình, cho phép liên kết tọa độ vật lý của robot với nội dung hướng dẫn và ngữ cảnh hội thoại.
- Phát triển hệ thống tương tác giọng nói tiếng Việt sử dụng mô hình trí tuệ nhân tạo trên máy chủ, trong đó câu trả lời được tạo dựa trên vị trí và trạng thái hiện tại của robot.
- Phát triển giao diện Web hỗ trợ quản lý bản đồ, cấu hình điểm tham quan, biên soạn nội dung, điều khiển tour và theo dõi trạng thái robot trong quá trình vận hành.

1.7 PHẠM VI VÀ GIỚI HẠN NGHIÊN CỨU

1.7.1 Phạm vi nghiên cứu

- Robot được thiết kế theo cấu hình bánh vi sai và hoạt động trên bề mặt tương đối bằng phẳng trong môi trường trong nhà.
- Môi trường triển khai và kiểm thử chính là phòng Lab C406B với bản đồ hai chiều và các điểm tham quan được cấu hình trước.
- Chức năng tự hành bao gồm xây dựng bản đồ bằng LiDAR, định vị trên bản đồ tĩnh, lập kế hoạch đường đi và điều khiển robot đến các điểm đích.
- Chức năng hướng dẫn bao gồm phát nội dung giới thiệu tại điểm đến, nhận câu hỏi tiếng Việt, tạo câu trả lời dựa trên ngữ cảnh và nhận xử lý một số lệnh điều khiển tour.
- Nền tảng Web được sử dụng để quản lý cấu hình tour và giám sát các trạng thái chính của robot;

1.7.2 Giới hạn nghiên cứu

- Hệ thống mới được triển khai ở mức nguyên mẫu và chưa được đánh giá trong môi trường có mật độ người di chuyển cao.
- Bản đồ sử dụng trong giai đoạn vận hành là bản đồ hai chiều tương đối tĩnh. Những thay đổi lớn về bố trí phòng có thể yêu cầu cập nhật hoặc xây dựng lại bản đồ.

- Các linh kiện được lựa chọn chủ yếu phục vụ nghiên cứu và có chi phí phù hợp, do đó độ bền, độ chính xác và mức độ an toàn chưa tương đương các nền tảng robot công nghiệp.
- Hệ thống hội thoại phụ thuộc vào chất lượng microphone, tiếng ồn môi trường, kết nối mạng và tài nguyên của máy chủ xử lý. Vì vậy, thời gian phản hồi và độ chính xác nhận dạng tiếng nói có thể thay đổi giữa các điều kiện thử nghiệm.
- Nội dung trả lời của mô hình ngôn ngữ được giới hạn bởi ngữ cảnh do quản trị viên cấu hình. Hệ thống chưa bảo đảm loại bỏ hoàn toàn mọi câu trả lời không chính xác và chưa được đánh giá bằng một nghiên cứu người dùng quy mô lớn.
- Chức năng dừng từ giao diện Web là cơ chế dừng bằng phần mềm và phụ thuộc vào kết nối giữa giao diện, máy chủ và robot; chưa thay thế cho nút dừng khẩn cấp phần cứng trong các hệ thống thương mại.
- Đề tài chưa đánh giá đầy đủ thời lượng pin, độ bền vận hành dài hạn.

1.8 BỐ CỤC ĐỀ TÀI

Báo cáo được tổ chức thành năm chương như sau:

- **Chương 1: Tổng quan.** Trình bày bối cảnh nghiên cứu, các công trình liên quan, lý do chọn đề tài, mục tiêu, nhiệm vụ, phương pháp nghiên cứu, các kết quả đóng góp dự kiến, phạm vi và giới hạn của hệ thống.
- **Chương 2: Cơ sở lý thuyết.** Trình bày cơ sở phần cứng, động học robot, nền tảng phần mềm điều hướng, truyền thông thời gian thực, xử lý tiếng nói, mô hình AI hội thoại và giao diện quản trị robot.
- **Chương 3: Thiết kế hệ thống.** Trình bày yêu cầu thiết kế, kiến trúc tổng thể, sơ đồ nguyên lý, thiết kế từng khối phần cứng và phần mềm và các lưu đồ hoạt động.
- **Chương 4: Thi công và đánh giá hệ thống.** Trình bày quá trình lắp ráp, tích hợp và kết quả kiểm thử đối với các chức năng xây dựng bản đồ, điều hướng, vận hành tour và tương tác giọng nói theo ngữ cảnh.
- **Chương 5: Kết luận và hướng phát triển.** Tổng kết kết quả đạt được, phân tích hạn chế và đề xuất các hướng cải tiến.

Chương 2

CƠ SỞ LÝ THUYẾT

2.1 TỔNG QUAN VỀ PHẦN CỨNG

2.1.1 Giới thiệu về LiDAR

Cảm biến LiDAR RPLIDAR A1M8-R6 của SLAMTEC [1] là một thiết bị quét laser 360° sử dụng phương pháp đo triangulation để thu thập dữ liệu khoảng cách trong môi trường xung quanh. Cảm biến có khả năng quét toàn vòng với phạm vi từ $0,15m$ đến $12m$, tốc độ lấy mẫu lên đến $8kHz$ và tần số quét từ $5-10Hz$, đáp ứng tốt các yêu cầu trong các ứng dụng robot tự hành.

Với giao tiếp UART ở tốc độ $115200bps$, RPLIDAR A1M8-R6 có thể kết nối với vi điều khiển hoặc máy tính nhúng để phục vụ các bài toán như xây dựng bản đồ, định vị và phát hiện vật cản. Thiết bị hoạt động ổn định trong môi trường trong nhà, có độ chính xác tương đối cao và kích thước nhỏ gọn, phù hợp cho các hệ thống robot di động cũng như các ứng dụng nghiên cứu, phát triển trong lĩnh vực robot và tự động hóa.

Hình ảnh thực tế của LiDAR RPLIDAR A1M8-R6 được trình bày ở Hình 2.1.1 và thông số kỹ thuật cơ bản được trình bày trong Bảng 2.1.1.



Hình 2.1.1: Hình ảnh thực tế của LiDAR RPLIDAR A1M8-R6 [1]

Bảng 2.1.1: Thông số kỹ thuật của LiDAR RPLIDAR A1M8-R6

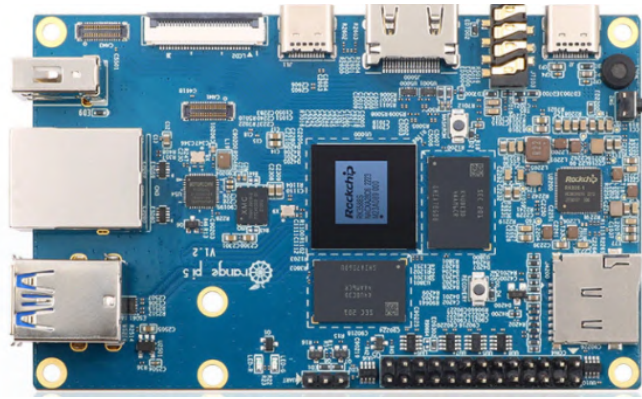
Thông số kỹ thuật	Giá trị
Góc quét	360°
Phạm vi đo	0,15 m – 12 m
Tốc độ lấy mẫu	Khoảng 8000 mẫu/giây
Tần số quét	5 – 10 Hz
Độ chính xác	Khoảng 1–2,5%
Giao tiếp	UART TTL 3,3V, 115200 bps; USB
Kích thước	Khoảng 96 × 70 × 55 mm
Điện áp hoạt động	5 V
Dòng điện	0,4 A

2.1.2 Giới thiệu về ORANGE PI 5

Orange Pi 5 [2] là một máy tính nhúng dạng đơn bo mạch (Single Board Computer – SBC) do hãng Orange Pi phát triển, sử dụng vi xử lý Rockchip RK3588S với kiến trúc 8 nhân 64-bit. Bộ xử lý này bao gồm 4 nhân Cortex-A76 hiệu năng cao và 4 nhân Cortex-A55 tiết kiệm năng lượng, cho phép thiết bị đáp ứng tốt các tác vụ tính toán trong hệ thống robot di động như xử lý dữ liệu cảm biến, truyền thông, điều phối phần mềm và chạy các node ROS 2.

Bên cạnh CPU, Orange Pi 5 còn được tích hợp GPU ARM Mali-G610 và bộ xử lý NPU có hiệu năng lên đến khoảng 6 TOPS, hỗ trợ thực thi một số tác vụ trí tuệ nhân tạo và xử lý hình ảnh ở mức nhúng. Bo mạch hỗ trợ nhiều giao tiếp ngoại vi như USB, HDMI, Ethernet, GPIO và khe M.2 PCIe, giúp mở rộng khả năng kết nối với cảm biến, bộ điều khiển ngoại vi và thiết bị lưu trữ. Nhờ khả năng chạy các hệ điều hành Linux như Ubuntu hoặc Debian, Orange Pi 5 phù hợp để đóng vai trò máy tính trung tâm trong các hệ thống robot sử dụng ROS 2. Trong kiến trúc của đề tài, Orange Pi 5 đảm nhiệm tầng xử lý cao gồm ROS 2, định vị, điều hướng, giao tiếp mạng và kết nối dịch vụ giọng nói; các tác vụ điều khiển thời gian thực ở tầng động cơ được tách sang STM32F103. Cách phân tầng này giúp tránh phụ thuộc vào độ trễ không xác định của hệ điều hành Linux đối với vòng điều khiển vận tốc bánh xe.

Hình ảnh thực tế của Orange Pi 5 được trình bày ở Hình 2.1.2 và thông số kỹ thuật cơ bản được trình bày trong Bảng 2.1.2.



Hình 2.1.2: Hình ảnh thực tế của Orange Pi 5 [2]

Bảng 2.1.2: Thông số kỹ thuật của Orange Pi 5

Thông số kỹ thuật	Giá trị
Vi xử lý	Rockchip RK3588S, 8 nhân 64-bit
CPU	4× Cortex-A76 + 4× Cortex-A55
Xung nhịp tối đa	Khoảng 2,4 GHz
GPU	ARM Mali-G610
NPU	Khoảng 6 TOPS
RAM	8 GB
Bộ nhớ lưu trữ	microSD, hỗ trợ mở rộng qua M.2 PCIe
Giao tiếp	USB, Ethernet, HDMI, GPIO, M.2 PCIe
Nguồn cấp	5 V
Hệ điều hành hỗ trợ	Linux, Android, Orange Pi OS

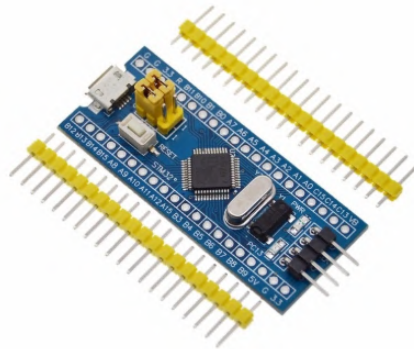
2.1.3 Giới thiệu về vi điều khiển STM32F103

STM32F103 là dòng vi điều khiển 32-bit của STMicroelectronics, sử dụng lõi ARM Cortex-M3 [3]. Vi điều khiển này hoạt động ở tần số tối đa 72 MHz, tích hợp bộ nhớ Flash và SRAM, đồng thời hỗ trợ nhiều ngoại vi phổ biến như GPIO, ADC, timer, PWM, USART, SPI, I2C, CAN và USB. Với khả năng xử lý thời gian thực và tiêu thụ

năng lượng thấp, STM32F103 thường được sử dụng trong các hệ thống nhúng, điều khiển động cơ, robot di động và các ứng dụng tự động hóa.

Trong hệ thống robot di động, STM32F103 có thể đảm nhiệm vai trò điều khiển tầng thấp, bao gồm đọc tín hiệu encoder, tạo xung PWM điều khiển động cơ, xử lý tín hiệu từ cảm biến và giao tiếp với máy tính nhúng thông qua UART hoặc các chuẩn truyền thông khác. Việc tách riêng tầng điều khiển thời gian thực sang vi điều khiển giúp giảm tải cho máy tính nhúng và tăng độ ổn định trong quá trình điều khiển phần cứng.

Hình ảnh thực tế của STM32F103 được trình bày ở Hình 2.1.3 và thông số kỹ thuật cơ bản được trình bày trong Bảng 2.1.3.



Hình 2.1.3: Hình ảnh thực tế của vi điều khiển STM32F103 [3]

Bảng 2.1.3: Thông số kỹ thuật của vi điều khiển STM32F103

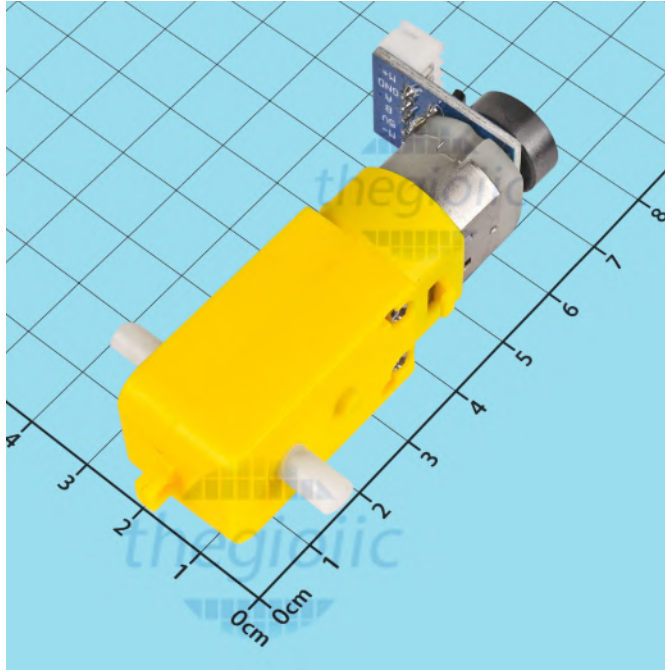
Thông số kỹ thuật	Giá trị
Kiến trúc CPU	ARM Cortex-M3, 32-bit
Tần số hoạt động tối đa	72MHz
Bộ nhớ Flash	Lên đến 128KB
SRAM	Khoảng 20KB
Điện áp hoạt động của vi điều khiển	Khoảng 2,0V – 3,6V
Giao tiếp ngoại vi	USART, SPI, I2C, CAN, USB
Timer	Timer 16-bit, hỗ trợ PWM và encoder interface
Ứng dụng phổ biến	Điều khiển động cơ, robot, IoT, hệ thống nhúng

2.1.4 Động cơ giảm tốc TT có Encoder

Động cơ giảm tốc TT có encoder 2 pha AB [4] là loại động cơ DC có chổi than, được tích hợp hộp số giảm tốc và cảm biến encoder để phản hồi chuyển động quay. Hộp số giúp giảm tốc độ quay ở trục đầu ra và tăng mô-men xoắn, phù hợp cho các mô hình robot di động cỡ nhỏ. Encoder tích hợp trên động cơ tạo ra hai tín hiệu xung vuông A và B lệch pha nhau 90° , cho phép xác định cả tốc độ và chiều quay của trục động cơ.

Nhờ có tín hiệu phản hồi từ encoder, động cơ có thể được sử dụng trong các hệ thống điều khiển vòng kín, chẳng hạn như điều khiển vận tốc bằng thuật toán PID. Trong robot bánh vi sai, hai động cơ encoder được dùng để điều khiển độc lập bánh trái và bánh phải, đồng thời cung cấp dữ liệu phản hồi phục vụ tính toán odometry. Với điện áp hoạt động thấp, kích thước nhỏ gọn và khả năng lắp ghép dễ dàng, động cơ TT encoder là lựa chọn phổ biến trong các mô hình robot giáo dục, robot tránh vật cản và robot tự hành quy mô nhỏ.

Hình ảnh thực tế của động cơ giảm tốc TT có encoder được trình bày ở Hình 2.1.4 và thông số kỹ thuật cơ bản được trình bày trong Bảng 2.1.4.



Hình 2.1.4: Hình ảnh thực tế của động cơ giảm tốc TT có encoder 2 pha AB [4]

Bảng 2.1.4: Thông số kỹ thuật của động cơ giảm tốc TT có encoder

Thông số kỹ thuật	Giá trị
Loại động cơ	DC có chổi than, tích hợp hộp số giảm tốc
Encoder	Hall encoder 2 pha A/B
Điện áp hoạt động	2 – 8VDC
Điện áp định mức	6V
Dòng định mức	Nhỏ hơn 200mA
Tỷ số truyền	1 : 120
Tốc độ đầu ra	Khoảng 166RPM
Độ phân giải encoder	Khoảng 1560 count/vòng trục ra theo thông số module; cần thống nhất theo chế độ giải mã thực tế
Điện áp encoder	3,3 – 5V

2.1.5 Mạch điều khiển động cơ L298N

Module điều khiển động cơ L298N [5] là mạch điều khiển công suất sử dụng IC cầu H kép L298N, cho phép điều khiển độc lập hai động cơ DC hoặc một động cơ bước hai pha. Cấu trúc cầu H cho phép thay đổi chiều dòng điện qua động cơ, từ đó điều khiển được chiều quay của động cơ. Ngoài ra, tốc độ động cơ có thể được điều chỉnh bằng tín hiệu PWM cấp vào các chân điều khiển cho phép của module.

Trong các hệ thống robot di động, L298N thường được sử dụng làm tầng công suất trung gian giữa vi điều khiển và động cơ DC. Vi điều khiển xuất tín hiệu logic để quy định chiều quay và tín hiệu PWM để điều chỉnh tốc độ, trong khi L298N cung cấp dòng điện cần thiết cho động cơ. Module này có ưu điểm là dễ sử dụng, chi phí thấp và phù hợp cho các mô hình robot công suất nhỏ. Tuy nhiên, L298N sử dụng tầng công suất transistor lưỡng cực nên tồn tại sụt áp trên cầu H và tổn hao nhiệt đáng kể. Giá trị dòng tối đa trong datasheet không nên được hiểu là dòng liên tục có thể duy trì trong mọi điều kiện; khả năng làm việc thực tế phụ thuộc điện áp nguồn, tản nhiệt, chu kỳ PWM và dòng tải của động cơ [5]. Do đó, mạch phù hợp với nguyên mẫu vận tốc thấp của đề tài nhưng không phải lựa chọn tối ưu cho hệ thống công suất cao hoặc yêu cầu hiệu suất năng lượng lớn.

Hình ảnh thực tế của mạch điều khiển động cơ L298N được trình bày ở Hình 2.1.5 và thông số kỹ thuật cơ bản được trình bày trong Bảng 2.1.5.



Hình 2.1.5: Hình ảnh thực tế của mạch điều khiển động cơ L298N [5]

Bảng 2.1.5: Thông số kỹ thuật của mạch điều khiển động cơ L298N

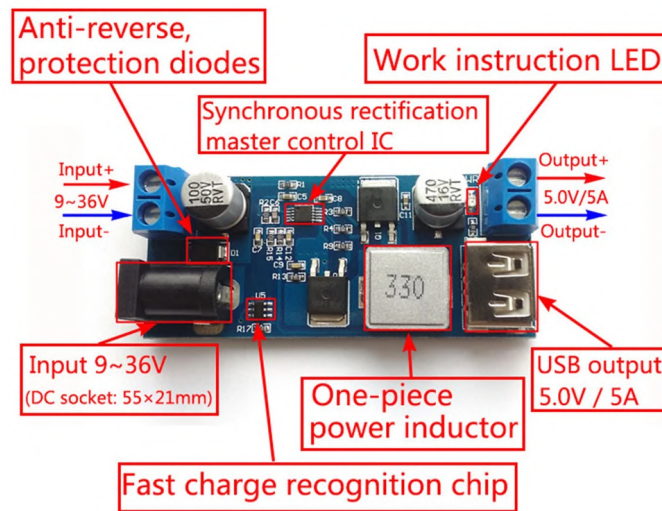
Thông số kỹ thuật	Giá trị
IC điều khiển	L298N Dual H-Bridge
Số kênh điều khiển	2 kênh động cơ DC
Điện áp cấp cho động cơ	5 – 35VDC
Điện áp điều khiển logic	3 – 5,5V
Dòng điều khiển tối đa	Khoảng 2A mỗi kênh
Phương pháp điều khiển tốc độ	PWM
Phương pháp điều khiển chiều quay	Tín hiệu logic điều khiển cầu H
Kích thước module	Khoảng 50 × 45 × 25 mm

2.1.6 Giới thiệu về mạch giảm áp buck DC-DC XY-3606

Mạch giảm áp Buck DC-DC XY-3606 5V 5A [6] là module nguồn chuyển đổi điện áp một chiều, có nhiệm vụ hạ điện áp đầu vào từ mức cao hơn xuống mức điện áp đầu ra ổn định 5V. Module này phù hợp cho các hệ thống nhúng và robot di động sử dụng nguồn pin có điện áp danh định lớn hơn điện áp hoạt động của các vi mạch xử lý.

Trong hệ thống robot, mạch Buck giúp tạo ra nguồn 5V ổn định để cấp cho máy tính nhúng, cảm biến và các module điều khiển. Việc sử dụng mạch giảm áp riêng giúp hạn chế hiện tượng sụt áp khi động cơ khởi động hoặc thay đổi tải đột ngột. Ngoài ra, việc tách nguồn xử lý và nguồn công suất cũng góp phần giảm nhiễu điện, giúp hệ thống hoạt động ổn định hơn trong quá trình vận hành.

Hình ảnh thực tế của mạch giảm áp Buck DC-DC XY-3606 được trình bày ở Hình 2.1.6 và thông số kỹ thuật cơ bản được trình bày trong Bảng 2.1.6.



Hình 2.1.6: Hình ảnh thực tế của mạch giảm áp Buck DC-DC XY-3606 5V 5A [6]

Bảng 2.1.6: Thông số kỹ thuật của mạch giảm áp Buck DC-DC XY-3606 5V 5A

Thông số kỹ thuật	Giá trị
Loại mạch	Bộ chuyển đổi DC-DC giảm áp
Điện áp đầu vào	9 – 36VDC
Điện áp đầu ra	5VDC
Dòng điện đầu ra tối đa	5A
Công suất đầu ra tối đa	Khoảng 30W
Chuẩn đầu ra	Cổng USB
Ứng dụng	Cấp nguồn ổn định cho hệ thống nhúng và robot di động

2.2 ĐỘNG HỌC ROBOT DI ĐỘNG BÁNH VI SAI VÀ ĐIỀU KHIỂN VẬN TỐC

Động học robot di động là phần lý thuyết mô tả quan hệ hình học giữa chuyển động của các cơ cấu truyền động và chuyển động của thân robot, chưa xét đến lực, mô-men, khối lượng hay ma sát động lực học. Đối với robot di động, mô hình động học đóng vai

trò nền tảng để xác định robot có thể di chuyển như thế nào trong không gian, cách tính vận tốc thân robot từ vận tốc bánh xe, và cách suy ra vận tốc bánh xe cần thiết từ lệnh điều khiển mong muốn. Theo Siegwart và Nourbakhsh, việc hiểu hành vi cơ học của robot là cần thiết cả trong thiết kế cơ cấu robot lẫn trong xây dựng phần mềm điều khiển cho nền tảng phần cứng cụ thể [12].

2.2.1 Mô hình hình học và Ràng buộc động học toán học

Robot bánh vi sai, hay *differential-drive robot*, là một cấu hình robot di động phổ biến, gồm hai bánh xe chủ động được bố trí trên cùng một trục và có thể được điều khiển độc lập. Khi hai bánh quay cùng chiều với cùng vận tốc, robot chuyển động thẳng. Khi hai bánh quay ngược chiều với cùng độ lớn vận tốc, robot quay tại chỗ quanh trung điểm của trục bánh. Khi hai bánh quay với vận tốc khác nhau, robot chuyển động theo một cung tròn [13, 14].

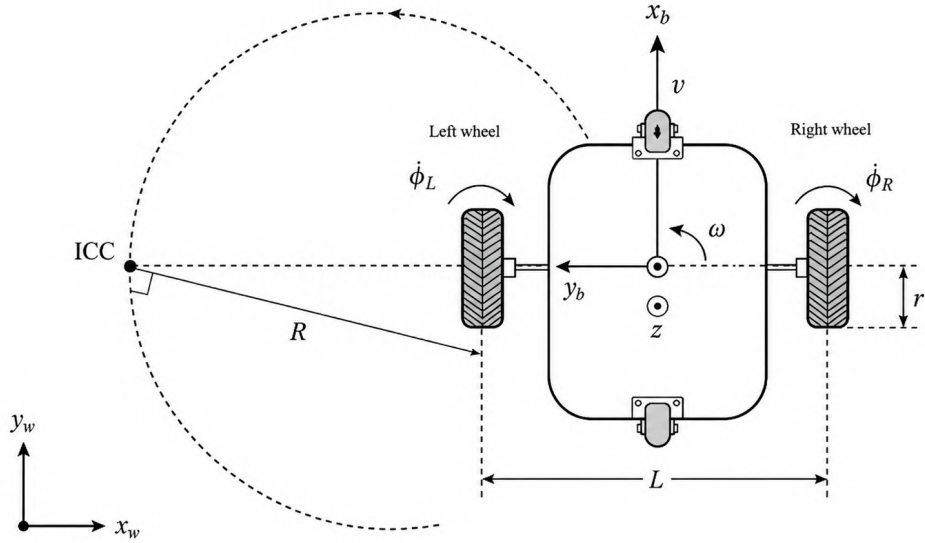
Theo tài liệu chính thức của `ros2_control`, robot bánh vi sai là một dạng robot tương đương về mặt động học với mô hình *unicycle* nhưng ổn định hơn về mặt cơ khí vì sử dụng hai bánh chủ động thay vì một bánh duy nhất. Trong mô hình này, hai biến điều khiển quan trọng của thân robot là vận tốc tuyến tính theo trục x_b , ký hiệu $v_{b,x}$, và vận tốc góc quanh trục z , ký hiệu $\omega_{b,z}$. Đây cũng là hai thành phần vận tốc thường được dùng trong lệnh điều khiển di chuyển của robot di động trong ROS [15].

Trong trường hợp robot hoạt động trên mặt phẳng hai chiều, tư thế của robot thường được biểu diễn bởi vector trạng thái:

$$\mathbf{q} = \begin{bmatrix} x & y & \theta \end{bmatrix}^T, \quad (2.1)$$

trong đó x và y là tọa độ của robot trong hệ tọa độ toàn cục, còn θ là góc hướng của robot. Đối với quy ước tọa độ trong ROS, hệ tọa độ gắn với thân robot thường tuân theo REP 103: trục x hướng về phía trước, trục y hướng sang trái, và trục z hướng lên trên [16]. Quy ước này giúp các mô hình động học, phép biến đổi tọa độ và dữ liệu điều hướng trong ROS thống nhất với nhau.

Trong Hình 2.2.1, robot bánh vi sai được minh họa theo góc nhìn từ trên xuống. Gốc hệ tọa độ thân robot được đặt tại trung điểm của trục nối hai bánh chủ động. Trục x_b biểu diễn hướng tiến của robot, trục y_b hướng sang trái, r là bán kính bánh xe và L là khoảng cách giữa hai bánh chủ động. Hình này là cơ sở trực quan để hiểu các phương trình động học thuận và động học ngược được trình bày ở các phần tiếp theo [14, 16].



Hình 2.2.1: Mô hình động học robot bánh vi sai và tâm quay tức thời ICC.

Robot bánh vi sai thuộc nhóm robot *non-holonomic*, nghĩa là robot không thể tạo vận tốc tức thời theo mọi hướng trong mặt phẳng. Cụ thể, do bánh xe thông thường chỉ lăn theo phương tiếp tuyến của bánh và không trượt ngang lý tưởng, robot không thể di chuyển tức thời theo phương ngang của thân robot, tức phương y_b . Vì vậy, robot muốn đi đến một điểm nằm lệch bên phải hoặc bên trái thì phải kết hợp quay hướng và tiến/lùi, thay vì trượt ngang trực tiếp như robot omnidirectional [15, 14].

Ràng buộc non-holonomic này không phải là “điểm kỳ dị” của robot, mà là một ràng buộc vận tốc sinh ra từ cơ cấu bánh xe và điều kiện lăn không trượt. Trong lý thuyết robot di động, các ràng buộc như vậy thường được mô tả ở mức vận tốc, chứ không đơn thuần là ràng buộc trực tiếp lên vị trí. Modern Robotics mô tả robot bánh xe non-holonomic như các nền tảng sử dụng bánh xe thông thường, không trượt ngang, dẫn đến ràng buộc vận tốc không thể tích phân thành một ràng buộc cấu hình tương đương [17].

2.2.2 Hệ phương trình động học thuận và ngược

Gọi r là bán kính bánh xe, L là khoảng cách giữa hai bánh chủ động, $\dot{\phi}_L$ là vận tốc góc bánh trái và $\dot{\phi}_R$ là vận tốc góc bánh phải. Khi bánh xe lăn không trượt, vận tốc dài tại mỗi bánh được tính từ vận tốc góc của bánh theo quan hệ:

$$v_L = r\dot{\phi}_L, \quad (2.2)$$

$$v_R = r\dot{\phi}_R, \quad (2.3)$$

trong đó v_L và v_R lần lượt là vận tốc dài của bánh trái và bánh phải tại điểm tiếp xúc với mặt sàn [14].

Từ hai vận tốc bánh xe, vận tốc tuyến tính của robot tại trung điểm trục bánh được tính bằng trung bình cộng của hai vận tốc bánh:

$$v = \frac{v_R + v_L}{2}. \quad (2.4)$$

Thay $v_R = r\dot{\phi}_R$ and $v_L = r\dot{\phi}_L$, ta có:

$$v = \frac{r}{2} (\dot{\phi}_R + \dot{\phi}_L). \quad (2.5)$$

Vận tốc góc của robot quanh trục z được xác định bởi độ chênh lệch vận tốc giữa hai bánh chia cho khoảng cách hai bánh:

$$\omega = \frac{v_R - v_L}{L}, \quad (2.6)$$

hay:

$$\omega = \frac{r}{L} (\dot{\phi}_R - \dot{\phi}_L). \quad (2.7)$$

Hai phương trình trên tạo thành mô hình động học thuận của robot bánh vi sai, cho phép tính vận tốc thân robot từ vận tốc quay của hai bánh [14, 15].

Khi biểu diễn trong hệ tọa độ toàn cục, vận tốc của robot phụ thuộc vào góc hướng hiện tại θ . Nếu robot chuyển động với vận tốc tuyến tính v theo hướng trục x_b của thân robot, phương trình động học trong hệ tọa độ thế giới được viết như sau:

$$\dot{x} = v \cos \theta, \quad (2.8)$$

$$\dot{y} = v \sin \theta, \quad (2.9)$$

$$\dot{\theta} = \omega. \quad (2.10)$$

Dạng phương trình này cho thấy vận tốc tuyến tính của robot luôn nằm theo hướng mà robot đang quay mặt tới, trong khi vận tốc góc quyết định tốc độ thay đổi hướng của robot [14, 15].

Bài toán động học ngược của robot bánh vi sai là xác định vận tốc quay cần thiết của bánh trái và bánh phải để robot đạt được vận tốc tuyến tính v và vận tốc góc ω mong muốn. Từ hệ phương trình động học thuận, ta suy ra:

$$\dot{\phi}_L = \frac{v - \frac{L}{2}\omega}{r}, \quad (2.11)$$

$$\dot{\phi}_R = \frac{v + \frac{L}{2}\omega}{r}. \quad (2.12)$$

Trong đó ϕ_L và ϕ_R là vận tốc góc cần cấp cho bánh trái và bánh phải. Nếu $\omega = 0$, hai bánh quay cùng tốc độ và robot đi thẳng. Nếu $v = 0$, hai bánh quay ngược chiều với cùng độ lớn và robot quay tại chỗ [14].

Trong ROS 2, ý tưởng này được hiện thực hóa trong `diff_drive_controller`: bộ điều khiển nhận lệnh vận tốc thân robot, sau đó chuyển đổi thành lệnh vận tốc cho hai bánh của nền tảng vi sai. Tài liệu chính thức của `ros2_control` cũng nêu rằng controller nhận lệnh vận tốc của thân robot, chuyển sang lệnh bánh xe, đồng thời tính toán và xuất bản odometry từ phản hồi phần cứng. Vì vậy, mô hình động học ngược không chỉ là công thức lý thuyết mà còn là cơ sở trực tiếp cho tầng điều khiển vận tốc bánh trong các robot di động dùng ROS 2 [18].

Khi hai bánh của robot bánh vi sai có vận tốc khác nhau, robot không chuyển động thẳng mà quay quanh một điểm gọi là *tâm cong tức thời* hoặc *Instantaneous Center of Curvature* – ICC. Tại một thời điểm tức thời, chuyển động của robot có thể xem như chuyển động quay quanh điểm ICC nằm trên đường thẳng trùng với trục nối hai bánh. Vị trí của ICC phụ thuộc vào tỉ lệ giữa vận tốc hai bánh. Khi hai bánh quay cùng vận tốc, ICC nằm ở vô cực và robot đi thẳng. Khi hai bánh quay ngược chiều cùng độ lớn, ICC trùng với trung điểm của trục bánh và robot quay tại chỗ [13].

Với R_{ICC} là khoảng cách từ trung điểm trục bánh xe đến ICC, vận tốc góc ω có thể được mô tả thông qua chênh lệch vận tốc giữa hai bánh. Nếu ký hiệu L là khoảng cách giữa hai bánh, v_L và v_R là vận tốc dài của bánh trái và bánh phải, ta có:

$$\omega = \frac{v_R - v_L}{L}, \quad (2.13)$$

$$R_{ICC} = \frac{L}{2} \frac{v_R + v_L}{v_R - v_L}. \quad (2.14)$$

Các phương trình này giúp giải thích vì sao việc thay đổi tương đối vận tốc hai bánh có thể tạo ra các quỹ đạo cong khác nhau cho robot. Nội dung này được minh họa cụ thể trong Hình 2.2.1 bằng cách vẽ thêm điểm ICC và cung tròn quỹ đạo của robot [13].

2.2.3 Mô hình tính toán định vị động học (Odometry)

Odometry là quá trình ước lượng sự thay đổi tư thế của robot bằng cách tích phân chuyển động theo thời gian. Đối với robot bánh vi sai, nếu biết vận tốc quay của hai bánh từ encoder và các tham số hình học r , L , ta có thể dùng phương trình động học thuận để tính v và ω , sau đó tích phân theo thời gian để cập nhật x , y , θ . Tài liệu `ros2_control` gọi quá trình tích phân mô hình động học từ encoder là *odometric localization*, *passive localization* hoặc *dead reckoning* [15].

Tuy nhiên, odometry chỉ là ước lượng tương đối và dễ tích lũy sai số theo thời gian do trượt bánh, sai số bán kính bánh, sai số khoảng cách hai bánh, độ rơ cơ khí và nhiễu đo từ encoder. Vì vậy, trong các hệ thống robot tự hành, odometry thường được kết hợp với các phương pháp định vị khác như IMU, LiDAR SLAM hoặc AMCL để giảm sai số tích lũy. Đây là lý do động học bánh vi sai được xem là nền tảng ban đầu cho điều khiển và định vị, nhưng không đủ để đảm bảo định vị toàn cục chính xác trong thời gian dài [12].

2.2.4 Phương pháp đo lường và tính toán vận tốc từ Encoder

Encoder gia tăng là cảm biến được dùng phổ biến để đo chuyển động quay của trục động cơ. Khác với encoder tuyệt đối, encoder gia tăng không trực tiếp cung cấp góc tuyệt đối tại mọi thời điểm, mà tạo ra các xung khi trục quay. Số lượng xung được đếm theo thời gian cho phép suy ra sự thay đổi vị trí góc, chiều quay và vận tốc quay của trục. Trong robot di động, encoder thường được gắn vào trục động cơ hoặc trục bánh xe để đo phản hồi cho vòng điều khiển vận tốc và tính toán odometry [19, 18].

Đối với encoder quadrature, cảm biến tạo ra hai kênh tín hiệu vuông, thường gọi là kênh A và kênh B. Hai tín hiệu này lệch pha nhau 90° điện. Tần số của hai tín hiệu liên quan đến tốc độ quay, còn quan hệ pha giữa kênh A và kênh B cho phép xác định chiều quay. Nếu kênh A dẫn pha kênh B, bộ đếm có thể tăng; nếu kênh B dẫn pha kênh A, bộ đếm có thể giảm, tùy quy ước của từng hệ thống [19, 20]. Một số encoder còn có thêm kênh Z hoặc *index channel*. Kênh này thường phát ra một xung duy nhất trong mỗi vòng quay và có thể được dùng như mốc tham chiếu để kiểm tra vị trí hoặc xác định một điểm chuẩn cơ khí. Tuy nhiên, trong bài toán odometry của robot di động, tín hiệu quan trọng nhất thường vẫn là hai kênh A/B để đo biến thiên vị trí và chiều quay liên tục [19].

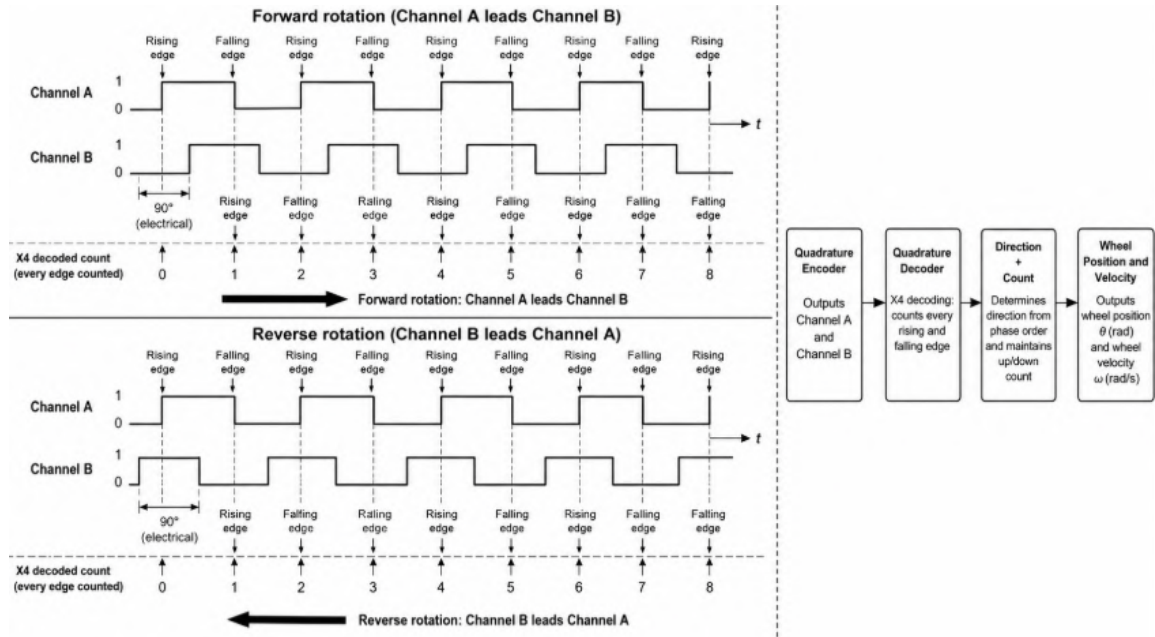
Tùy cách đếm cạnh xung, tín hiệu quadrature có thể được giải mã theo các chế độ X1, X2 hoặc X4. Ở chế độ X1, bộ đếm chỉ tăng hoặc giảm tại một cạnh nhất định của một kênh. Ở chế độ X2, bộ đếm sử dụng cả sườn lên và sườn xuống của một kênh. Ở chế độ X4, bộ đếm sử dụng cả sườn lên và sườn xuống của cả hai kênh A và B. Vì vậy, chế độ X4 tạo ra bốn lần số đếm trên mỗi chu kỳ quadrature, giúp tăng độ phân giải đo vị trí so với cách đếm một cạnh đơn lẻ [20].

Nếu encoder có độ phân giải cơ bản là *PPR*, tức số xung trên mỗi vòng quay ở một kênh, thì độ phân giải đếm hiệu dụng khi dùng X4 có thể viết gần đúng là:

$$CPR = 4 \cdot PPR. \quad (2.15)$$

Trong đó *CPR* là số count trên mỗi vòng quay sau giải mã. Giá trị *CPR* càng lớn thì độ phân giải đo vị trí càng cao, nhưng bộ xử lý cũng phải xử lý nhiều cạnh xung hơn

trong cùng một khoảng thời gian, đặc biệt ở tốc độ cao [20, 21]. Về mặt triển khai, bộ giải mã quadrature thường hoạt động như một bộ đếm vị trí có hướng. Tín hiệu A/B được chuyển thành thông tin chiều quay và xung đếm. Khi trục quay theo một chiều, bộ đếm tăng; khi trục quay theo chiều ngược lại, bộ đếm giảm như minh họa ở hình 2.2.2. Một số bộ giải mã còn kiểm tra các chuyển trạng thái không hợp lệ, ví dụ hai kênh thay đổi đồng thời, vì đây có thể là dấu hiệu của nhiễu hoặc lỗi lấy mẫu [19].



Hình 2.2.2: Tín hiệu quadrature encoder và nguyên lý giải mã X4.

Sau khi giải mã encoder, ta thu được số count tích lũy của bánh xe theo thời gian. Gọi N_k là số count tại thời điểm lấy mẫu k , N_{k-1} là số count tại thời điểm trước đó, và T_s là chu kỳ lấy mẫu. Số count thay đổi trong một chu kỳ là:

$$\Delta N_k = N_k - N_{k-1}. \quad (2.16)$$

Nếu CPR là số count trên mỗi vòng quay của bánh hoặc trục đang đo, vận tốc góc có thể tính bằng:

$$\omega_k = \frac{2\pi \Delta N_k}{CPR \cdot T_s}. \quad (2.17)$$

Trong đó ω_k có đơn vị rad/s. Nếu encoder gắn trên trục động cơ trước hộp số, cần tính thêm tỷ số truyền G . Khi đó, số count tương ứng với một vòng quay của bánh xe sẽ là $CPR \cdot G$, và vận tốc góc bánh xe được tính bởi:

$$\omega_{wheel,k} = \frac{2\pi \Delta N_k}{CPR \cdot G \cdot T_s}. \quad (2.18)$$

Từ vận tốc góc bánh xe, vận tốc dài của bánh được tính bằng:

$$v_{wheel,k} = r\omega_{wheel,k}. \quad (2.19)$$

Trong đó r là bán kính bánh xe. Vận tốc dài này được đưa trở lại mô hình động học thuận của robot bánh vi sai để tính vận tốc thân robot và cập nhật odometry. Vì vậy, encoder không chỉ phục vụ vòng điều khiển vận tốc cục bộ của từng bánh, mà còn là nguồn dữ liệu quan trọng cho quá trình ước lượng chuyển động của toàn robot [18].

Tuy nhiên, vận tốc tính từ encoder luôn chịu ảnh hưởng bởi độ phân giải, chu kỳ lấy mẫu và nhiễu cơ khí. Nếu T_s quá nhỏ hoặc encoder có độ phân giải thấp, số count thay đổi trong một chu kỳ có thể rất ít, làm vận tốc ước lượng bị giật theo bậc thang. Nếu T_s quá lớn, tín hiệu vận tốc mượt hơn nhưng độ trễ phản hồi tăng lên. Do đó, trong điều khiển động cơ, lựa chọn chu kỳ lấy mẫu và phương pháp lọc vận tốc là một bước quan trọng để cân bằng giữa độ mượt, độ trễ và độ chính xác [22, 21].

2.2.5 Thuật toán điều khiển PID rời rạc cho động cơ DC

PID, viết tắt của *Proportional–Integral–Derivative*, là bộ điều khiển phản hồi vòng kín được sử dụng phổ biến trong điều khiển động cơ. Bộ điều khiển này tạo tín hiệu điều khiển bằng cách kết hợp ba thành phần: tỷ lệ P, tích phân I và vi phân D. Thành phần P phản ứng với sai số hiện tại, thành phần I tích lũy sai số theo thời gian để giảm sai số xác lập, còn thành phần D phản ứng với tốc độ thay đổi của sai số nhằm cải thiện đáp ứng quá độ và giảm dao động [23].

Dạng liên tục cơ bản của bộ điều khiển PID được viết như sau:

$$u(t) = K_P e(t) + K_I \int_0^t e(\tau) d\tau + K_D \frac{de(t)}{dt}. \quad (2.20)$$

Trong đó $u(t)$ là tín hiệu điều khiển, $e(t)$ là sai số giữa giá trị đặt và giá trị đo, K_P , K_I , K_D lần lượt là các hệ số tỷ lệ, tích phân và vi phân. Đối với điều khiển vận tốc bánh xe, $u(t)$ thường là tín hiệu PWM gửi xuống driver động cơ, còn sai số được xác định từ vận tốc đặt và vận tốc đo từ encoder [23, 24].

Do bộ điều khiển được triển khai trên vi điều khiển hoặc máy tính nhúng, PID cần được rời rạc hóa theo chu kỳ lấy mẫu T_s . Sai số tại bước lấy mẫu thứ k được xác định bởi:

$$e_k = \omega_{ref,k} - \omega_{measured,k}. \quad (2.21)$$

Khi đó, dạng PID rời rạc có thể được biểu diễn như sau:

$$u_k = K_P e_k + K_I T_s \sum_{i=0}^k e_i + K_D \frac{e_k - e_{k-1}}{T_s}. \quad (2.22)$$

Trong hệ điều khiển số, PID cũng có thể được triển khai theo dạng gia tăng:

$$u_k = u_{k-1} + \Delta u_k. \quad (2.23)$$

Dạng này phù hợp với điều khiển nhúng vì tín hiệu điều khiển được cập nhật dựa trên thay đổi so với bước trước, giúp hạn chế sự thay đổi đột ngột ở đầu ra [23, 25].

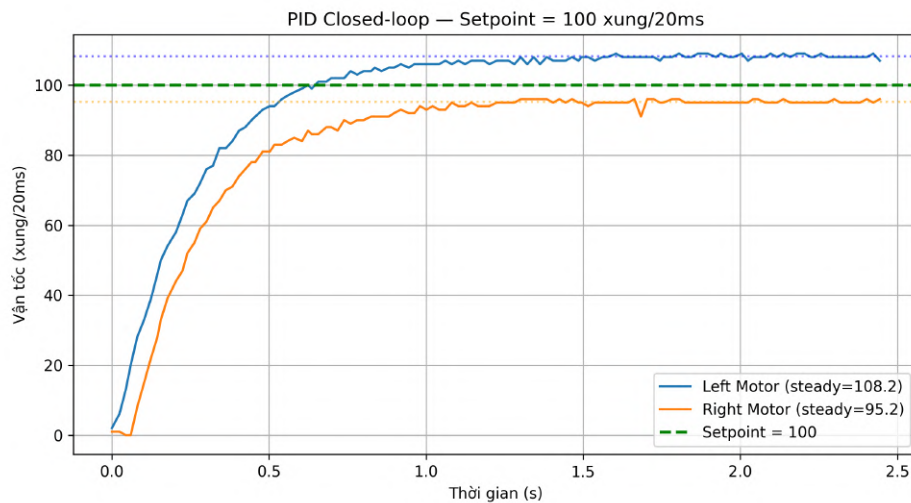
Trong thực tế, tín hiệu điều khiển luôn bị giới hạn bởi phần cứng, chẳng hạn như giới hạn PWM, điện áp nguồn và khả năng dòng của driver động cơ. Khi tín hiệu điều khiển vượt quá giới hạn vật lý, đầu ra bị bão hòa. Nếu thành phần tích phân vẫn tiếp tục cộng dồn sai số trong giai đoạn này, hệ thống có thể gặp hiện tượng *integral windup*, gây quá điều chỉnh lớn hoặc làm hệ thống hồi phục chậm sau khi thoát bão hòa [23, 25]. Vì vậy, bộ điều khiển cần có cơ chế chống windup, ví dụ như giới hạn thành phần tích phân hoặc chặn biên tín hiệu điều khiển:

$$u_k = \text{sat}(u_k, u_{\min}, u_{\max}). \quad (2.24)$$

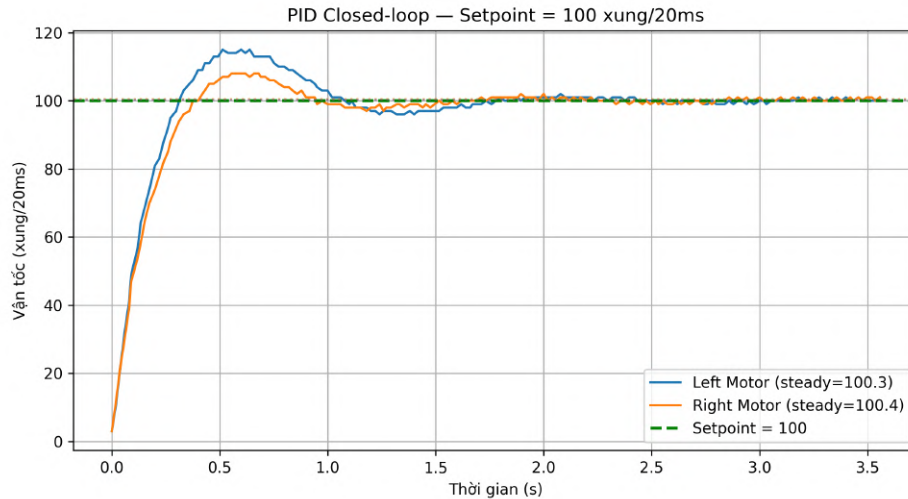
Ngoài ra, do tín hiệu encoder có tính rời rạc và dễ bị nhiễu lượng tử hóa, thành phần vi phân có thể khuếch đại nhiễu nếu không được lọc phù hợp. Vì vậy, trong nhiều hệ thống điều khiển vận tốc bánh xe, bộ điều khiển PI hoặc PID với thành phần D nhỏ thường được sử dụng để đạt đáp ứng ổn định hơn [23, 22].

Khảo sát đáp ứng động cơ trước và sau điều khiển PID

Để đánh giá ảnh hưởng của bộ điều khiển PID đến khả năng điều khiển vận tốc bánh xe, tiến hành khảo sát đáp ứng của hai động cơ DC trong hai trường hợp: không sử dụng PID và sử dụng PID. Kết quả đo vận tốc từ encoder được thể hiện trong Hình 2.2.3 và Hình 2.2.4. Để thí nghiệm có thể được tái lập, báo cáo cần ghi rõ chu kỳ điều khiển, điện áp nguồn, tải cơ khí, giá trị K_P , K_I , K_D , giới hạn PWM và phương pháp lọc vận tốc.



Hình 2.2.3: Đáp ứng vận tốc hai động cơ khi chưa có bộ điều khiển PID



Hình 2.2.4: Đáp ứng vận tốc hai động cơ khi sử dụng điều khiển PID

Từ kết quả khảo sát thực tế trên hai động cơ DC, có thể thấy bộ điều khiển PID giúp vận tốc hai bánh bám tốt hơn giá trị đặt. Như thể hiện trong Hình 2.2.4, ở trường hợp điều khiển vòng kín với giá trị đặt $100 \text{ xung}/20\text{ms}$, vận tốc động cơ trái và phải sau giai đoạn quá độ lần lượt ổn định xấp xỉ $100,3$ và $100,4 \text{ xung}/20 \text{ ms}$, gần như trùng với giá trị đặt. Mặc dù ban đầu có xuất hiện quá điều chỉnh, đặc biệt ở động cơ trái, hệ thống sau đó nhanh chóng hội tụ về vùng ổn định.

Ngược lại, Hình 2.2.3 cho thấy khi chưa có bộ điều khiển PID phù hợp, hai động cơ không đạt cùng một vận tốc xác lập. Động cơ trái ổn định quanh $108,2 \text{ xung}/20\text{ms}$, trong khi động cơ phải chỉ đạt khoảng $95,2 \text{ xung}/20 \text{ ms}$. Sai lệch này cho thấy nếu chỉ cấp cùng mức điều khiển mà không có phản hồi encoder, hai bánh xe có thể quay với vận tốc khác nhau do sai số cơ khí, ma sát, tải trọng hoặc đặc tính riêng của từng động cơ. Vì robot sử dụng cơ cấu truyền động vi sai, sự chênh lệch vận tốc giữa hai bánh sẽ làm robot bị lệch hướng khi di chuyển thẳng. Do đó, vòng điều khiển PID riêng cho từng bánh là cần thiết để đồng bộ vận tốc hai động cơ và cải thiện độ chính xác của chuyển động.

2.2.6 Hệ thống phản hồi vòng kín và mối quan hệ giữa PID - Odometry

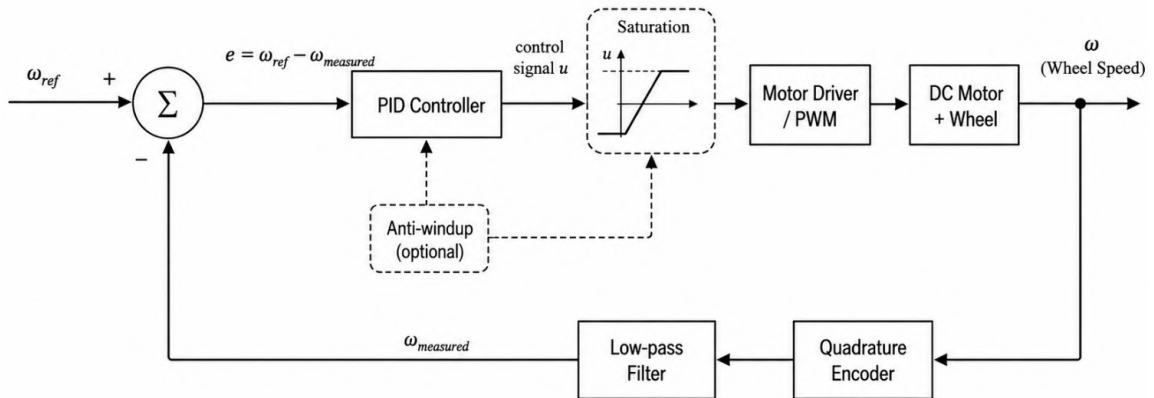
Trong robot di động bánh vi sai, các phương trình động học ngược cho phép chuyển đổi vận tốc mong muốn của thân robot, gồm vận tốc tuyến tính v và vận tốc góc ω , thành vận tốc góc yêu cầu cho bánh trái và bánh phải. Tuy nhiên, để robot thực tế di chuyển đúng với vận tốc đã tính toán, mỗi động cơ bánh xe cần có một vòng điều khiển vận tốc riêng. Vòng điều khiển này nhận vận tốc đặt của bánh xe, đo vận tốc thực tế thông qua encoder, sau đó điều chỉnh tín hiệu điều khiển động cơ để giảm sai lệch giữa vận tốc

đặt và vận tốc đo được. Trong `diff_drive_controller` của ROS 2, lệnh vận tốc của thân robot được chuyển thành lệnh vận tốc bánh xe, trong khi odometry được tính toán từ phản hồi phần cứng và xuất bản ra hệ thống [18].

Về bản chất điều khiển, đây là một hệ thống điều khiển phản hồi vòng kín. Giá trị đặt là vận tốc góc mong muốn của bánh xe, thường ký hiệu là ω_{ref} . Giá trị phản hồi là vận tốc góc thực tế của bánh xe, ký hiệu là $\omega_{measured}$, được ước lượng từ tín hiệu encoder. Sai số vận tốc được tính bởi:

$$e(t) = \omega_{ref}(t) - \omega_{measured}(t). \quad (2.25)$$

Bộ điều khiển sử dụng sai số này để tạo ra tín hiệu điều khiển $u(t)$, ví dụ PWM hoặc lệnh dòng/áp đưa tới motor driver (như L298N). Mục tiêu của vòng điều khiển là làm cho $\omega_{measured}$ bám theo ω_{ref} với sai số nhỏ, đáp ứng đủ nhanh và hạn chế dao động [23]. Tác động của từng thành phần trong điều khiển vận tốc bánh xe có thể hiểu như sau. Thành phần P tạo đáp ứng nhanh khi vận tốc thực tế lệch khỏi vận tốc đặt, nhưng nếu dùng riêng lẻ có thể còn sai số xác lập. Thành phần I giúp triệt tiêu sai số xác lập bằng cách cộng dồn sai số theo thời gian, nhưng nếu đặt quá lớn có thể gây quá điều chỉnh và dao động. Thành phần D giúp làm dịu đáp ứng bằng cách phản ứng với tốc độ thay đổi của tín hiệu, nhưng nó nhạy với nhiễu đo, đặc biệt khi vận tốc được tính từ encoder có độ phân giải hữu hạn [23].



Hình 2.2.5: Vòng điều khiển vận tốc bánh xe dùng PID và encoder.

Trong hệ thống robot di động thực tế, luồng truyền thông và dữ liệu được thực thi đồng bộ giữa các tầng phần cứng: Máy tính nhúng Orange Pi chạy ROS 2 nhận tín hiệu

điều hướng cao cấp từ mạng hoặc giải thuật tự hành, tính toán động học và gửi gói lệnh vận tốc (Command) xuống vi điều khiển STM32F103 qua giao tiếp Serial. Tại đây, STM32 tính toán PID thời gian thực, xuất xung điều khiển mạch cầu H L298N để thay đổi tốc độ thực tế của Motor Left và Motor Right. Song song với quá trình điều khiển bám, hệ thống liên tục trích xuất ngược dữ liệu xung Encoder từ động cơ, STM32 thực hiện nhiệm vụ đếm, giải mã và gửi ngược giá trị Encoder Count này lên lại Orange Pi qua đường truyền Serial. Tại Orange Pi, dữ liệu thô này được nạp vào mô hình động học thuận để tính toán Odometry cấp cao phục vụ cho cấu trúc định vị toàn cục. Do đó, hai vai trò này có liên hệ chặt chẽ: nếu vòng điều khiển vận tốc PID tại phần cứng cấp thấp không ổn định hoặc encoder đo sai, odometry ở tầng phần mềm cao cấp cũng sẽ bị sai lệch theo. Điều khiển vận tốc động cơ và xử lý encoder bảo đảm cho các module cấp cao hơn như định vị, lập bản đồ và điều hướng hoạt động ổn định. Sai số ở lớp này có thể lan truyền lên odometry, làm ảnh hưởng đến TF, localization, costmap và khả năng bám đường của toàn bộ hệ thống robot.

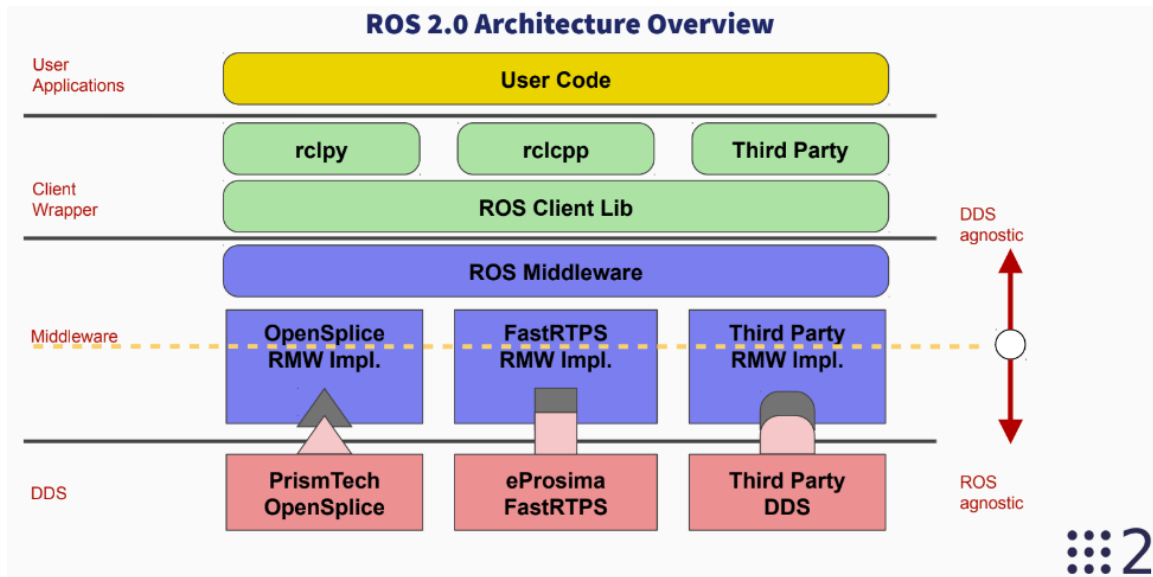
2.3 FRAMEWORK ROS 2 VÀ CÔNG CỤ PHÁT TRIỂN HỆ THỐNG

Để hiện thực hóa các thuật toán điều hướng, xây dựng bản đồ và quản lý dòng dữ liệu truyền thông thời gian thực trên máy tính nhúng Orange Pi 5, việc lựa chọn một nền tảng phần mềm trung gian ổn định là điều bắt buộc. Khung phần mềm ROS 2 kết hợp với công cụ trực quan hóa RViz2 tạo thành nền tảng phần mềm phục vụ tích hợp, kiểm thử và giám sát robot trên phần cứng thực tế.

2.3.1 Tổng quan về ROS 2 và Mô hình truyền thông

Robot Operating System 2 không phải là một hệ điều hành truyền thống quản lý phần cứng cốt lõi, mà là một khung phần mềm trung gian phân tán chạy trên nền hệ điều hành Linux [26]. Điểm cải tiến cách mạng của ROS 2 so với thế hệ ROS 1 tiền nhiệm là việc loại bỏ hoàn toàn nút quản lý trung tâm và thay thế bằng kiến trúc truyền thông dựa trên tiêu chuẩn DDS (Data Distribution Service). Sự thay đổi này loại bỏ sự phụ thuộc vào một ROS Master trung tâm, đồng thời cung cấp cơ chế khám phá phân tán và các chính sách Quality of Service. Khả năng đáp ứng thời gian thực của toàn hệ thống vẫn phụ thuộc vào hệ điều hành, executor, cấu hình DDS và cách triển khai node [27].

Kiến trúc phân tầng của ROS 2 được mô tả cụ thể trong Hình 2.3.1, thể hiện sự độc lập giữa mã nguồn ứng dụng của người dùng ở tầng trên và các nhà cung cấp dịch vụ DDS ở tầng dưới thông qua lớp trừu tượng RMW (ROS Middleware Interface).



Hình 2.3.1: Kiến trúc phân tầng tổng quan của hệ điều hành robot ROS 2[7]

Mô hình tính toán của ROS 2 dựa trên kiến trúc đồ thị (Computation Graph), trong đó các tác vụ xử lý độc lập được thực thi bởi các nút mạng gọi là Node. Các Node trao đổi dữ liệu với nhau qua ba cơ thức giao tiếp cơ bản:

- **Topic (Chủ đề):** Hoạt động theo mô hình Publisher/Subscriber, cho phép truyền dữ liệu liên tục không đồng bộ một chiều. Đây là cơ chế dùng để truyền dữ liệu tần số cao như dữ liệu quét 2D LaserScan, odometry và trạng thái cảm biến.
- **Service (Dịch vụ):** Hoạt động theo mô hình yêu cầu–phản hồi giữa client và server. Lỗi gọi có thể được thực hiện theo cách đồng bộ hoặc bất đồng bộ; cách bất đồng bộ thường được ưu tiên khi node vẫn cần tiếp tục xử lý callback khác trong thời gian chờ phản hồi. Service phù hợp với các tác vụ ngắn như truy vấn trạng thái, lưu bản đồ hoặc thay đổi cấu hình [28].
- **Action (Hành động):** Là cơ chế truyền thông không đồng bộ thời gian dài, cho phép theo dõi tiến độ (feedback) và có thể hủy bỏ giữa chừng (cancel). Action rất quan trọng đối với các tác vụ di chuyển của khung Nav2, khi mục tiêu điều hướng cần nhiều thời gian để hoàn thành.

2.3.2 Cơ chế Quality of Service trong ROS 2

Do ROS 2 sử dụng DDS làm tầng truyền thông, publisher và subscriber có thể cấu hình các chính sách Quality of Service (QoS) thay vì chỉ sử dụng một chế độ truyền cố định. Các chính sách quan trọng gồm *history* và *depth* để quy định số mẫu được lưu trong hàng đợi, *reliability* để lựa chọn truyền tin cậy hoặc cố gắng tối đa, và *durability* để xác định dữ liệu cũ có được cung cấp cho subscriber tham gia muộn hay không [29].

Đối với dữ liệu cảm biến tần số cao như */scan*, chế độ *best effort* thường phù hợp vì một mẫu bị mất có thể được thay thế nhanh bởi mẫu kế tiếp và việc chờ truyền lại có thể làm tăng độ trễ. Ngược lại, các dữ liệu cấu hình hoặc bản đồ tĩnh cần tính nhất quán cao hơn thường sử dụng chế độ *reliable*; bản đồ hoặc các thông tin cần cung cấp cho node tham gia muợn có thể sử dụng *transient local*. Publisher và subscriber chỉ kết nối khi cấu hình QoS tương thích, vì vậy việc lựa chọn sai profile có thể làm một topic tồn tại nhưng không có dữ liệu được trao đổi. Trong đề tài, hiểu rõ QoS đặc biệt quan trọng khi tích hợp LiDAR, SLAM Toolbox, AMCL và Navigation2 trên cùng máy tính nhúng.

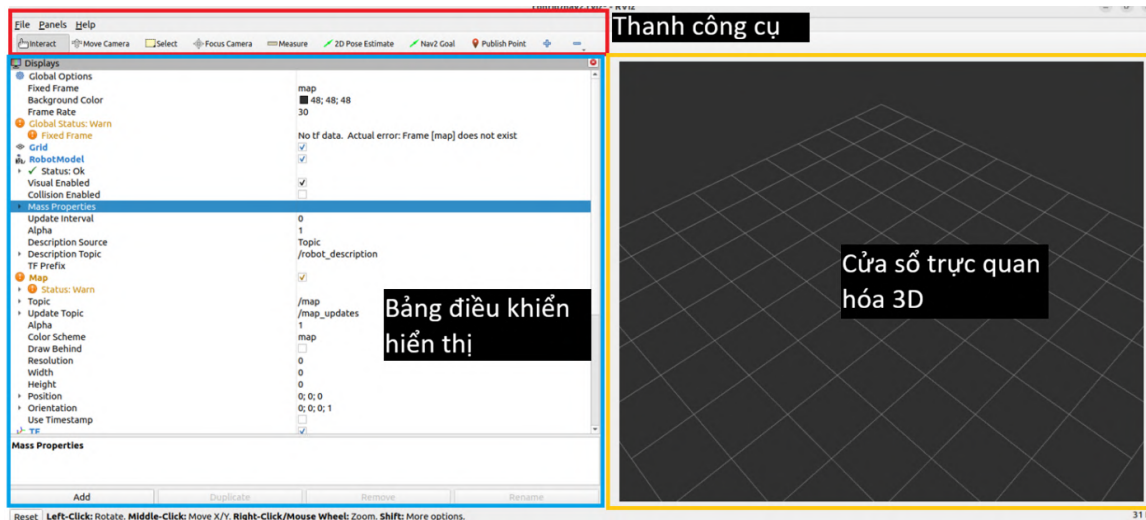
2.3.3 Công cụ trực quan hóa RViz2

RViz2 là công cụ trực quan hóa dữ liệu chính trong hệ sinh thái ROS 2, được sử dụng rộng rãi trong quá trình phát triển, kiểm thử và giám sát hoạt động của robot tự hành [30]. Công cụ này cho phép hiển thị trực quan các dữ liệu được trao đổi trên hệ thống ROS 2 theo thời gian thực, giúp người phát triển dễ dàng theo dõi trạng thái và đánh giá kết quả hoạt động của các thuật toán.

Khác với các môi trường mô phỏng vật lý, RViz2 tập trung vào việc hiển thị dữ liệu và trạng thái của robot. Thông qua RViz2, người dùng có thể quan sát mô hình robot, dữ liệu cảm biến LiDAR, bản đồ môi trường, cây biến đổi tọa độ TF, Costmap, quỹ đạo điều hướng và kết quả định vị của robot. Đây là công cụ quan trọng trong quá trình phát triển các ứng dụng sử dụng ROS 2, đặc biệt đối với các bài toán định vị, lập bản đồ và điều hướng tự động [31].

Giao diện làm việc của RViz2 được chia thành các khu vực chức năng chính như minh họa trong Hình 2.3.2:

- **Thanh công cụ điều khiển (Khu vực viền đỏ):** Cung cấp các công cụ tương tác như điều khiển góc nhìn, đo khoảng cách, ước lượng vị trí ban đầu (2D Pose Estimate) và gửi mục tiêu điều hướng (Nav2 Goal).
- **Bảng điều khiển hiển thị (Khu vực viền xanh dương):** Cho phép cấu hình hệ tọa độ tham chiếu (Fixed Frame) và quản lý các thành phần dữ liệu cần hiển thị như bản đồ, mô hình robot, dữ liệu cảm biến và cây TF.
- **Cửa sổ trực quan hóa 3D (Khu vực viền vàng):** Hiển thị môi trường và trạng thái hoạt động của robot theo thời gian thực, giúp người dùng theo dõi kết quả của các thuật toán định vị và điều hướng.



Hình 2.3.2: Công cụ trực quan hóa RViz2

Trong đồ án này, RViz2 được sử dụng để quan sát bản đồ môi trường, theo dõi vị trí robot, kiểm tra dữ liệu cảm biến LiDAR và giám sát quá trình điều hướng tự động của hệ thống. Nhờ khả năng trực quan hóa theo thời gian thực, RViz2 hỗ trợ hiệu quả cho quá trình phát triển, kiểm thử và đánh giá hoạt động của robot.

2.4 SLAM, ĐỊNH VỊ VÀ ĐIỀU HƯỚNG TỰ HÀNH

Phần này trình bày cơ sở lý thuyết của tầng thuật toán tự hành trong robot di động. Tầng này có nhiệm vụ tiếp nhận dữ liệu từ cảm biến LiDAR, kết hợp với thông tin odometry của hệ truyền động để thực hiện ba chức năng chính: xây dựng bản đồ, định vị robot trên bản đồ đã biết và điều hướng robot đến vị trí mục tiêu. Trong kiến trúc của đề tài, các chức năng này được tổ chức trên nền tảng ROS 2, kết hợp với các thành phần phần mềm như SLAM Toolbox, AMCL và Navigation2 [26, 32, 33].

Về mặt tổng thể, quá trình tự hành có thể được chia thành hai giai đoạn. Ở giai đoạn đầu, robot di chuyển trong môi trường chưa biết và sử dụng dữ liệu cảm biến để xây dựng bản đồ. Ở giai đoạn vận hành, robot sử dụng bản đồ đã có để định vị vị trí hiện tại, lập kế hoạch đường đi và điều khiển chuyển động bám theo quỹ đạo đã được tính toán. Cách tổ chức này giúp tách biệt rõ ràng giữa bài toán xây dựng bản đồ và bài toán điều hướng trên bản đồ đã biết.

2.4.1 Bài toán SLAM và mô hình bản đồ lưới chiếm dụng

SLAM, viết tắt của *Simultaneous Localization and Mapping*, là bài toán trong đó robot vừa ước lượng tư thế của chính nó, vừa xây dựng bản đồ của môi trường chưa biết dựa trên dữ liệu cảm biến thu thập được theo thời gian [34]. Đối với robot di động hoạt

động trên mặt phẳng, tư thế của robot tại thời điểm t thường được biểu diễn bởi:

$$\mathbf{x}_t = [x_t, y_t, \theta_t]^T, \quad (2.26)$$

trong đó x_t và y_t là tọa độ vị trí, còn θ_t là góc hướng của robot trong hệ tọa độ phẳng.

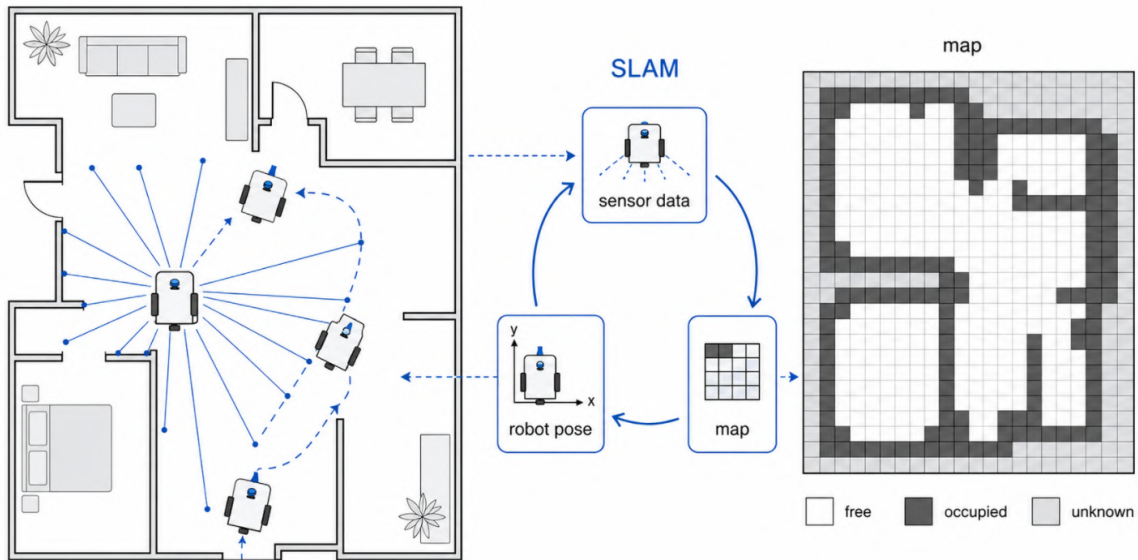
Về mặt xác suất, quá trình SLAM có thể được mô tả thông qua mô hình chuyển động và mô hình đo đạc:

$$p(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{u}_t), \quad (2.27)$$

$$p(\mathbf{z}_t | \mathbf{x}_t, m). \quad (2.28)$$

Trong đó $\mathbf{u}_t$ là thông tin chuyển động của robot, $\mathbf{z}_t$ là dữ liệu đo từ cảm biến và m là bản đồ môi trường cần ước lượng. Do odometry có thể tích lũy sai số theo thời gian bởi trượt bánh, sai số encoder hoặc sai số cơ khí, dữ liệu LiDAR được sử dụng để hỗ trợ hiệu chỉnh tư thế và giảm sai lệch khi xây dựng bản đồ.

Trong các hệ thống robot di động 2D, bản đồ thường được biểu diễn bằng mô hình lưới chiếm dụng, hay *Occupancy Grid Map*. Môi trường được chia thành các ô lưới hai chiều, trong đó mỗi ô biểu diễn trạng thái của một vùng không gian nhỏ: trống, bị chiếm dụng hoặc chưa biết [35] như Hình 2.4.1.



Hình 2.4.1: Mô tả bài toán SLAM

Cách biểu diễn này phù hợp với dữ liệu LiDAR 2D vì các tia quét khoảng cách có thể được chiếu trực tiếp lên mặt phẳng để cập nhật trạng thái các ô lưới.

Để cập nhật xác suất chiếm dụng một cách hiệu quả, bản đồ lưới có thể sử dụng dạng log-odds:

$$l_t(m_i) = l_{t-1}(m_i) + \text{logit}(P(m_i | \mathbf{z}_t, \mathbf{x}_t)) - l_0. \quad (2.29)$$

Trong đó $l_t(m_i)$ là giá trị log-odds của ô lưới m_i tại thời điểm t , l_0 là giá trị log-odds ban đầu, và:

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right). \quad (2.30)$$

Từ giá trị log-odds, xác suất chiếm dụng có thể được tính lại bằng:

$$P(m_i) = \frac{1}{1 + e^{-l_t(m_i)}} \quad (2.31)$$

Ví dụ, giả sử ban đầu robot chưa có thông tin về ô lưới m_i , nên:

$$P(m_i) = 0.5.$$

Khi đó:

$$l_0 = \text{logit}(0.5) = \log\left(\frac{0.5}{1-0.5}\right) = \log(1) = 0.$$

Sau một lần quét LiDAR, robot đánh giá xác suất ô lưới bị chiếm dụng là:

$$P(m_i | \mathbf{z}_t, \mathbf{x}_t) = 0.8.$$

Giá trị log-odds tương ứng được tính:

$$\text{logit}(0.8) = \log\left(\frac{0.8}{0.2}\right) = \log(4) \approx 1.386.$$

Do $l_{t-1}(m_i) = 0$, nên giá trị log-odds mới:

$$l_t(m_i) = 0 + 1.386 - 0 = 1.386.$$

Từ đó, xác suất chiếm dụng của ô lưới được tính lại:

$$P(m_i) = \frac{1}{1 + \exp(-1.386)} \approx 0.8.$$

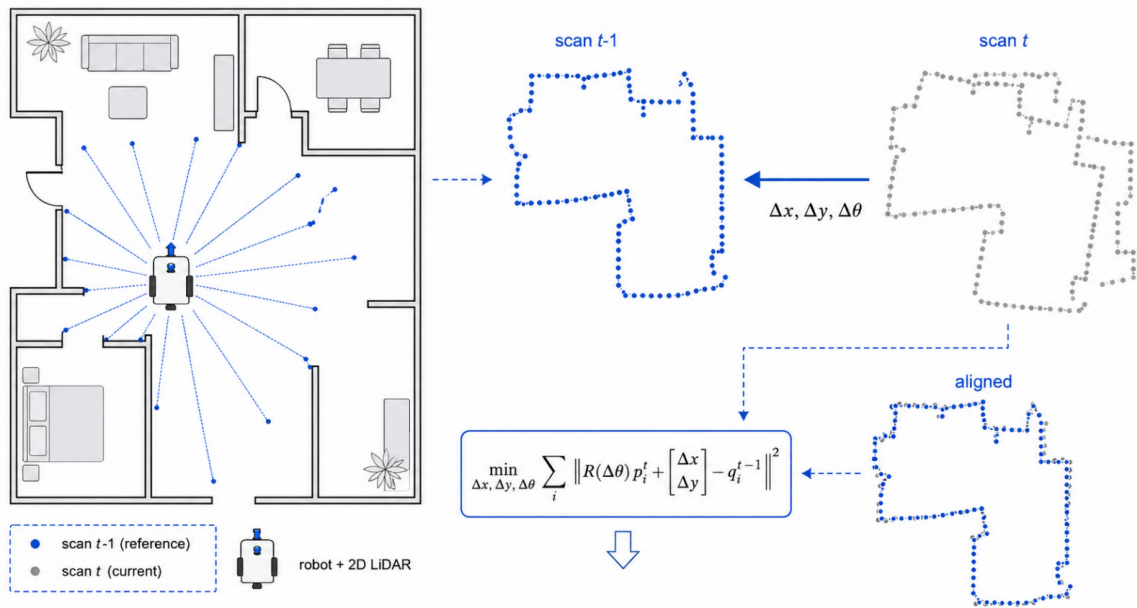
Điều này cho thấy robot tin rằng ô lưới đang xét có khoảng 80% khả năng bị chiếm dụng. Sau nhiều lần cập nhật liên tiếp từ dữ liệu LiDAR, các ô chứa vật cản sẽ có xác suất ngày càng tiến gần đến 1, trong khi các ô trống sẽ tiến gần đến 0.

2.4.2 Dữ liệu 2D LiDAR và kỹ thuật Scan Matching

Cảm biến 2D LiDAR cung cấp dữ liệu khoảng cách theo nhiều hướng quét xung quanh robot. Mỗi lần quét có thể được xem như một tập hợp các điểm trong mặt phẳng, biểu diễn khoảng cách từ robot đến các vật cản xung quanh như tường, bàn, vách ngăn, thiết bị trưng bày hoặc lối đi trong khu vực. Dữ liệu này đóng vai trò quan trọng trong các nhiệm vụ phát hiện vật cản, xây dựng bản đồ và hỗ trợ định vị robot [34].

Trong quá trình di chuyển, nếu robot chỉ sử dụng dữ liệu odometry từ encoder bánh xe để ước lượng vị trí, sai số sẽ tăng dần theo thời gian do hiện tượng trượt bánh, sai số cơ khí hoặc nhiễu cảm biến. Ví dụ, khi robot di chuyển dọc theo hành lang trong khu vực, encoder có thể dự đoán rằng robot đã đi được đúng 5 m, tuy nhiên vị trí thực tế có thể bị lệch vài centimet. Sai số này nếu tích lũy lâu dài sẽ khiến robot định vị không chính xác trên bản đồ.

Để giảm sai số đó, các hệ thống SLAM 2D thường sử dụng kỹ thuật *scan matching*. Kỹ thuật này thực hiện việc so khớp dữ liệu quét LiDAR hiện tại với dữ liệu quét trước đó hoặc với bản đồ cục bộ đã xây dựng nhằm ước lượng lại tư thế chính xác hơn của robot. Ví dụ về scan matching có thể tham khảo ở Hình 2.4.2.



Hình 2.4.2: Kỹ thuật scan matching

Một cách tổng quát, bài toán scan matching có thể được biểu diễn dưới dạng tối ưu hóa:

$$\Delta \mathbf{x}^* = \arg \min_{\Delta \mathbf{x}} \sum_i \|\mathbf{r}_i(\Delta \mathbf{x})\|^2, \quad (2.32)$$

trong đó $\Delta \mathbf{x} = [\Delta x, \Delta y, \Delta \theta]^T$ biểu diễn sự thay đổi tư thế (pose) của robot cần được ước lượng, bao gồm:

- dịch chuyển theo trục x ,
- dịch chuyển theo trục y ,
- và góc quay θ .

Hiểu một cách trực quan, khi robot di chuyển, dữ liệu odometry từ encoder có thể khiến robot “nghĩ” rằng nó đang ở một vị trí hơi khác so với thực tế. Ví dụ, robot dự đoán khoảng cách đến một vách ngăn là $4,95m$, trong khi dữ liệu LiDAR thực tế đo được khoảng cách là $5,00m$. Khi đó, thuật toán scan matching sẽ suy ra rằng tư thế hiện tại của robot đang bị lệch khoảng $5cm$ và cập nhật lại tọa độ của robot trong hệ thống để phù hợp hơn với môi trường thực tế.

Điểm quan trọng là robot không cần phải di chuyển lại $5cm$ đó, mà hệ thống chỉ hiệu chỉnh lại pose của robot trong bản đồ. Sau khi pose được cập nhật chính xác hơn, các thành phần khác như localization, path planning và obstacle avoidance sẽ tiếp tục sử dụng thông tin vị trí mới để điều khiển robot di chuyển chính xác.

Kết quả scan matching sau đó có thể được sử dụng để tạo các ràng buộc (constraints) giữa các tư thế trong đồ thị SLAM. Các ràng buộc này giúp hệ thống tối ưu hóa toàn bộ bản đồ và giảm sai số tích lũy theo thời gian, từ đó cải thiện độ chính xác của localization và mapping [32].

2.4.3 Gói SLAM Toolbox và phương pháp SLAM dựa trên đồ thị tư thế

SLAM Toolbox không phải là một thuật toán SLAM đơn lẻ mà là một gói phần mềm SLAM 2D được phát triển cho ROS 2. Gói này cung cấp các chức năng phục vụ xây dựng bản đồ, định vị robot và tiếp tục mở rộng bản đồ đã lưu trước đó [32]. Trong đề tài này, SLAM Toolbox được sử dụng như thành phần chính để xây dựng bản đồ 2D dựa trên dữ liệu LiDAR và odometry từ encoder bánh xe.

Về nguyên lý hoạt động, SLAM Toolbox sử dụng hướng tiếp cận *pose graph SLAM* (SLAM dựa trên đồ thị tư thế). Trong mô hình này, hệ thống sẽ lưu lại các tư thế của robot theo thời gian dưới dạng một đồ thị. Mỗi đỉnh (node) trong đồ thị biểu diễn một tư thế của robot tại một thời điểm, bao gồm vị trí và góc quay của robot. Trong khi đó, mỗi cạnh (edge) biểu diễn mối quan hệ tương đối giữa hai tư thế liên tiếp, chẳng hạn robot đã di chuyển bao xa hoặc quay bao nhiêu độ giữa hai thời điểm.

Các ràng buộc giữa các tư thế có thể được hình thành từ dữ liệu odometry hoặc từ kết quả scan matching của LiDAR. Ví dụ, khi robot di chuyển dọc tường, encoder bánh xe có thể cung cấp thông tin robot đã đi thêm khoảng 1 m. Đồng thời, dữ liệu LiDAR sẽ được sử dụng để so sánh môi trường hiện tại với các dữ liệu quét trước đó nhằm hiệu chỉnh lại sai số của odometry. Tuy nhiên, do encoder luôn tồn tại sai số tích lũy, quỹ đạo robot có thể bị lệch dần theo thời gian.

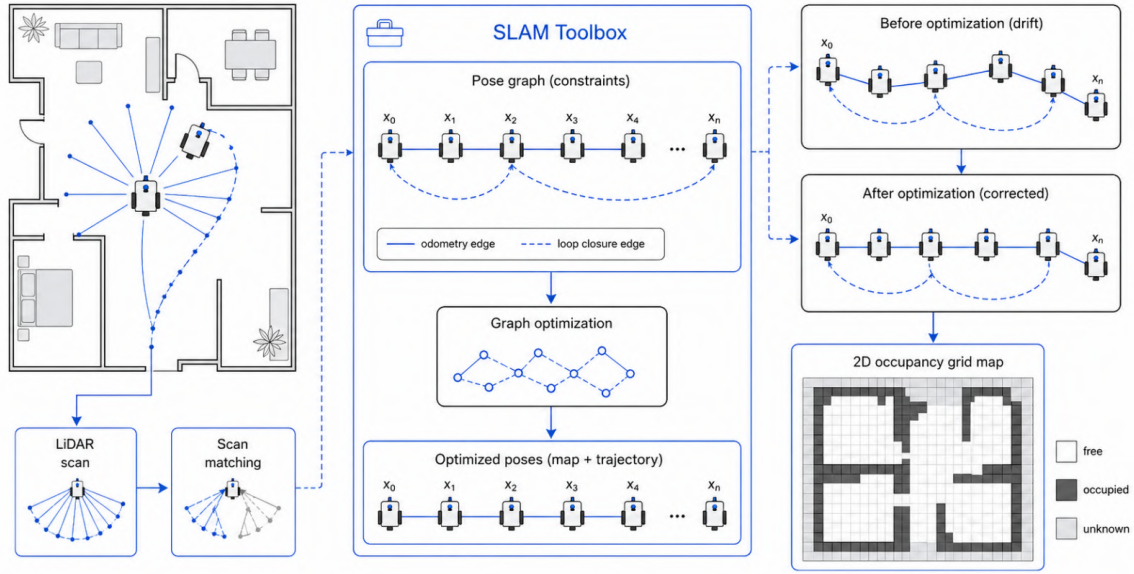
Khi robot quay trở lại một khu vực đã đi qua trước đó, chẳng hạn quay lại gần một khu vực có cấu trúc quen thuộc, dữ liệu LiDAR có thể nhận ra rằng môi trường hiện tại rất giống với dữ liệu đã quan sát trước đó. Khi đó, cơ chế *loop closure* (khép vòng) sẽ được thực hiện để tạo thêm một ràng buộc mới giữa tư thế hiện tại và tư thế cũ trong đồ thị. Ràng buộc này giúp hệ thống phát hiện rằng quỹ đạo hiện tại đang bị sai lệch và cần được hiệu chỉnh lại.

Sau khi các ràng buộc được tạo ra, SLAM Toolbox sẽ thực hiện quá trình tối ưu hóa đồ thị tư thế nhằm tìm ra tập tư thế robot phù hợp nhất với toàn bộ các ràng buộc đã quan sát được. Bài toán tối ưu này có thể được biểu diễn dưới dạng:

$$\mathbf{x}^* = \arg \min_{\mathbf{x}} \sum_{i,j} \mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j)^T \Omega_{ij} \mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j). \quad (2.33)$$

Trong đó, $\mathbf{x}_i$ và $\mathbf{x}_j$ là hai tư thế của robot trong đồ thị, còn $\mathbf{e}_{ij}$ biểu diễn sai số giữa mối quan hệ thực tế và mối quan hệ được dự đoán giữa hai tư thế này. Ma trận Ω_{ij} thể hiện độ tin cậy của từng ràng buộc; các ràng buộc có độ tin cậy cao sẽ ảnh hưởng mạnh hơn trong quá trình tối ưu hóa.

Hiểu một cách trực quan, quá trình tối ưu đồ thị có thể được xem như việc “điều chỉnh lại” toàn bộ quỹ đạo của robot sao cho dữ liệu odometry, scan matching và các vị trí khép vòng khớp với nhau tốt nhất. Ví dụ, nếu robot di chuyển vòng quanh một vị trí và quay lại vị trí ban đầu nhưng bản đồ hiện tại bị lệch vài chục centimet, SLAM Toolbox sẽ sử dụng các ràng buộc khép vòng để hiệu chỉnh lại toàn bộ quỹ đạo, từ đó giúp bản đồ cuối cùng trở nên chính xác và nhất quán hơn, quá trình Slam toolbox điều chỉnh được mô tả trong ảnh 2.4.3.



Hình 2.4.3: Slam Toolbox trong Ros2

2.4.4 Định vị thời gian thực bằng AMCL

Sau khi bản đồ tĩnh của môi trường đã được xây dựng, robot có thể chuyển sang chế độ định vị trên bản đồ đã biết thay vì tiếp tục thực hiện SLAM. Trong đề tài này, quá trình định vị được thực hiện bằng AMCL (*Adaptive Monte Carlo Localization*), là một phương pháp định vị xác suất dựa trên bộ lọc hạt [33]. Mục tiêu của AMCL là giúp robot xác định vị trí và hướng hiện tại của mình trên bản đồ trong thời gian thực.

Về nguyên lý hoạt động, AMCL không giả định robot chỉ đang ở một vị trí duy nhất, mà tạo ra nhiều giả thuyết khác nhau về vị trí của robot trên bản đồ. Mỗi giả thuyết được biểu diễn bằng một *particle* (hạt), trong đó mỗi hạt chứa thông tin về vị trí và hướng của robot. Có thể hiểu trực quan rằng mỗi hạt là một “phỏng đoán” của hệ thống về việc robot đang ở đâu trong môi trường.

Quá trình AMCL gồm ba bước chính: dự đoán, cập nhật và tái mẫu.

- **Ở bước dự đoán:** Tập hạt được cập nhật dựa trên chuyển động của robot từ dữ liệu odometry. Với robot truyền động vi sai, hệ thống sử dụng vận tốc tịnh tiến và vận tốc góc để dự đoán robot đã di chuyển như thế nào. Tuy nhiên, do encoder bánh xe luôn tồn tại sai số thực tế như trượt bánh hoặc nhiễu cơ khí, quá trình cập nhật này có bổ sung nhiễu ngẫu nhiên để mô phỏng độ bất định của chuyển động. Kết quả là các hạt sẽ dịch chuyển theo hướng chuyển động của robot nhưng không hoàn toàn giống nhau.
- **Ở bước cập nhật:** Dữ liệu LiDAR hiện tại sẽ được so sánh với bản đồ tĩnh đã

lưu. Hệ thống sẽ kiểm tra từng hạt để đánh giá xem nếu robot thực sự ở vị trí đó thì dữ liệu LiDAR có khớp với môi trường trên bản đồ hay không. Những hạt có dữ liệu dự đoán phù hợp với dữ liệu cảm biến sẽ nhận trọng số cao hơn, trong khi các hạt không phù hợp sẽ bị giảm trọng số. Ví dụ, nếu robot đang di chuyển giữa các dãy kệ sách trong thư viện, những hạt có vị trí dự đoán tạo ra dữ liệu LiDAR giống với cấu trúc kệ sách thực tế sẽ được đánh giá cao hơn. Sau nhiều lần cập nhật, các hạt sẽ dần tập trung quanh vị trí thực của robot [34, 33].

- **Ở bước tái mẫu:** Hệ thống sẽ tạo ra tập hạt mới dựa trên trọng số của các hạt hiện tại. Các hạt có trọng số cao sẽ có xác suất được giữ lại hoặc nhân bản nhiều hơn, trong khi các hạt có trọng số thấp sẽ dần bị loại bỏ. Nhờ đó, phân bố hạt ngày càng hội tụ về tư thế thực của robot.

AMCL sử dụng cơ chế lấy mẫu thích ứng để thay đổi số lượng hạt theo mức độ bất định của quá trình định vị [36]. Khi robot đã định vị ổn định và dữ liệu LiDAR khớp tốt với bản đồ, hệ thống có thể giảm số lượng hạt để tiết kiệm tài nguyên tính toán. Ngược lại, nếu robot bị mất định vị hoặc môi trường có nhiều thay đổi, AMCL sẽ tăng số lượng hạt để mở rộng phạm vi tìm kiếm và cải thiện khả năng xác định lại vị trí của robot.

2.4.5 Khung điều hướng Navigation2 và bản đồ chi phí

Navigation2 là khung điều hướng trong ROS 2, cung cấp các thành phần phục vụ định vị, lập kế hoạch đường đi, điều khiển chuyển động, tránh vật cản và tổ chức hành vi điều hướng ở mức cao [33]. Trong kiến trúc điều hướng, Nav2 không chỉ tạo đường đi từ vị trí hiện tại đến mục tiêu, mà còn điều phối các thành phần lập kế hoạch, điều khiển và phục hồi thông qua mô hình Behavior Tree.

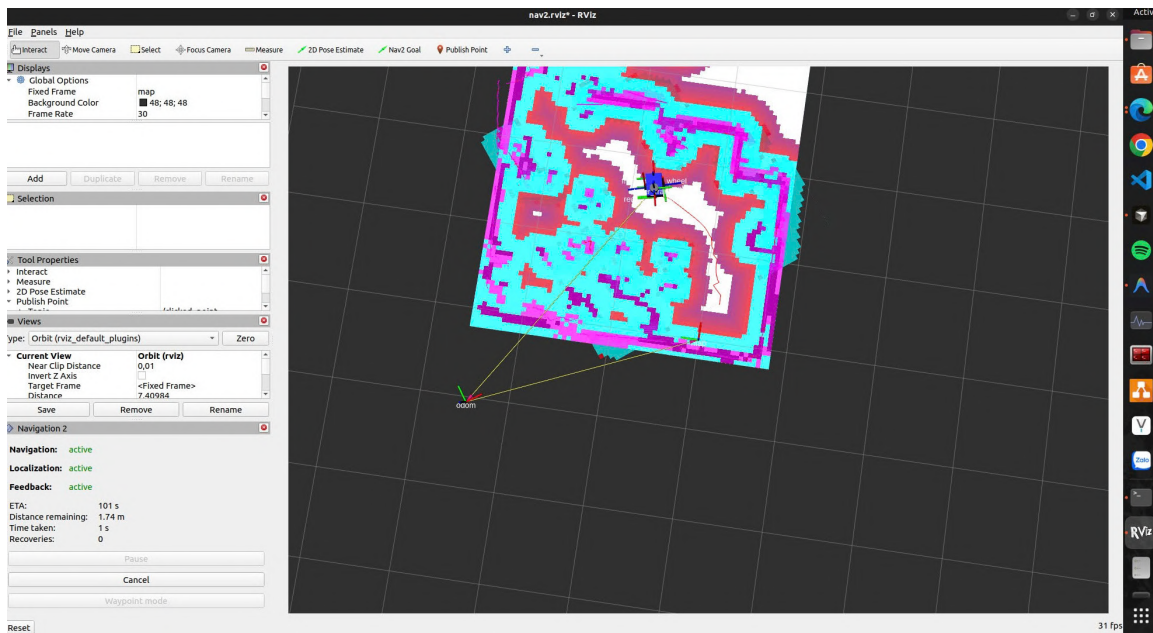
Một thành phần quan trọng của Nav2 là bản đồ chi phí, hay *Costmap*. Costmap biểu diễn môi trường dưới dạng lưới hai chiều, trong đó mỗi ô mang một giá trị chi phí di chuyển. Vùng trống có chi phí thấp, vùng gần vật cản có chi phí cao hơn, còn vùng vật cản trực tiếp được xem là không thể đi qua. Nhờ đó, các bộ lập kế hoạch và bộ điều khiển có thể lựa chọn quỹ đạo an toàn hơn thay vì chỉ tối ưu theo khoảng cách hình học.

Nav2 thường sử dụng hai loại costmap. *Global Costmap* biểu diễn môi trường ở phạm vi toàn cục và được dùng để lập đường đi dài hạn từ vị trí robot đến vị trí đích. *Local Costmap* biểu diễn vùng không gian gần robot và được dùng để hỗ trợ điều khiển bám quỹ đạo, kiểm tra vật cản gần và phản ứng với các thay đổi cục bộ trong môi trường. *Global Costmap* và *Local Costmap* phục vụ hai mục đích khác nhau trong hệ thống điều hướng của Nav2.

Global Costmap mô tả môi trường ở phạm vi rộng, thường được xây dựng từ bản đồ

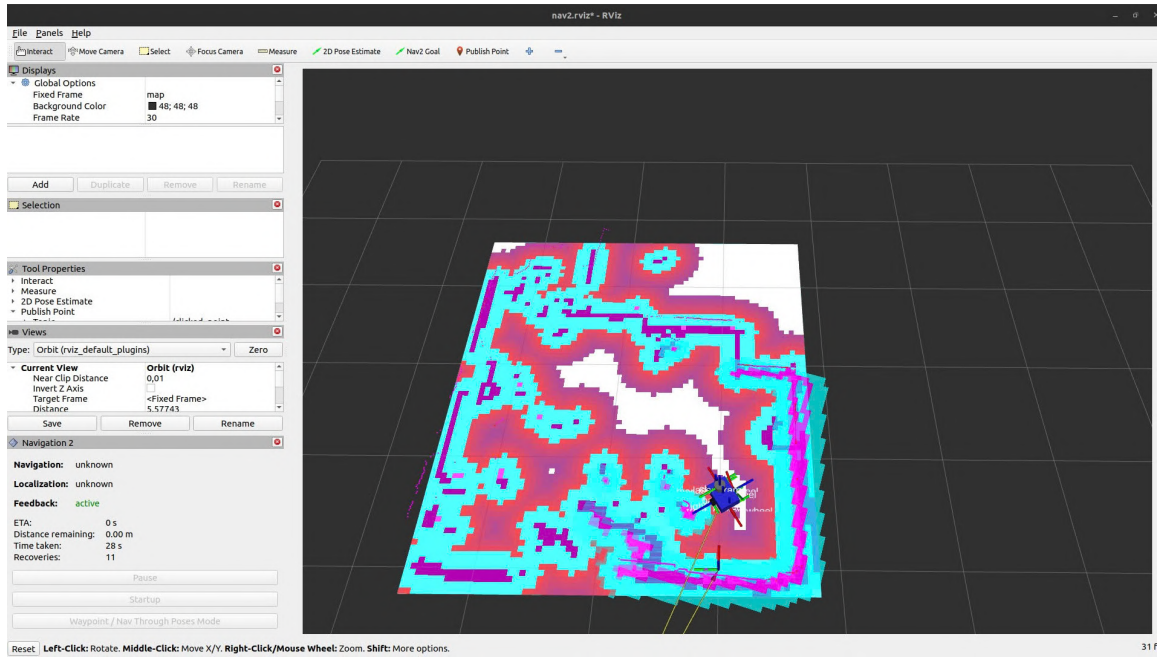
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

tính của môi trường kết hợp với dữ liệu vật cản cố định. Bản đồ này được sử dụng bởi bộ lập kế hoạch toàn cục (*global planner*) để tìm đường đi từ vị trí hiện tại của robot đến mục tiêu cuối cùng. Vì làm việc trên phạm vi lớn nên global costmap thường cập nhật chậm hơn và tập trung vào cấu trúc tổng quát của môi trường, ví dụ như tường, hành lang hoặc các khu vực cấm di chuyển. Đường đi sinh ra từ global costmap thường mang tính tối ưu toàn cục, giúp robot xác định hướng di chuyển tổng thể để đến đích. Hình 2.4.4 minh họa đường sinh ra từ global.



Hình 2.4.4: Đường đi trong bản đồ Global có vật cản cố định

Ngược lại, **Local Costmap** chỉ mô tả vùng không gian nhỏ xung quanh robot trong thời gian thực. Costmap này liên tục cập nhật dữ liệu từ các cảm biến như LiDAR hoặc camera để phát hiện vật cản động và các thay đổi cục bộ trong môi trường. Bộ điều khiển cục bộ (*local controller*) sử dụng local costmap để điều chỉnh vận tốc, tránh va chạm và bám theo quỹ đạo do global planner tạo ra. Nhờ local costmap, robot có thể phản ứng nhanh với các vật cản xuất hiện bất ngờ như con người hoặc đồ vật di chuyển.



Hình 2.4.5: Bản đồ chi phí trong Ros2

Hình 2.4.5 minh họa một bản đồ chi phí. Costmap được biểu diễn dưới dạng lưới chi phí bao quanh robot và được cập nhật liên tục từ dữ liệu cảm biến. Các ô trên bản đồ mang các mức chi phí khác nhau, trong đó vùng gần vật cản có chi phí cao hơn vùng không gian trống. Những vật cản được cảm biến phát hiện trực tiếp xuất hiện dưới dạng các cụm điểm trên costmap, đồng thời lớp inflation tạo ra vùng đệm an toàn xung quanh chúng bằng cách tăng dần giá trị chi phí theo khoảng cách đến vật cản.

Nhờ cơ chế này, bộ điều khiển cục bộ không chỉ tránh các ô bị chiếm dụng hoàn toàn mà còn có xu hướng giữ khoảng cách an toàn với tường và các vật cản lân cận. Khi robot di chuyển hoặc khi môi trường thay đổi, local costmap được cập nhật theo thời gian thực để phản ánh trạng thái mới của môi trường và hỗ trợ điều chỉnh quỹ đạo nhằm tránh va chạm.

2.4.6 Quy hoạch đường đi toàn cục bằng thuật toán A*

Sau khi robot đã có bản đồ và biết vị trí hiện tại, bài toán tiếp theo là tìm một đường đi khả thi từ vị trí bắt đầu đến vị trí mục tiêu. Đây là nhiệm vụ của bộ lập kế hoạch toàn cục. Trong Nav2, bộ lập kế hoạch toàn cục có thể sử dụng nhiều phương pháp khác nhau; một trong những phương pháp phổ biến trên bản đồ lưới là thuật toán A* [33].

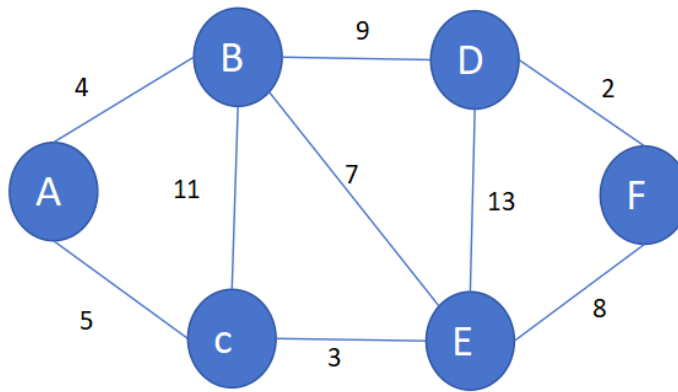
Thuật toán A* tìm đường đi từ ô bắt đầu S đến ô đích G bằng cách đánh giá mỗi ô n thông qua hàm:

$$f(n) = g(n) + h(n). \quad (2.34)$$

Trong đó $g(n)$ là chi phí thực tế tích lũy từ điểm bắt đầu đến ô hiện tại, còn $h(n)$ là chi phí ước lượng từ ô hiện tại đến điểm đích. Đối với bản đồ lưới hai chiều, hàm heuristic thường được chọn dựa trên khoảng cách Euclidean hoặc Manhattan, tùy theo mô hình lân cận của lưới:

$$h(n) = \sqrt{(x_n - x_G)^2 + (y_n - y_G)^2}. \quad (2.35)$$

So với Dijkstra, A* sử dụng thêm hàm heuristic để định hướng quá trình tìm kiếm về phía mục tiêu, nhờ đó thường giảm số lượng ô cần duyệt trong quá trình lập kế hoạch [37]. Khi hoạt động trên costmap, chi phí của đường đi không chỉ phụ thuộc vào chiều dài hình học mà còn phụ thuộc vào mức độ an toàn của các ô mà đường đi đi qua. Vì vậy, kết quả quy hoạch là một đường đi có chi phí tổng thể thấp, tránh vùng vật cản và ưu tiên các vùng an toàn hơn.



Hình 2.4.6: Ví dụ minh họa thuật toán A* trên đồ thị có trọng số

Hình 2.4.6 minh họa quá trình tìm đường của thuật toán A* trên một đồ thị có trọng số. Giả sử robot cần di chuyển từ đỉnh A đến đỉnh đích F, trong đó trọng số trên mỗi cạnh biểu diễn chi phí di chuyển giữa hai đỉnh.

Để minh họa nguyên lý hoạt động của thuật toán, giả sử mỗi đỉnh được gán một vị trí trên mặt phẳng và hàm heuristic được tính bằng khoảng cách Euclidean đến đỉnh đích F. Giả sử đỉnh F có tọa độ (10,0) và các đỉnh còn lại có tọa độ:

$$B = (0,0), \quad C = (3,0), \quad D = (8,0), \quad E = (4,0). \quad (2.36)$$

Khi đó giá trị heuristic được xác định theo công thức:

$$h(B) = \sqrt{(10 - 0)^2} = 10, \quad (2.37)$$

$$h(C) = \sqrt{(10 - 3)^2} = 7, \quad (2.38)$$

$$h(D) = \sqrt{(10 - 8)^2} = 2, \quad (2.39)$$

$$h(E) = \sqrt{(10 - 4)^2} = 6, \quad (2.40)$$

$$h(F) = 0. \quad (2.41)$$

Ban đầu, đỉnh A được đưa vào danh sách mở (*open list*) với chi phí tích lũy bằng 0. Từ đỉnh A , thuật toán có hai lựa chọn là B và C .

Đối với đỉnh B :

$$g(B) = 4, \quad (2.42)$$

$$f(B) = g(B) + h(B) = 4 + 10 = 14. \quad (2.43)$$

Đối với đỉnh C :

$$g(C) = 5, \quad (2.44)$$

$$f(C) = g(C) + h(C) = 5 + 7 = 12. \quad (2.45)$$

Mặc dù chi phí thực tế để đi tới C lớn hơn B , thuật toán A^* vẫn ưu tiên mở rộng C do có giá trị $f(C)$ nhỏ hơn. Điều này cho thấy vai trò của hàm heuristic trong việc định hướng quá trình tìm kiếm về phía mục tiêu thay vì chỉ xét chi phí đã đi.

Từ đỉnh C , thuật toán mở rộng đến đỉnh E :

$$g(E) = g(C) + 3 = 8, \quad (2.46)$$

$$f(E) = g(E) + h(E) = 8 + 6 = 14. \quad (2.47)$$

Tiếp theo, từ đỉnh E đến đỉnh đích:

$$g(F) = g(E) + 6 = 14, \quad (2.48)$$

$$f(F) = g(F) + h(F) = 14 + 0 = 14. \quad (2.49)$$

Do đó, đường đi được lựa chọn là:

$$A \rightarrow C \rightarrow E \rightarrow F, \quad (2.50)$$

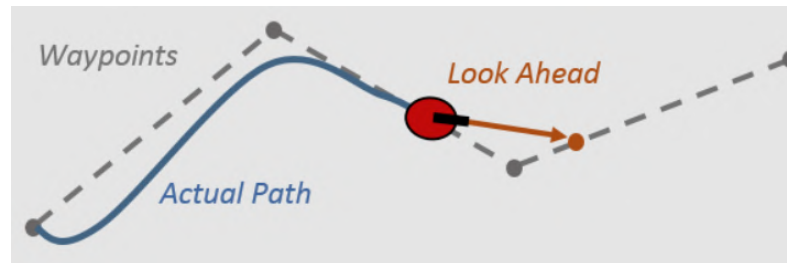
với tổng chi phí bằng 14.

Thuật toán A* tìm được đường đi tối ưu này bằng cách kết hợp giữa chi phí thực tế đã đi $g(n)$ và chi phí ước lượng đến đích $h(n)$, thay vì mở rộng đồng đều mọi hướng như thuật toán Dijkstra. Nhờ sử dụng heuristic, thuật toán có xu hướng ưu tiên các đỉnh nằm gần mục tiêu hơn, từ đó giảm số lượng đỉnh cần khảo sát trong quá trình tìm kiếm.

Trong hệ thống Nav2, mỗi ô trên global costmap có thể được xem như một đỉnh của đồ thị. Chi phí $g(n)$ được tính từ tổng chi phí của các ô đã đi qua, bao gồm cả chi phí di chuyển và chi phí của từng ô trên costmap. Trong khi đó, $h(n)$ thường được xác định bằng khoảng cách Euclidean từ ô hiện tại đến vị trí mục tiêu. Nhờ vậy, A* có khả năng tìm kiếm theo hướng đích đến và giảm đáng kể số lượng ô cần duyệt so với Dijkstra, đồng thời vẫn đảm bảo tìm được đường đi có chi phí thấp trên bản đồ.

2.4.7 Điều khiển bám quỹ đạo cục bộ bằng Regulated Pure Pursuit

Đường đi toàn cục là một chuỗi pose trong hệ tọa độ bản đồ. Để robot thực sự di chuyển theo đường này, Controller Server của Navigation2 gọi một controller plugin nhằm tạo lệnh vận tốc tuyến tính và vận tốc góc ở mỗi chu kỳ điều khiển. Đề tài sử dụng Regulated Pure Pursuit (RPP), một biến thể của Pure Pursuit được thiết kế cho robot dịch vụ và robot công nghiệp, trong đó vận tốc được điều tiết theo độ cong quỹ đạo, khoảng cách tới vật cản và khoảng cách tới đích như minh họa hình 2.4.7 [38, 33].



Hình 2.4.7: Nguyên lý lựa chọn điểm nhìn trước trong bộ điều khiển Pure Pursuit [8]

Nguyên lý cơ bản của Pure Pursuit là chọn một điểm nằm phía trước robot trên đường đi, gọi là điểm nhìn trước (*lookahead point*). Giả sử điểm này sau khi được biến đổi về

hệ tọa độ thân robot có tọa độ (x_L, y_L) và cách robot một khoảng L_d , độ cong hình học của cung tròn đi qua điểm nhìn trước được xác định bởi:

$$\kappa = \frac{2y_L}{L_d^2}, \quad (2.51)$$

trong đó y_L là độ lệch ngang của điểm nhìn trước. Với robot bánh vi sai, lệnh vận tốc góc có thể được tính từ $\omega = v\kappa$. Khoảng nhìn trước nhỏ giúp robot phản ứng nhanh với sai lệch đường đi nhưng có thể gây dao động khi hệ thống có trễ; khoảng nhìn trước lớn tạo chuyển động mượt hơn nhưng có thể làm robot cắt góc tại các đoạn cong gấp.

RPP bổ sung các cơ chế điều tiết chính sau :

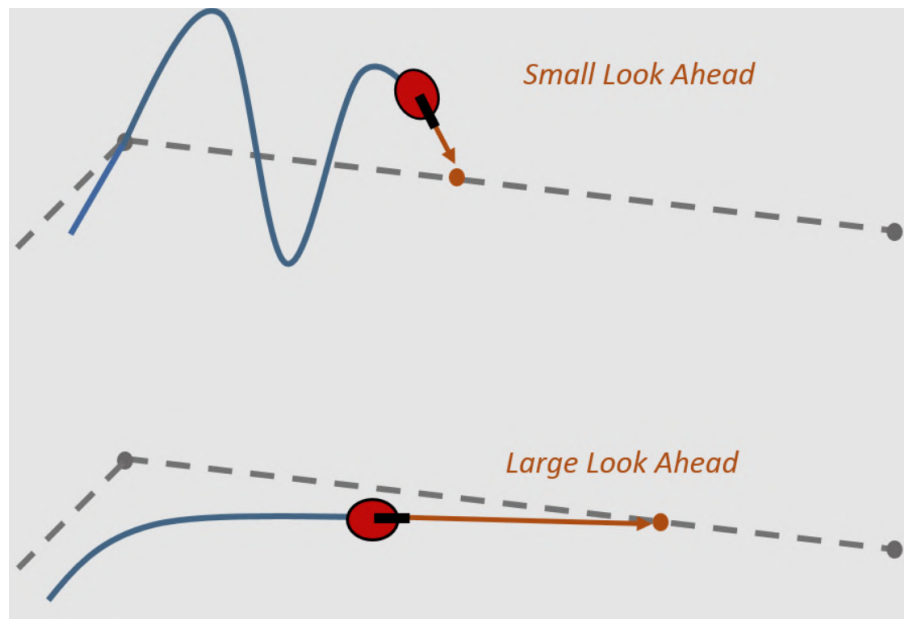
- **Khoảng nhìn trước thích ứng theo vận tốc:** Khi bật chế độ điều chỉnh theo vận tốc, khoảng nhìn trước được xác định gần đúng bởi

$$L_d = \text{clip}(vt_{la}, L_{min}, L_{max}), \quad (2.52)$$

trong đó t_{la} là thời gian nhìn trước và hàm clip giới hạn kết quả trong khoảng cầu hình. Trong cấu hình thử nghiệm của đề tài, $t_{la} = 1,5s$, $L_{min} = 0,3m$ và $L_{max} = 0,9m$.

- **Điều tiết vận tốc theo độ cong:** Khi bán kính cong nhỏ, robot cần quay gấp hơn; RPP giảm vận tốc tuyến tính để hạn chế vượt quá quỹ đạo và giảm nguy cơ va chạm tại góc khuất. Cơ chế này không thay đổi đường toàn cục mà thay đổi tốc độ bám đường.
- **Điều tiết theo chi phí vật cản:** Controller có thể giảm vận tốc khi robot đi vào vùng có chi phí cao trên local costmap. Điều này giúp robot di chuyển thận trọng hơn gần tường hoặc vật cản, nhưng không đồng nghĩa controller tự sinh một đường vòng mới quanh vật cản.
- **Xoay theo hướng đường đi:** Tùy cấu hình, robot vi sai có thể xoay tại chỗ khi sai lệch hướng lớn trước khi tiếp tục bám đường. Trong hệ thống thử nghiệm, ngưỡng xoay, vận tốc góc và gia tốc góc được cấu hình lần lượt khoảng $0,785\text{ rad}$, $0,8\text{ rad/s}$ và $3,2\text{ rad/s}^2$. Đây là tham số triển khai của đề tài, không phải hằng số cố định của thuật toán.
- **Giảm tốc khi tiếp cận đích:** RPP có cơ chế giảm vận tốc theo khoảng cách còn lại của đường đi. Trong cấu hình thử nghiệm, vận tốc mong muốn là $0,2\text{ m/s}$, khoảng giảm tốc gần đích là $0,6m$ và vận tốc tối thiểu được đặt để hạn chế tình trạng động cơ không thắng được ma sát tĩnh.

- **Kiểm tra va chạm theo cung chuyển động dự đoán:** Khi bắt phát hiện va chạm, controller chiếu footprint của robot theo cung chuyển động trong một khoảng thời gian cấu hình. Nếu một pose dự đoán giao với vùng va chạm trên costmap, controller không phát lệnh tiến theo cung đó. Đây là lớp bảo vệ phần mềm của controller, không thay thế nút dừng khẩn cấp phần cứng.



Hình 2.4.8: Ảnh hưởng của khoảng nhìn trước đến hành vi bám quỹ đạo [8]

Hình 2.4.8 minh họa sự đánh đổi khi lựa chọn L_d . Khoảng nhìn trước quá nhỏ có thể khiến robot liên tục hiệu chỉnh và dao động quanh đường tham chiếu, trong khi giá trị quá lớn làm phản ứng chậm và tăng xu hướng cắt góc. Vì vậy, các tham số cần được hiệu chỉnh đồng thời với tốc độ điều khiển, tần số odometry, kích thước robot và hình học môi trường.

Cần phân biệt vai trò của controller và cơ chế xác nhận hoàn thành mục tiêu. RPP tạo lệnh vận tốc để bám đường, còn việc kết luận robot đã đến đích do *goal checker* của Controller Server thực hiện. Với SimpleGoalChecker, mục tiêu được xem là hoàn thành khi sai số vị trí và sai số góc nằm trong các dung sai cấu hình. Do đó, không cần mô tả một thuật toán ngưỡng khoảng cách riêng như một phần bắt buộc của RPP.

RPP phù hợp với yêu cầu bám đường chính xác của robot hướng dẫn viên, nhưng không phải controller chuyên chủ động lệch khỏi đường để tránh vật cản động. Khi đường bị chặn, hệ thống cần dựa vào cập nhật costmap, lập kế hoạch lại và Behavior Tree của Nav2. Đối với yêu cầu an toàn cao hơn, Nav2 còn cung cấp Collision Monitor như một lớp phần mềm độc lập nằm gần đầu ra vận tốc; tuy nhiên cả RPP và Collision

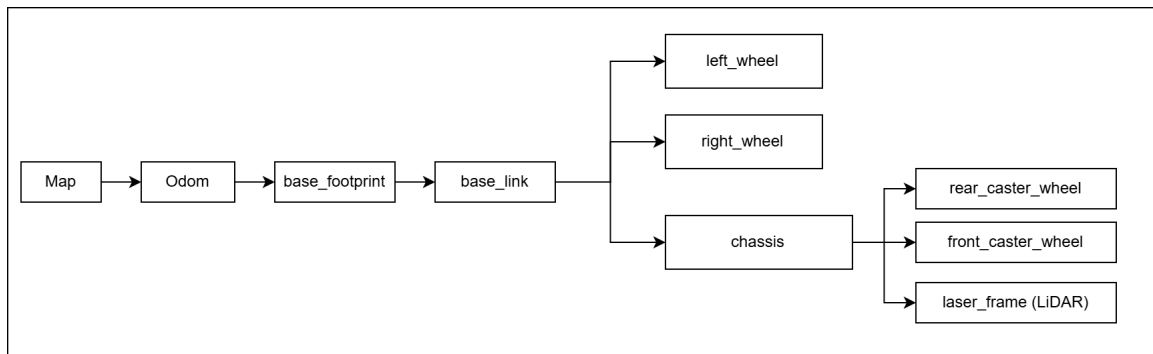
Monitor đều không thay thế mạch dừng khẩn cấp vật lý [33].

2.4.8 Thiết kế cây tọa độ TF trong ROS 2

Trong ROS 2, thư viện tf2 duy trì quan hệ vị trí và hướng giữa nhiều hệ tọa độ theo thời gian, cho phép dữ liệu cảm biến và trạng thái robot được biến đổi về hệ quy chiếu cần thiết [39]. Nhờ đó, dữ liệu từ cảm biến, bộ điều khiển và các thuật toán điều hướng có thể được quy đổi về cùng một hệ quy chiếu để xử lý.

Đối với robot Wall-E, cây tọa độ được thiết kế như Hình 2.4.9. Frame map đại diện cho hệ tọa độ toàn cục của môi trường. Frame odom biểu diễn vị trí tương đối của robot được tính từ dữ liệu encoder. Frame base_footprint là hình chiếu của robot xuống mặt sàn, trong khi base_link được đặt tại tâm trục hai bánh chủ động và được sử dụng làm hệ quy chiếu chính cho động học robot.

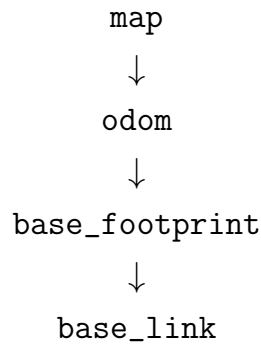
Hai bánh chủ động left_wheel và right_wheel được liên kết trực tiếp với base_link thông qua các khớp quay liên tục (*continuous joint*). Thân xe được biểu diễn bởi frame chassis, đóng vai trò là frame trung gian dùng để gắn các thành phần cơ khí và cảm biến. Các bánh phụ và cảm biến LiDAR được gắn với frame chassis thông qua các khớp cố định (*fixed joint*). Cấu trúc này phản ánh đúng hình học thực tế của robot đồng thời giúp việc mở rộng hệ thống trong tương lai trở nên thuận tiện hơn.



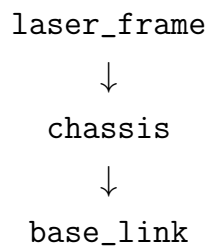
Hình 2.4.9: Cây tọa độ TF của robot Wall-E

Cây TF không biểu diễn luồng dữ liệu giữa các node mà biểu diễn mối quan hệ tọa độ giữa các frame. Khi cần chuyển đổi tọa độ giữa hai frame bất kỳ, hệ thống sẽ tìm đường đi duy nhất trên cây TF và ghép các transform trung gian để tính toán transform cần thiết. Đối với bài toán định vị (Localization), mục tiêu là xác định vị trí hiện tại của robot trên bản đồ. Thuật toán AMCL subscribe topic /scan – dữ liệu quét LiDAR được ghi nhận trong frame laser_frame – và sử dụng hệ thống TF để chiếu các điểm quét về frame base_footprint trước khi so sánh với bản đồ môi trường. Kết quả so sánh được dùng để hiệu chỉnh transform map → odom. Sau đó vị trí robot trên bản đồ được xác

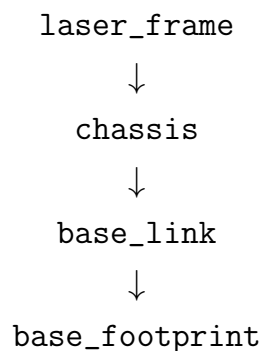
định thông qua chuỗi transform:

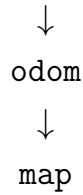


Trong quá trình đo quãng đường di chuyển (Odometry), dữ liệu encoder được sử dụng để cập nhật transform $\text{odom} \rightarrow \text{base_footprint}$. Transform này phản ánh chuyển động tương đối của robot kể từ thời điểm khởi động và được cập nhật liên tục với tần số cao. Đối với bài toán lập kế hoạch đường đi toàn cục (Global Planning), bộ lập kế hoạch cần biết vị trí hiện tại của robot trong hệ tọa độ bản đồ. Thông tin này được xác định thông qua transform giữa map và base_link . Từ vị trí hiện tại và vị trí đích, thuật toán tìm đường sẽ tạo ra quỹ đạo toàn cục trên bản đồ. Trong quá trình tránh vật cản cục bộ (Local Planning), dữ liệu từ LiDAR được đo trong frame laser_frame . Các điểm đo này được chuyển đổi về hệ tọa độ của robot hoặc hệ tọa độ bản đồ thông qua cây TF. Ví dụ, để xác định vị trí của vật cản trong hệ tọa độ robot, hệ thống sử dụng chuỗi transform:



Trong khi đó, để biểu diễn vật cản trên bản đồ, hệ thống sử dụng chuỗi transform:





Nhờ cơ chế này, mọi thành phần trong hệ thống đều có thể quy đổi dữ liệu về cùng một hệ quy chiếu, tạo nền tảng cho các chức năng định vị, lập kế hoạch đường đi và tránh vật cản của robot.

2.5 TRUYỀN THÔNG THỜI GIAN THỰC GIỮA ROBOT VÀ MÁY CHỦ

Hệ thống robot hướng dẫn viên cần truyền âm thanh hai chiều giữa robot và máy chủ xử lý AI với độ trễ đủ thấp để duy trì nhịp hội thoại. Khác với dữ liệu cấu hình có thể trao đổi theo mô hình yêu cầu–phản hồi, âm thanh cần được truyền liên tục, thích nghi với biến động mạng và cho phép robot cũng như Voice Agent tham gia hoặc rời phiên làm việc độc lập. Đề tài sử dụng WebRTC làm tầng truyền thông đa phương tiện và LiveKit làm nền tảng quản lý phiên, participant và các luồng dữ liệu thời gian thực.

2.5.1 Giao thức WebRTC và nền tảng LiveKit

WebRTC (*Web Real-Time Communication*) là tập hợp API và giao thức cho phép truyền media và dữ liệu ứng dụng theo thời gian thực giữa các thiết bị [40]. Âm thanh WebRTC được chia thành các frame, mã hóa, đóng gói và truyền liên tục; phía nhận sử dụng bộ đệm phát để hạn chế ảnh hưởng của jitter. WebRTC ưu tiên đường truyền UDP nhằm giảm độ trễ, nhưng quá trình thiết lập kết nối phải giải quyết sự khác biệt địa chỉ mạng, NAT và firewall.

Cơ chế Interactive Connectivity Establishment (ICE) thu thập và kiểm tra nhiều candidate để lựa chọn đường truyền khả dụng. ICE sử dụng STUN để xác định địa chỉ công khai và có thể sử dụng TURN làm máy chủ chuyển tiếp khi hai đầu không thể kết nối trực tiếp [41, 42]. Việc sử dụng TURN làm tăng quãng đường truyền và tải máy chủ, nhưng cải thiện khả năng kết nối trong các mạng doanh nghiệp hoặc mạng có NAT đối xứng.

LiveKit xây dựng trên WebRTC và tổ chức phiên giao tiếp bằng ba khái niệm chính: *Room*, *Participant* và *Track*. Room đại diện cho một phiên đang hoạt động; robot, Voice Agent hoặc giao diện kiểm thử có thể tham gia Room như các participant. Mỗi participant có thể publish audio, video hoặc dữ liệu và subscribe có chọn lọc các track của participant khác.

Trong kiến trúc đề tài, robot publish luồng microphone lên Room. Voice Agent

subscribe luồng này để xử lý VAD, STT, LLM và TTS. Âm thanh tổng hợp sau đó được Voice Agent publish thành một audio track mới; robot subscribe track phản hồi và phát qua loa. LiveKit chịu trách nhiệm vận chuyển media và quản lý phiên, trong khi các mô hình AI được triển khai như các dịch vụ xử lý độc lập trên máy chủ.

2.5.2 Audio track, dữ liệu ứng dụng và vai trò của SFU

Trong LiveKit, *TrackPublication* chứa metadata và trạng thái publish của một track, còn *Track* biểu diễn luồng media thực tế mà subscriber nhận được. Việc tách publication khỏi media cho phép participant biết một track tồn tại ngay cả khi chưa subscribe hoặc track đang bị mute.

Đối với hệ thống robot, audio track được sử dụng cho hai hướng truyền chính:

- **Robot đến Voice Agent:** microphone trên robot tạo các frame âm thanh và publish một local audio track. Voice Agent subscribe track này để đưa frame vào pipeline xử lý tiếng nói.
- **Voice Agent đến robot:** TTS tạo các frame âm thanh phản hồi; Voice Agent publish chúng thành audio track và robot subscribe để phát ra loa.

Ngoài media, WebRTC còn hỗ trợ data channel cho dữ liệu ứng dụng. LiveKit cung cấp các API mức cao như text stream, byte stream, RPC, data packet và state synchronization [43]. Trong bài toán robot, kênh dữ liệu có thể mang transcript, trạng thái nghe–nói, mã phiên, thông báo lỗi hoặc lệnh mức cao. Dữ liệu điều khiển quan trọng cần cơ chế xác nhận nên sử dụng chế độ tin cậy hoặc API request–response; dữ liệu telemetry cập nhật liên tục có thể ưu tiên độ trễ thấp và chấp nhận mất một số mẫu. Không nên gửi lệnh PWM hoặc điều khiển động cơ mức thấp trực tiếp từ LLM qua data channel.

LiveKit Server hoạt động theo kiến trúc Selective Forwarding Unit (SFU). SFU nhận track từ publisher và chuyển tiếp đến các subscriber phù hợp thay vì trộn tất cả media thành một luồng duy nhất như MCU. Cách tổ chức này giảm chi phí giải mã–mã hóa lại tại máy chủ và cho phép từng participant lựa chọn track cần nhận. Trong đề tài, SFU chỉ đảm nhiệm định tuyến media thời gian thực; Voice Agent vẫn là một participant riêng thực hiện suy luận AI.

2.5.3 Thiết lập phiên, NAT traversal và khả năng phục hồi kết nối

Trước khi tham gia Room, participant nhận access token chứa danh tính và quyền publish/subscribe. Client thiết lập kết nối signaling bảo mật tới LiveKit Server, trao đổi thông tin phiên và metadata track, sau đó thực hiện ICE để lựa chọn đường truyền media.

Trong điều kiện thuận lợi, media được truyền qua UDP; khi UDP không khả dụng, hệ thống có thể chuyển sang WebRTC over TCP hoặc TURN tùy cấu hình hạ tầng [43].

Độ ổn định hội thoại chịu ảnh hưởng bởi băng thông, packet loss, jitter, thời gian khứ hồi và việc chuyển đổi giữa các candidate. Vì robot di chuyển trong không gian trong nhà, chất lượng Wi-Fi có thể thay đổi theo vị trí. Ứng dụng robot cần theo dõi trạng thái Room, chất lượng kết nối, sự kiện track bị mute hoặc participant rời Room và thực hiện reconnect khi cần. Khi Voice Agent mất kết nối, tầng điều hướng vẫn phải hoạt động độc lập; việc mất dịch vụ hội thoại không được làm robot tiếp tục thực hiện một lệnh chưa được xác nhận.

2.6 XỬ LÝ TIẾNG NÓI VÀ MÔ HÌNH AI HỘI THOẠI

Hệ thống sử dụng pipeline dạng nối tầng STT–LLM–TTS. Âm thanh người dùng được phân đoạn thành lượt nói, chuyển thành văn bản, kết hợp với ngữ cảnh của địa điểm và trạng thái tour, sau đó mô hình ngôn ngữ sinh phản hồi. Văn bản phản hồi được tổng hợp thành tiếng nói và truyền lại robot. Kiến trúc nối tầng cho phép thay thế từng mô hình độc lập, lưu transcript để đánh giá và tách rõ phần hiểu lời nói, suy luận ngôn ngữ và phát âm [43].

2.6.1 Phát hiện hoạt động giọng nói và xác định lượt hội thoại

Voice Activity Detection (VAD) là bài toán phân loại một frame âm thanh ngắn thành hai trạng thái: có tiếng nói hoặc không có tiếng nói. Các phương pháp VAD truyền thống sử dụng năng lượng, đặc trưng phổ và mô hình xác suất; các phương pháp hiện đại có thể sử dụng mạng nơ-ron để tăng khả năng chống nhiễu. Công trình của Sohn và cộng sự là một nền tảng tiêu biểu cho VAD dựa trên kiểm định tỉ số khả năng và cơ chế *hang-over* nhằm tránh cắt mất phần cuối của lời nói [44].

VAD không thực hiện chuyển giọng nói thành văn bản. Vai trò của nó là xác định vùng âm thanh có khả năng chứa tiếng nói và hỗ trợ endpointing. Một pipeline thực tế thường cần các tham số:

- ngưỡng xác suất hoặc năng lượng để bắt đầu phát hiện tiếng nói;
- thời lượng nói tối thiểu để loại các âm thanh ngắn không có nghĩa;
- thời lượng im lặng tối thiểu để kết thúc một lượt nói;
- khoảng đệm trước và sau lời nói để hạn chế mất âm đầu hoặc âm cuối.

Nếu thời lượng im lặng đặt quá ngắn, câu hỏi có thể bị cắt tại các khoảng ngừng; nếu đặt quá dài, độ trễ phản hồi tăng. LiveKit Agents kết hợp VAD với cơ chế

turn detection và endpointing để quyết định khi nào lượt nói của người dùng thực sự hoàn thành. Hệ thống cũng cần xử lý *barge-in*: khi người dùng bắt đầu nói trong lúc robot đang phát âm thanh, phản hồi hiện tại có thể được tạm dừng hoặc hủy để nhường lượt cho người dùng.

2.6.2 Chuyển đổi tiếng nói thành văn bản

Speech-to-Text (STT), còn gọi là Automatic Speech Recognition (ASR), nhận chuỗi mẫu âm thanh và sinh chuỗi token văn bản tương ứng. Hệ thống streaming có thể tạo transcript tạm thời trong khi người dùng đang nói và phát transcript cuối sau khi endpoint được xác định. Transcript tạm thời giúp giao diện phản hồi nhanh, nhưng chỉ transcript cuối nên được dùng cho các lệnh điều khiển hoặc đánh giá chính thức.

Whisper là một ví dụ tiêu biểu cho mô hình ASR đa ngôn ngữ được huấn luyện trên tập dữ liệu âm thanh quy mô lớn và có khả năng khái quát trong nhiều điều kiện ghi âm [45]. Việc trích dẫn Whisper ở đây nhằm minh họa nguyên lý và xu hướng mô hình STT hiện đại; tên mô hình cụ thể được triển khai trong đề tài cần được trình bày ở Chương 3 cùng phiên bản và cấu hình suy luận.

Độ chính xác STT thường được đánh giá bằng Word Error Rate (WER):

$$\text{WER} = \frac{S + D + I}{N}, \quad (2.53)$$

trong đó S , D và I lần lượt là số từ bị thay thế, bị xóa và được chèn, còn N là tổng số từ trong câu tham chiếu. Với tiếng Việt, việc tách từ có thể làm WER phụ thuộc công cụ tokenizer; vì vậy báo cáo cũng có thể bổ sung Character Error Rate nếu cần.

Hiệu năng STT trên robot chịu ảnh hưởng bởi khoảng cách người nói, hướng microphone, tiếng vọng từ loa, tiếng ồn quạt và động cơ, chất lượng mạng, giọng vùng miền và từ chuyên ngành. Cơ chế khử vọng âm học và kiểm soát mức âm lượng loa có ý nghĩa đặc biệt vì microphone và loa được đặt trên cùng thân robot.

2.6.3 Mô hình ngôn ngữ lớn và hội thoại theo ngữ cảnh vị trí

Các mô hình ngôn ngữ lớn hiện đại chủ yếu dựa trên kiến trúc Transformer, sử dụng self-attention để mô hình hóa quan hệ giữa các token trong chuỗi [46]. Với đầu vào c biểu diễn ngữ cảnh và chuỗi phản hồi $y_1, \dots, y_T$, mô hình sinh tự hồi quy có thể được mô tả bởi:

$$P(\mathbf{y} \mid c) = \prod_{t=1}^T P(y_t \mid y_{<t}, c). \quad (2.54)$$

Trong robot hướng dẫn viên, câu hỏi giống nhau có thể cần câu trả lời khác nhau tùy vị trí. Vì vậy, đầu vào của LLM không chỉ gồm transcript mà còn gồm một gói ngữ cảnh

có cấu trúc:

- **Ngữ cảnh toàn cục:** vai trò của robot, thông tin chung về địa điểm, quy tắc trả lời và các giới hạn an toàn;
- **Ngữ cảnh khu vực:** mô tả Area mà robot đang đứng hoặc đang phục vụ;
- **Ngữ cảnh điểm tham quan:** nội dung cụ thể của Point/POI, kịch bản giới thiệu và các dữ kiện được phép sử dụng;
- **Ngữ cảnh vận hành:** bản đồ hiện tại, Area, Point, trạng thái navigation, trạng thái tour và lượt hội thoại gần nhất;
- **Yêu cầu người dùng:** transcript cuối cùng từ STT.

Thứ tự ưu tiên ngữ cảnh cần được xác định rõ: quy tắc hệ thống và an toàn có mức ưu tiên cao hơn yêu cầu người dùng; thông tin của Point hiện tại cụ thể hơn Area; Area cụ thể hơn ngữ cảnh toàn cục. Khi câu hỏi vượt ngoài dữ liệu được cung cấp, mô hình nên thừa nhận giới hạn thay vì tự suy diễn. Cách tổ chức này giảm nguy cơ trả lời sai khu vực và hỗ trợ tái sử dụng cùng một Voice Agent cho nhiều bản đồ khác nhau.

Lệnh điều khiển tour cần được phân biệt với câu hỏi thông tin. LLM chỉ nên sinh một ý định mức cao có cấu trúc, ví dụ `CONTINUE_TOUR`, `STOP_TOUR` hoặc `NAVIGATE_TO_POINT`, kèm tham số cần thiết. Một mô-đun kiểm tra độc lập xác thực trạng thái, quyền và điểm đích trước khi gửi goal cho Nav2. Việc tách phản hồi ngôn ngữ khỏi lệnh thực thi giúp tránh trường hợp văn bản tự do của mô hình được diễn giải trực tiếp thành lệnh động cơ.

2.6.4 Chuyển đổi văn bản thành tiếng nói

Text-to-Speech (TTS) chuyển văn bản phản hồi thành waveform âm thanh. Các hệ thống truyền thống thường gồm mô-đun chuẩn hóa văn bản, mô hình âm học sinh đặc trưng phổ và vocoder chuyển đặc trưng thành waveform. Các mô hình end-to-end như VITS kết hợp mô hình biến phân, normalizing flow và huấn luyện đối kháng để tổng hợp tiếng nói trực tiếp với chất lượng tự nhiên cao [47].

Trong ứng dụng hội thoại, chỉ chất lượng phát âm chưa đủ; thời gian tới frame âm thanh đầu tiên cũng rất quan trọng. TTS streaming có thể tổng hợp theo câu hoặc theo đoạn ngắn ngay khi LLM tạo được phần đầu phản hồi. Tuy nhiên, chia văn bản quá nhỏ có thể làm ngữ điệu thiếu tự nhiên hoặc phát âm sai khi câu chưa hoàn chỉnh. Vì vậy, pipeline cần cân bằng giữa kích thước đoạn, dấu câu, độ trễ và tính tự nhiên của giọng nói.

Mô-đun TTS cũng cần chuẩn hóa số, đơn vị, tên viết tắt và thuật ngữ tiếng Anh xuất hiện trong nội dung tham quan. Đối với tên thiết bị hoặc tên riêng, hệ thống có thể cần

từ điển phát âm hoặc cách viết thay thế để giọng tổng hợp dễ hiểu hơn.

2.6.5 Pipeline hội thoại đầu cuối và các thành phần độ trễ

LiveKit Agents hỗ trợ pipeline STT–LLM–TTS theo kiểu streaming, trong đó các giai đoạn có thể chồng lấp một phần để giảm thời gian chờ. Đối với cách đo từ thời điểm người dùng kết thúc câu hỏi đến khi robot phát frame âm thanh đầu tiên, độ trễ có thể được phân tích gần đúng:

$$T_{resp} \approx T_{endpoint} + T_{STT} + T_{context} + T_{LLM}^{first} + T_{TTS}^{first} + T_{net} + T_{payout}, \quad (2.55)$$

trong đó $T_{endpoint}$ là thời gian xác nhận kết thúc lượt nói; T_{LLM}^{first} là thời gian đến token đầu tiên; T_{TTS}^{first} là thời gian đến frame âm thanh đầu tiên. Vì các giai đoạn streaming có thể chạy chồng lấp, biểu thức trên dùng để xác định nguồn gây trễ chứ không nhất thiết bằng tổng thời gian xử lý toàn bộ phản hồi.

Các biện pháp giảm độ trễ gồm điều chỉnh endpointing phù hợp, sử dụng STT streaming, giới hạn kích thước ngữ cảnh gửi LLM, sinh phản hồi ngắn gọn, phát TTS theo đoạn và đặt dịch vụ AI gần hạ tầng LiveKit về mặt mạng. Việc đo riêng từng thành phần sẽ giúp xác định độ trễ đến từ mạng, STT, LLM hay TTS thay vì chỉ ghi nhận một giá trị tổng.

2.7 CƠ SỞ CẤU HÌNH TOUR VÀ GIAO DIỆN QUẢN LÝ HỆ THỐNG

Web Admin cung cấp lớp cấu hình và vận hành ở mức ứng dụng, tách khỏi công cụ phát triển RViz2. Mục tiêu của giao diện không phải thay thế toàn bộ ROS 2, mà chuyển các dữ liệu kỹ thuật như bản đồ, pose và navigation goal thành các đối tượng có ý nghĩa đối với người quản trị tour: Map, Area, Point, Context và Tour.

2.7.1 Mô hình dữ liệu Map–Area–Point–Tour

Bản đồ điều hướng được biểu diễn bằng metadata chứa độ phân giải, kích thước và pose gốc của bản đồ [48, 49]. Khi lưu bản đồ, Map Server thường quản lý ảnh raster cùng tệp YAML mô tả đường dẫn ảnh, resolution, origin và các ngưỡng phân loại ô trống/chiếm dụng.

Trên nền bản đồ hình học, hệ thống bổ sung các lớp dữ liệu ngữ nghĩa:

- **Map:** tập tin bản đồ, metadata, trạng thái Draft/Active và phiên bản cấu hình;
- **Area:** một vùng có ý nghĩa trong môi trường, có thể biểu diễn bằng polygon hoặc tập biên; mỗi Area chứa ngữ cảnh dùng chung;

- **Point/POI:** một pose mục tiêu (x, y, θ) , bán kính đến điểm, nội dung giới thiệu và ngữ cảnh chi tiết;
- **Tour:** danh sách Point có thứ tự cùng quy tắc chuyển tiếp, dừng hoặc bỏ qua;
- **Runtime State:** map, area, point hiện tại, pose robot, navigation state và voice state.

Mô hình này cho phép cùng một phần mềm robot được tái cấu hình cho thư viện, bảo tàng, khu triển lãm hoặc phòng thí nghiệm bằng cách thay bản đồ và dữ liệu ngữ cảnh, không cần thay đổi thuật toán điều hướng cốt lõi.

2.7.2 Chuyển đổi tọa độ ảnh bản đồ và tọa độ ROS 2

Giao diện Studio hiển thị ảnh bản đồ theo tọa độ pixel, trong khi Nav2 nhận pose theo đơn vị mét trong frame map. Giả sử ảnh có chiều cao H pixel, điểm được chọn có cột u tính từ trái sang và hàng v tính từ trên xuống, độ phân giải bản đồ là s mét/pixel, origin là (x_0, y_0, θ_0) . Tọa độ cục bộ của pixel so với góc dưới trái ảnh được xác định bởi:

$$\mathbf{p}_{img} = \begin{bmatrix} us \\ (H - 1 - v)s \end{bmatrix}. \quad (2.56)$$

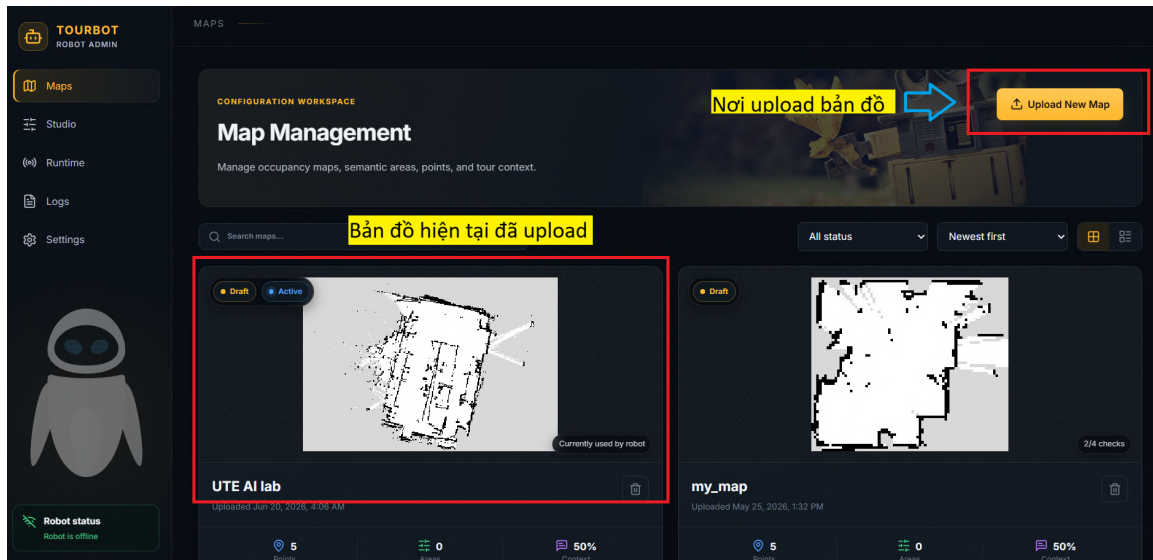
Tọa độ trong frame map là:

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} + \begin{bmatrix} \cos \theta_0 & -\sin \theta_0 \\ \sin \theta_0 & \cos \theta_0 \end{bmatrix} \mathbf{p}_{img}. \quad (2.57)$$

Đối với bản đồ phổ biến có $\theta_0 = 0$, công thức rút gọn thành $x = x_0 + us$ và $y = y_0 + (H - 1 - v)s$. Phép đảo trục v là cần thiết vì trục dọc của ảnh tăng từ trên xuống, trong khi trục y của bản đồ tăng theo quy ước tọa độ Cartesian. Khi giao diện bị co giãn để phù hợp khung hiển thị, tọa độ chuột phải được quy đổi về kích thước ảnh gốc trước khi áp dụng công thức trên.

2.7.3 Giao diện quản lý bản đồ

Màn hình quản lý bản đồ là lớp giao tiếp đầu tiên trong quy trình cấu hình robot. Phân hệ này lưu trữ ảnh bản đồ, tệp metadata và các cấu hình ngữ nghĩa gắn với từng bản đồ. Giao diện quản lý bản đồ được minh họa trong Hình 2.7.1.

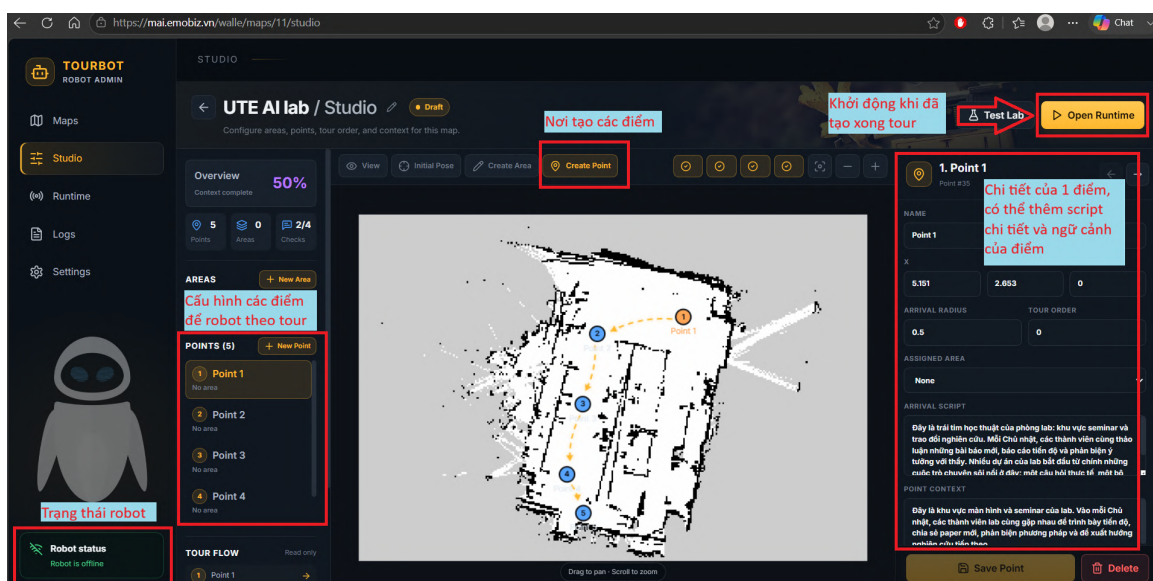


Hình 2.7.1: Giao diện quản lý bản đồ trên nền tảng Web Admin

Các chức năng chính gồm tải lên ảnh và YAML từ SLAM Toolbox, hiển thị danh sách bản đồ, chỉnh sửa metadata, xóa cấu hình không còn sử dụng và lựa chọn một bản đồ Active cho phiên vận hành. Khi thay đổi bản đồ Active, hệ thống cần kiểm tra tính tương thích giữa bản đồ, cấu hình Area/Point và trạng thái định vị hiện tại trước khi cho phép robot nhận goal mới.

2.7.4 Giao diện thiết kế khu vực, điểm tham quan và lộ trình

Sau khi lựa chọn bản đồ, quản trị viên sử dụng Studio để chuyển tọa độ hình học thành đối tượng tham quan có ngữ nghĩa. Giao diện Studio được trình bày trong Hình 2.7.2.



Hình 2.7.2: Giao diện thiết kế khu vực, điểm tham quan và ngữ cảnh tại Studio

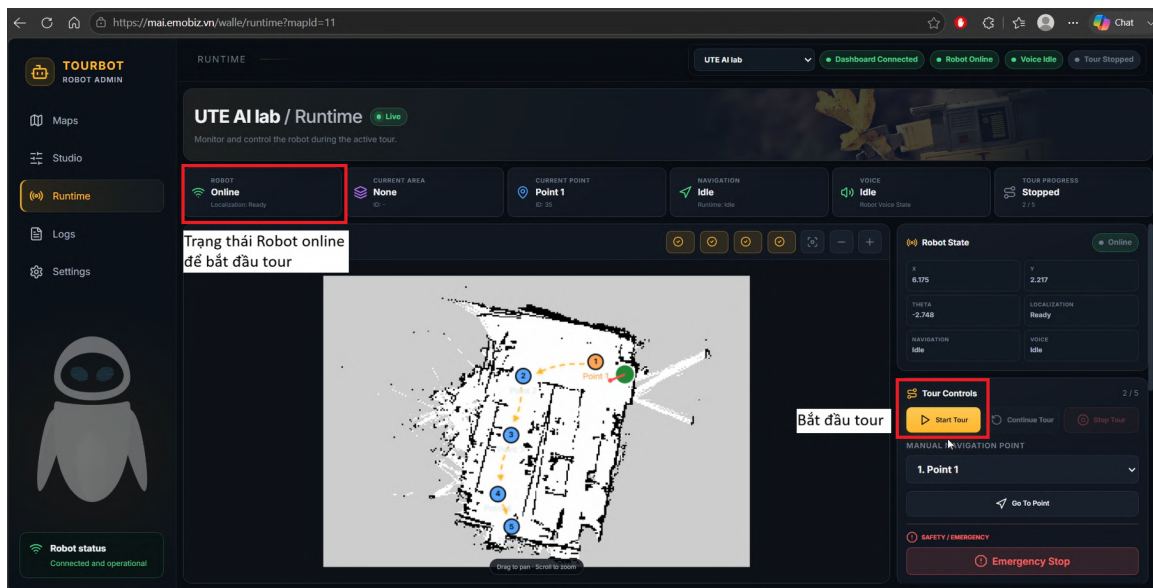
Studio có hỗ trợ:

- Tạo và chỉnh sửa Area trên bản đồ, nhập Area Context;
- Tạo Point/POI và cấu hình pose (x, y, θ) , arrival radius, tên hiển thị;
- Nhập Arrival Script dùng để giới thiệu tự động khi robot đến điểm;
- Nhập Point Context dùng để trả lời câu hỏi tại vị trí hiện tại;
- Sắp xếp các Point thành Tour Flow và kiểm tra điểm có nằm trong vùng điều hướng hợp lệ hay không;
- Thử câu hỏi với context của từng Area/Point trước khi đưa cấu hình vào vận hành.

Việc tách Arrival Script và Point Context là cần thiết: script là nội dung chủ động, tương đối ổn định và có thể được tạo audio trước; context là tập dữ liệu cung cấp cho LLM để sinh câu trả lời linh hoạt theo câu hỏi.

2.7.5 Giao diện giám sát và điều khiển thời gian thực

Runtime Dashboard đóng vai trò trung tâm vận hành và hiển thị trạng thái hệ thống, như Hình 2.7.3.



Hình 2.7.3: Giao diện vận hành và giám sát robot thời gian thực

Các nhóm thông tin cần hiển thị gồm trạng thái online/offline, localization, pin và tài nguyên máy tính; pose (x, y, θ) ; Map, Area và Point hiện tại; tiến độ tour; câu hỏi của người dùng; transcript và câu trả lời của robot. Các lệnh mức cao gồm Start Tour, Continue Tour, Stop Tour và điều hướng thủ công tới một POI.

Nút *Emergency Stop* trên giao diện nên được mô tả chính xác là cơ chế dừng từ xa bằng phần mềm. Khi được kích hoạt, backend có thể hủy goal Nav2 đang chạy và phát lệnh vận tốc bằng không theo kiến trúc điều khiển của hệ thống. Cơ chế này vẫn phụ thuộc mạng, tiến trình phần mềm và bộ điều khiển; do đó không tương đương nút dừng khẩn cấp vật lý có khả năng cắt hoặc vô hiệu hóa tăng công suất độc lập.

Trạng thái Runtime thay đổi liên tục nên cần một kênh sự kiện hai chiều giữa robot, backend và giao diện, chẳng hạn WebSocket hoặc cơ chế đồng bộ trạng thái tương đương. Các sự kiện điều khiển cần có mã phiên, timestamp và phản hồi xác nhận để tránh thực thi lại lệnh cũ sau khi robot reconnect. Dữ liệu hiển thị có thể được cập nhật theo tần số thấp hơn dữ liệu ROS nội bộ nhằm giảm tải mạng mà vẫn đủ cho người vận hành theo dõi.

Chương 3

THIẾT KẾ HỆ THỐNG

3.1 YÊU CẦU THIẾT KẾ VÀ SƠ ĐỒ KHỐI TỔNG THỂ

3.1.1 Yêu cầu thiết kế

Yêu cầu người dùng

- Robot có khả năng di chuyển tự động trong môi trường phòng Lab theo lịch trình đề ra
- Robot có khả năng dẫn đường và cung cấp thông tin cho người dùng bằng âm thanh.
- Robot có thể tránh vật cản và di chuyển an toàn trong môi trường có người.
- Robot hỗ trợ tương tác giọng nói cơ bản (nhận lệnh, trả lời thông tin).
- Có thể triển khai trong không gian thực tế với chi phí hợp lý.
- Dễ bảo trì, thay thế linh kiện và nâng cấp trong tương lai.

Yêu cầu kỹ thuật

Chức năng kỹ thuật:

- Xây dựng bản đồ và định vị bằng LiDAR (SLAM).
- Điều hướng tự động và tránh vật cản.
- Đi theo tour đến các vị trí cụ thể được chọn trên bản đồ.
- Phát âm thanh hướng dẫn cho người dùng tại các điểm đến trên bản đồ .
- Nhận diện, xử lý giọng nói và trả lời câu hỏi.

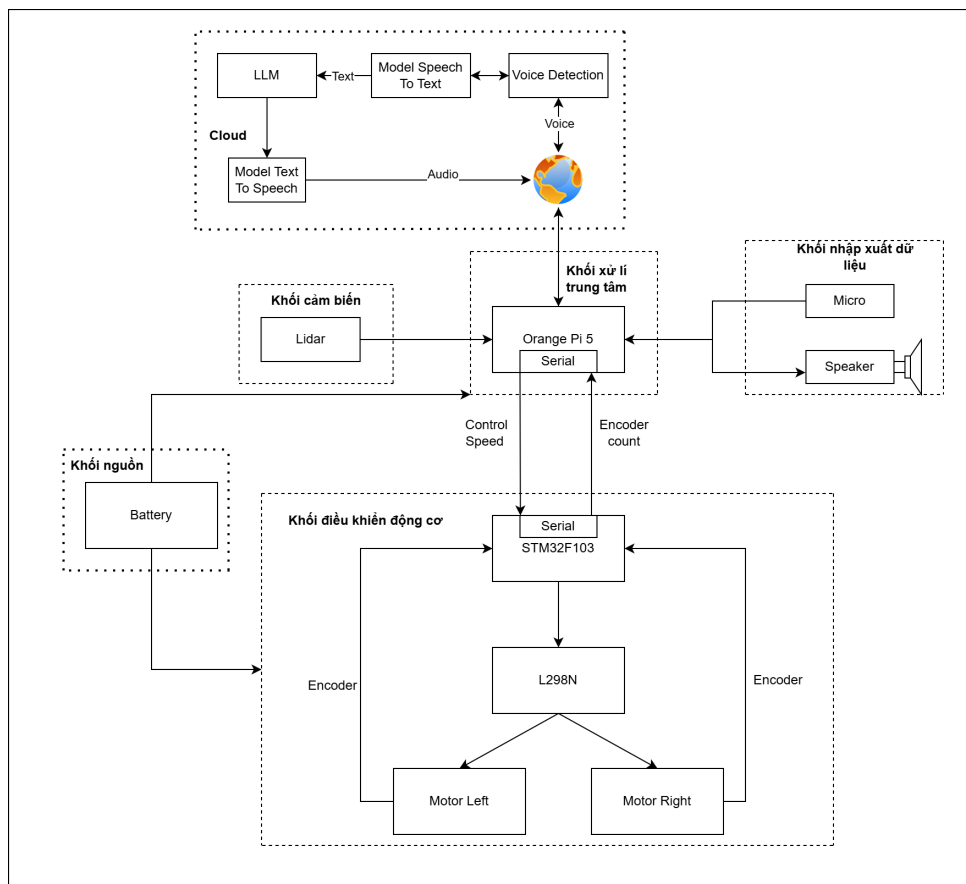
Thông số kỹ thuật:

- Điện áp hoạt động: 5 VDC (5 A).
- Công suất tiêu thụ: 10 W.
- Độ chính xác cảm biến LiDAR: 1%.
- Tốc độ di chuyển tối đa: 0,5 m/s.
- Tốc độ quay tối đa: 1,0 rad/s.
- Tần số cập nhật vị trí: 50 Hz.

- Thời gian sử dụng: 2 giờ.
- Thời gian trễ khi xử lý tín hiệu âm thanh: 10 giây.
- Khối lượng: 3 kg..
- Kích thước: $25 \times 18 \times 13,5$ cm.

3.1.2 Sơ đồ khối tổng thể hệ thống

Sơ đồ khối tổng thể của hệ thống được thể hiện như Hình 3.1.1 sau:



Hình 3.1.1: Sơ đồ khối tổng thể của hệ thống Robot

- **Khối xử lý trung tâm :** Đóng vai trò là bộ não của robot, sử dụng máy tính nhúng Orange Pi 5. Khối này chịu trách nhiệm thu thập và xử lý toàn bộ dữ liệu từ cảm biến LiDAR, điều phối các node chức năng trong hệ sinh thái ROS 2, đồng thời quản lý giao tiếp Internet kết nối với hạ tầng Cloud qua mạng không dây.
- **Khối điều khiển động cơ :** Sử dụng vi điều khiển STM32F103 để đảm nhiệm các tác vụ thời gian thực. Khối này kết nối trực tiếp với mạch cầu H L298N nhằm điều khiển công suất cho hai động cơ trái (Motor Left) và động cơ phải (Motor

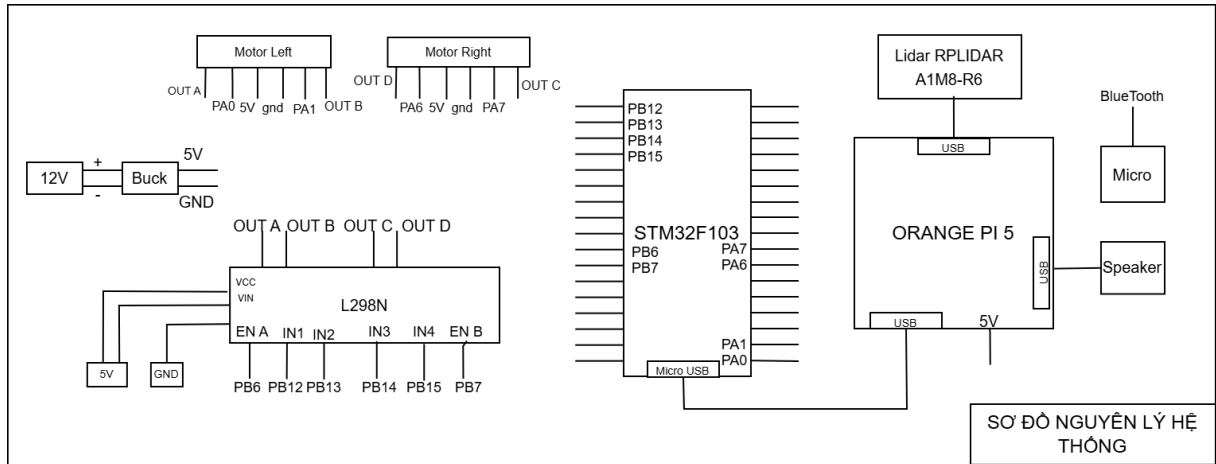
Right). Đồng thời, các tín hiệu phản hồi Encoder trên động cơ được gửi liên tục về vi điều khiển để phục vụ thuật toán kiểm soát vận tốc chính xác. Giao tiếp giữa Orange Pi 5 và STM32F103 được thực hiện thông qua chuẩn truyền thông nối tiếp Serial, trong đó máy tính nhúng gửi lệnh vận tốc và vi điều khiển phản hồi lại giá trị đếm xung.

- **Khối cảm biến :** Thành phần cốt lõi là cảm biến Lidar RPLIDAR A1M8-R6, thực hiện quét không gian xung quanh để cung cấp dữ liệu khoảng cách dạng đám mây điểm về cho Orange Pi 5 nhằm phục vụ bài toán xây dựng bản đồ và tránh vật cản.
- **Khối nhập xuất dữ liệu :** Bao gồm thiết bị thu âm (Micro) và thiết bị phát thanh (Speaker) kết nối trực tiếp với Orange Pi 5, cấu thành giao tiếp ngoại vi hỗ trợ con người tương tác trực tiếp với robot thông qua tiếng nói.
- **Khối xử lý đám mây :** Tích hợp các mô hình trí tuệ nhân tạo nâng cao được triển khai từ xa. Luồng dữ liệu thoại từ Micro qua mạng Internet sẽ được xử lý lần lượt qua khối nhận dạng giọng nói, mô hình chuyển đổi tiếng nói thành văn bản , sau đó truyền dữ liệu văn bản vào mô hình ngôn ngữ lớn để xử lý ngữ nghĩa. Phản hồi từ mô hình sẽ được chuyển đổi ngược lại thành âm thanh thông qua mô hình tổng hợp giọng nói trước khi gửi trả về loa trên robot.
- **Khối nguồn :** Sử dụng khối pin (Battery) để cung cấp năng lượng hoạt động độc lập cho toàn bộ hệ thống phần cứng bao gồm máy tính nhúng Orange Pi 5 và khối điều khiển động cơ.

3.2 THIẾT KẾ TỪNG KHỐI TRONG HỆ THỐNG

3.2.1 Sơ đồ nguyên lý hệ thống

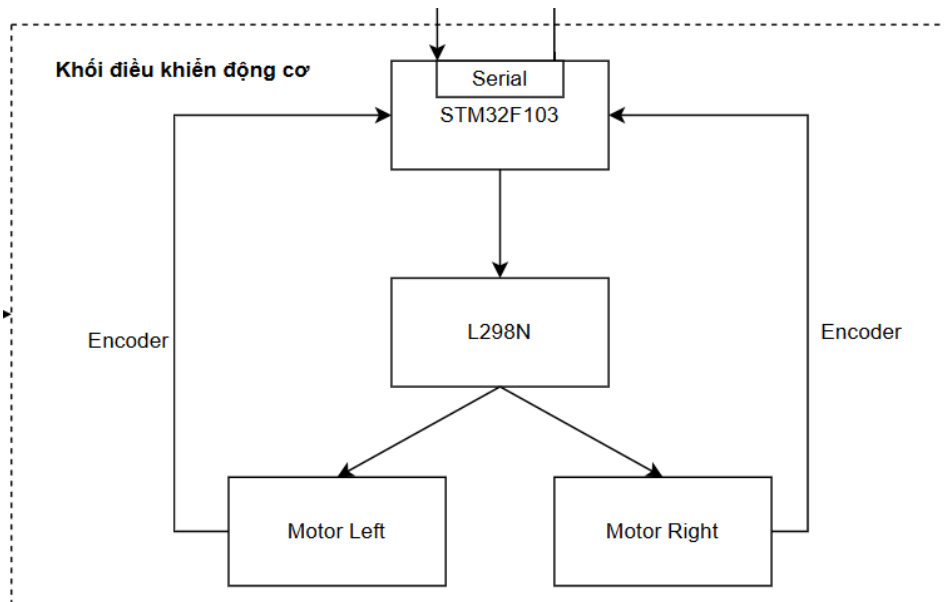
Hình 3.2.1 trình bày sơ đồ nguyên lý của hệ thống robot di động. Nguồn 12 V được hạ áp xuống 5 V thông qua bộ Buck để cấp nguồn cho các thành phần trong hệ thống. STM32F103 thực hiện điều khiển động cơ thông qua mạch cầu H L298N và đọc tín hiệu encoder từ hai bánh xe. Orange Pi 5 đóng vai trò máy tính trung tâm, giao tiếp với STM32F103 qua USB và xử lý dữ liệu từ cảm biến LiDAR RPLIDAR A1M8-R6 phục vụ cho định vị, lập bản đồ và điều hướng. Ngoài ra, hệ thống còn tích hợp Bluetooth và loa để hỗ trợ giao tiếp và phát thông báo.



Hình 3.2.1: Sơ đồ nguyên lý của hệ thống Robot

3.2.2 Khối điều khiển động cơ

Khối điều khiển động cơ có nhiệm vụ nhận lệnh vận tốc từ Orange Pi 5 và điều khiển chuyển động của robot. STM32F103 thực hiện việc tạo tín hiệu điều khiển cho mạch cầu H L298N để điều chỉnh tốc độ và chiều quay của hai động cơ DC. Đồng thời, tín hiệu encoder từ các động cơ được đưa về STM32F103 để đo tốc độ thực tế và phục vụ cho quá trình điều khiển vòng kín.



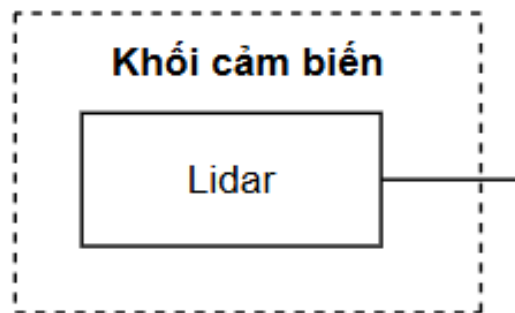
Hình 3.2.2: Khối điều khiển động cơ

Hình 3.2.2 mô tả cấu trúc của khối điều khiển động cơ. STM32F103 giao tiếp với Orange Pi 5 thông qua kết nối Serial, điều khiển mạch L298N để dẫn động hai động cơ trái và phải, đồng thời thu thập dữ liệu encoder để phản hồi trạng thái chuyển động của

robot.

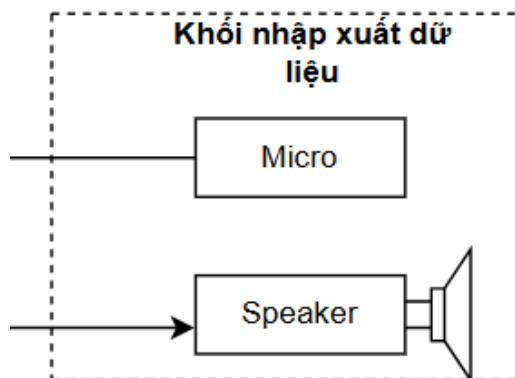
3.2.3 Khối cảm biến và khối nhập xuất dữ liệu

Khối cảm biến và khối nhập xuất dữ liệu đảm nhiệm việc thu thập thông tin từ môi trường cũng như hỗ trợ tương tác giữa robot và người dùng. Trong hệ thống, cảm biến LiDAR được sử dụng để quét môi trường xung quanh, trong khi microphone và loa phục vụ cho chức năng nhận dạng và phản hồi bằng giọng nói.



Hình 3.2.3: Khối cảm biến

Hình 3.2.3 mô tả khối cảm biến của hệ thống, bao gồm cảm biến LiDAR dùng để thu thập dữ liệu khoảng cách và phát hiện vật cản phục vụ cho các tác vụ định vị, lập bản đồ và điều hướng.

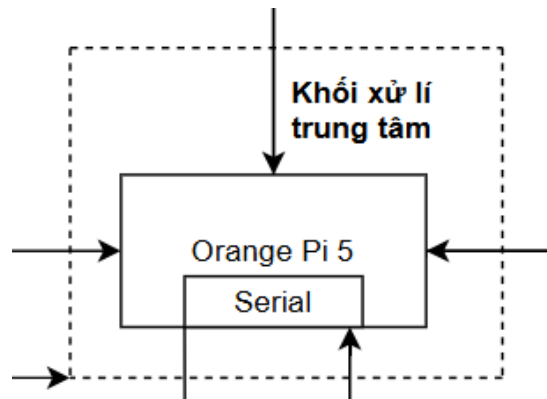


Hình 3.2.4: Khối nhập xuất dữ liệu

Hình 3.2.4 mô tả khối nhập xuất dữ liệu. Microphone được sử dụng để thu nhận tín hiệu âm thanh từ người dùng, trong khi loa đảm nhiệm việc phát âm thanh phản hồi từ hệ thống.

3.2.4 Khối xử lý trung tâm

Khối xử lý trung tâm là thành phần chịu trách nhiệm thực hiện các thuật toán xử lý dữ liệu, điều hướng và giao tiếp với các khối chức năng khác trong hệ thống. Orange Pi 5 được lựa chọn nhờ khả năng xử lý mạnh, đáp ứng tốt các tác vụ của ROS 2 và các mô hình trí tuệ nhân tạo.

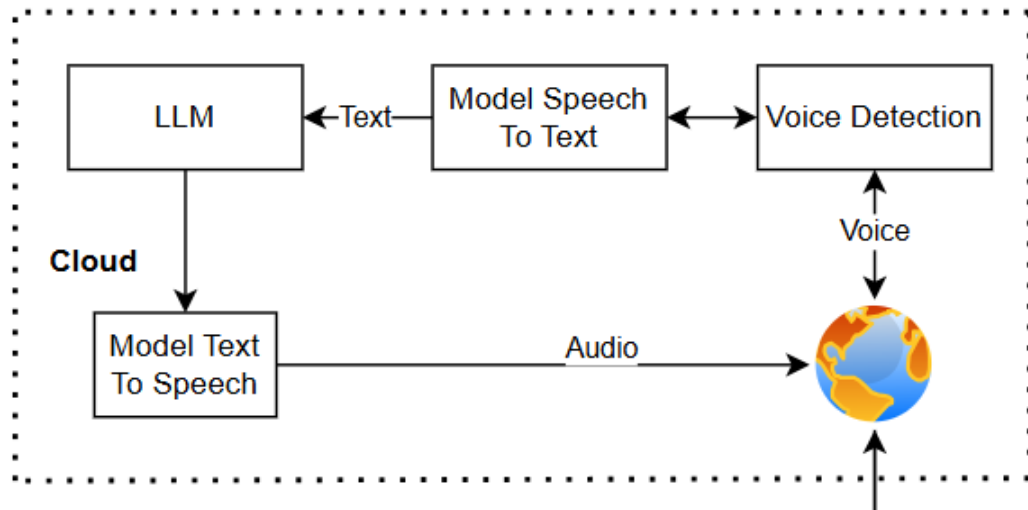


Hình 3.2.5: Khối xử lý trung tâm

Hình 3.2.5 mô tả khối xử lý trung tâm sử dụng Orange Pi 5. Thiết bị này giao tiếp với STM32F103 thông qua kết nối Serial để gửi lệnh điều khiển và nhận dữ liệu trạng thái từ robot.

3.2.5 Khối Cloud

Khối Cloud cung cấp các dịch vụ xử lý giọng nói và mô hình ngôn ngữ lớn (LLM), giúp robot có khả năng tương tác bằng ngôn ngữ tự nhiên với người dùng. Dữ liệu âm thanh được gửi lên Cloud để nhận dạng giọng nói, xử lý ngôn ngữ và tổng hợp phản hồi dưới dạng âm thanh.

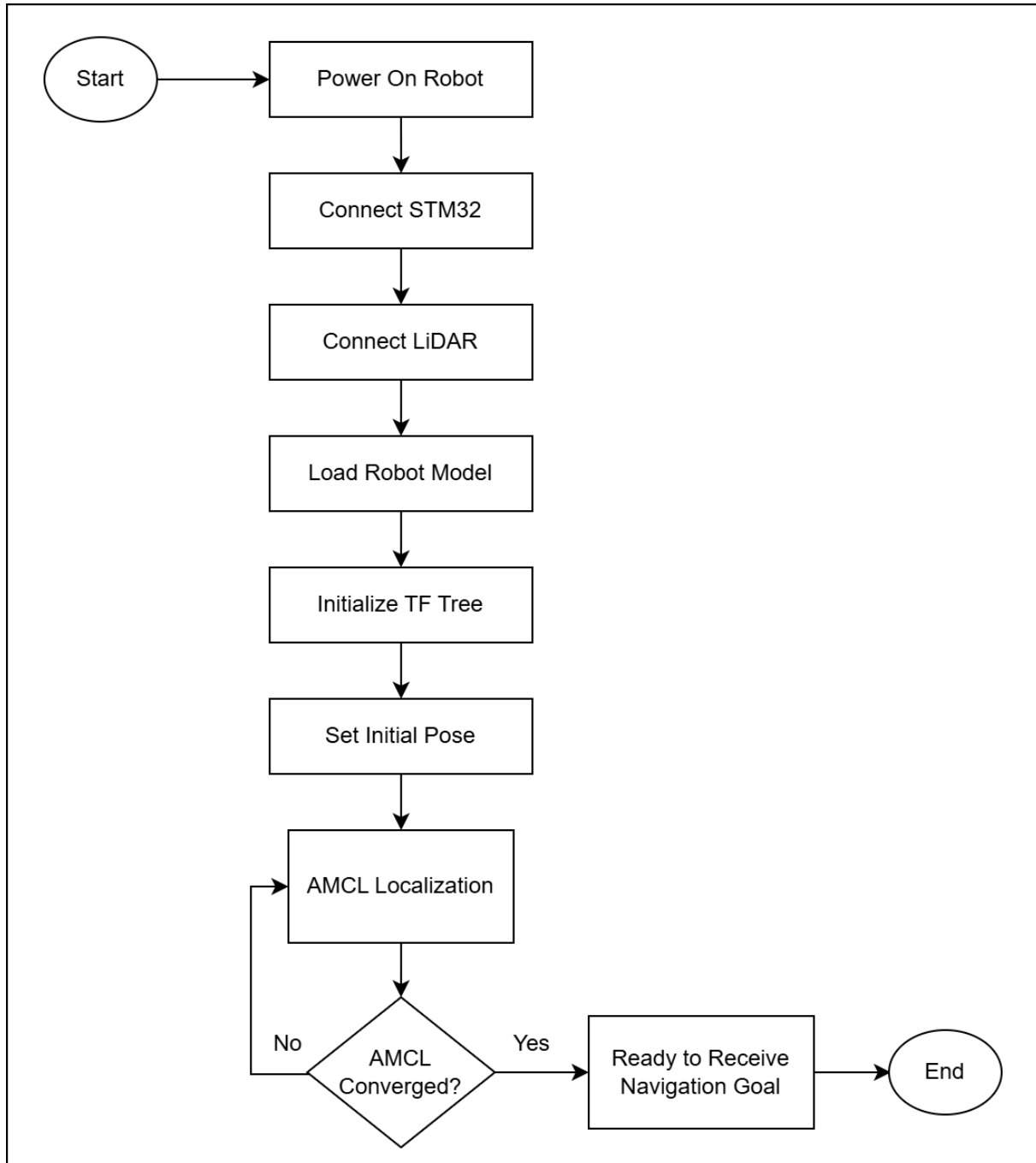


Hình 3.2.6: Khối Cloud

Hình 3.2.6 mô tả luồng xử lý của hệ thống Cloud. Tín hiệu giọng nói từ người dùng được nhận dạng thông qua mô hình Speech-to-Text để chuyển thành văn bản. Văn bản này được gửi đến mô hình ngôn ngữ lớn (LLM) để sinh phản hồi. Kết quả sau đó được chuyển qua mô hình Text-to-Speech nhằm tạo tín hiệu âm thanh và gửi trở lại robot để phát cho người dùng.

3.3 LƯU ĐỒ GIẢI THUẬT

3.3.1 Lưu đồ khởi động hệ thống



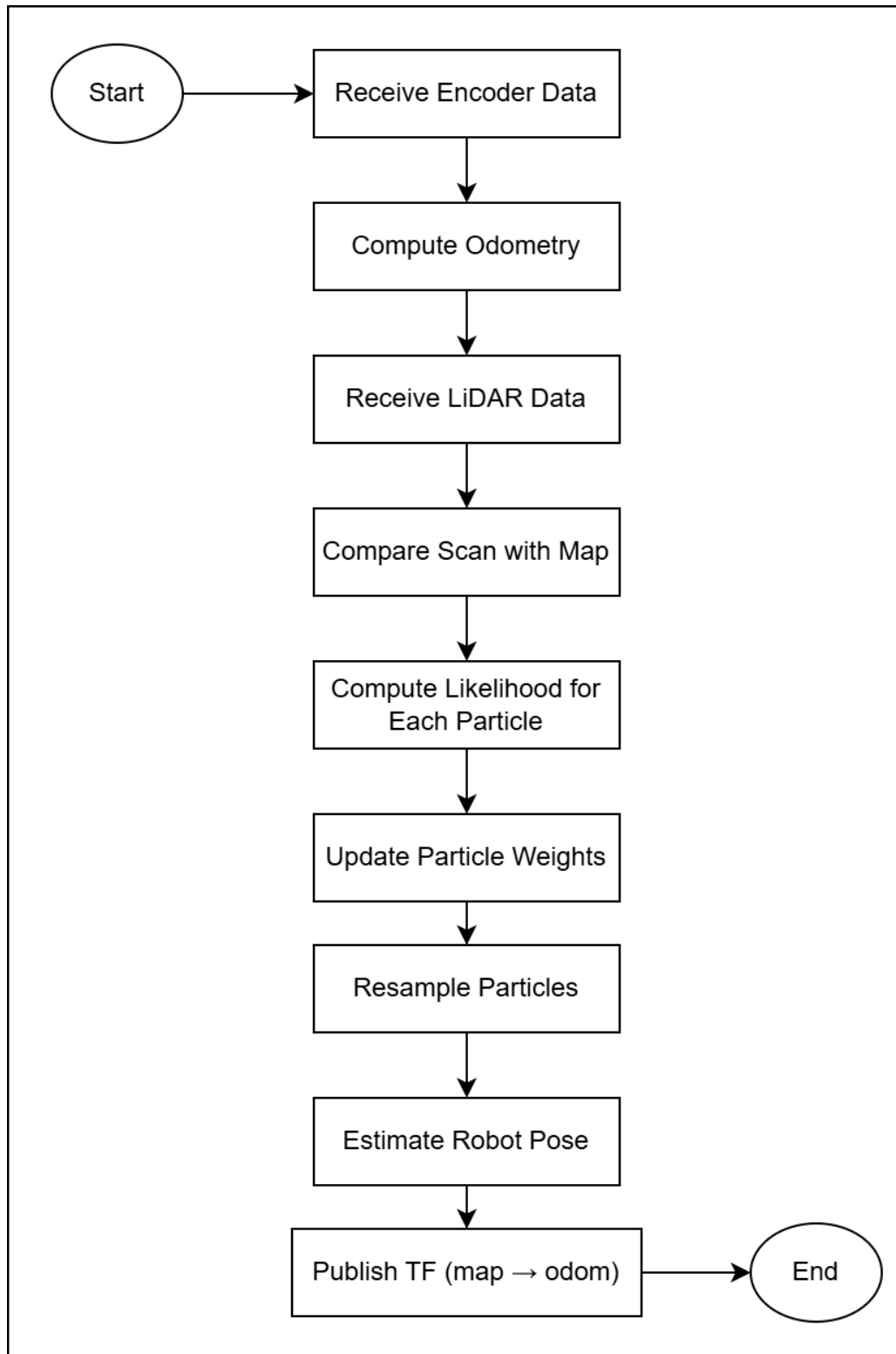
Hình 3.3.1: Lưu đồ khởi động hệ thống ROS 2 Navigation

Hình 3.3.1 mô tả quá trình khởi động hệ thống robot. Sau khi Orange Pi được cấp nguồn, hệ thống tiến hành khởi tạo các thành phần phần cứng bao gồm vi điều khiển STM32, encoder, cảm biến LiDAR và mô hình robot.

Tiếp theo, cây biến đổi tọa độ TF được thiết lập để xác định mối quan hệ giữa các khung tọa độ của robot. Hệ thống sau đó khởi động các thành phần của Navigation2, bao gồm Map Server để cung cấp bản đồ môi trường, AMCL để định vị robot trên bản đồ, Planner Server để lập kế hoạch đường đi toàn cục, Controller Server để điều khiển robot bám theo quỹ đạo đã lập.

Sau khi các nút ROS 2 được khởi tạo thành công, vị trí ban đầu của robot được thiết lập trên bản đồ. Thuật toán AMCL sử dụng dữ liệu từ encoder và LiDAR để xác định vị trí hiện tại của robot. Khi quá trình định vị hoàn tất, hệ thống sẵn sàng tiếp nhận các mục tiêu điều hướng và thực hiện di chuyển tự động trong môi trường.

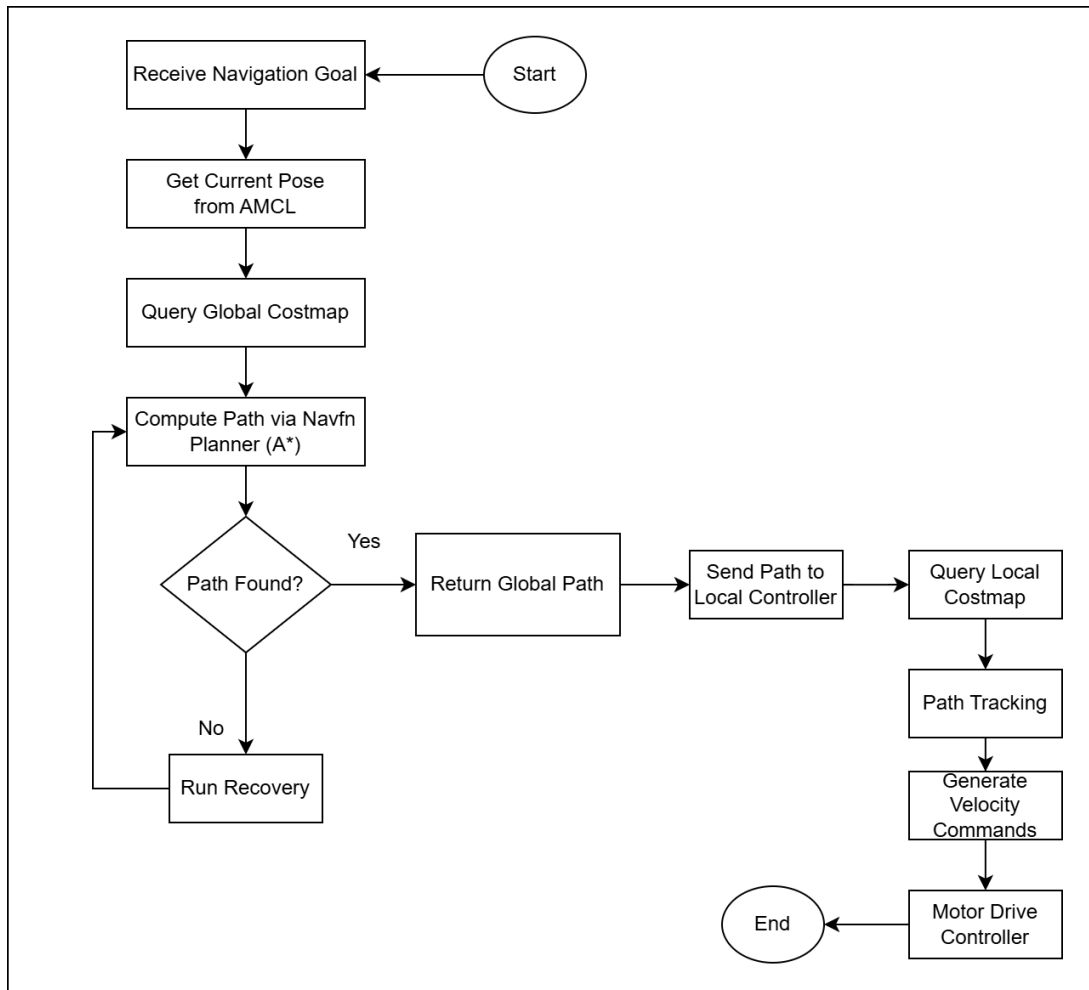
3.3.2 Lưu đồ định vị AMCL



Hình 3.3.2: Lưu đồ thuật toán định vị AMCL

Hình 3.3.2 mô tả quá trình định vị robot bằng thuật toán Adaptive Monte Carlo Localization (AMCL) trong hệ thống ROS 2 Navigation2. Mục tiêu của thuật toán là ước lượng chính xác vị trí và hướng hiện tại của robot trên bản đồ đã biết bằng cách kết hợp dữ liệu odometry từ encoder bánh xe và dữ liệu cảm biến LiDAR. Trước hết, hệ thống thu nhận tín hiệu encoder từ hai bánh xe và tính toán odometry để ước lượng chuyển động của robot, đồng thời cập nhật phép biến đổi giữa khung tọa độ odom và base_footprint. Tiếp theo, dữ liệu quét từ LiDAR được thu thập và so sánh với bản đồ môi trường đã được xây dựng trước đó nhằm đánh giá mức độ phù hợp giữa quan sát thực tế và các giả thuyết vị trí của robot. Trong AMCL, trạng thái của robot được biểu diễn bằng một tập hợp các particle, trong đó mỗi particle đại diện cho một giả thuyết về vị trí và hướng của robot trên bản đồ. Đối với từng particle, thuật toán tính toán xác suất dựa trên mức độ tương đồng giữa dữ liệu LiDAR quan sát được và dữ liệu mô phỏng từ bản đồ, từ đó cập nhật trọng số tương ứng. Sau bước này, quá trình tái lấy mẫu (resampling) được thực hiện nhằm loại bỏ các particle có trọng số thấp và tập trung các particle có trọng số cao quanh khu vực có khả năng cao nhất. Cuối cùng, từ tập particle sau tái lấy mẫu, hệ thống ước lượng trạng thái hiện tại của robot và công bố phép biến đổi tọa độ từ khung map sang odom, làm cơ sở cho các thành phần khác trong Navigation2 như lập kế hoạch đường đi và điều khiển chuyển động của robot.

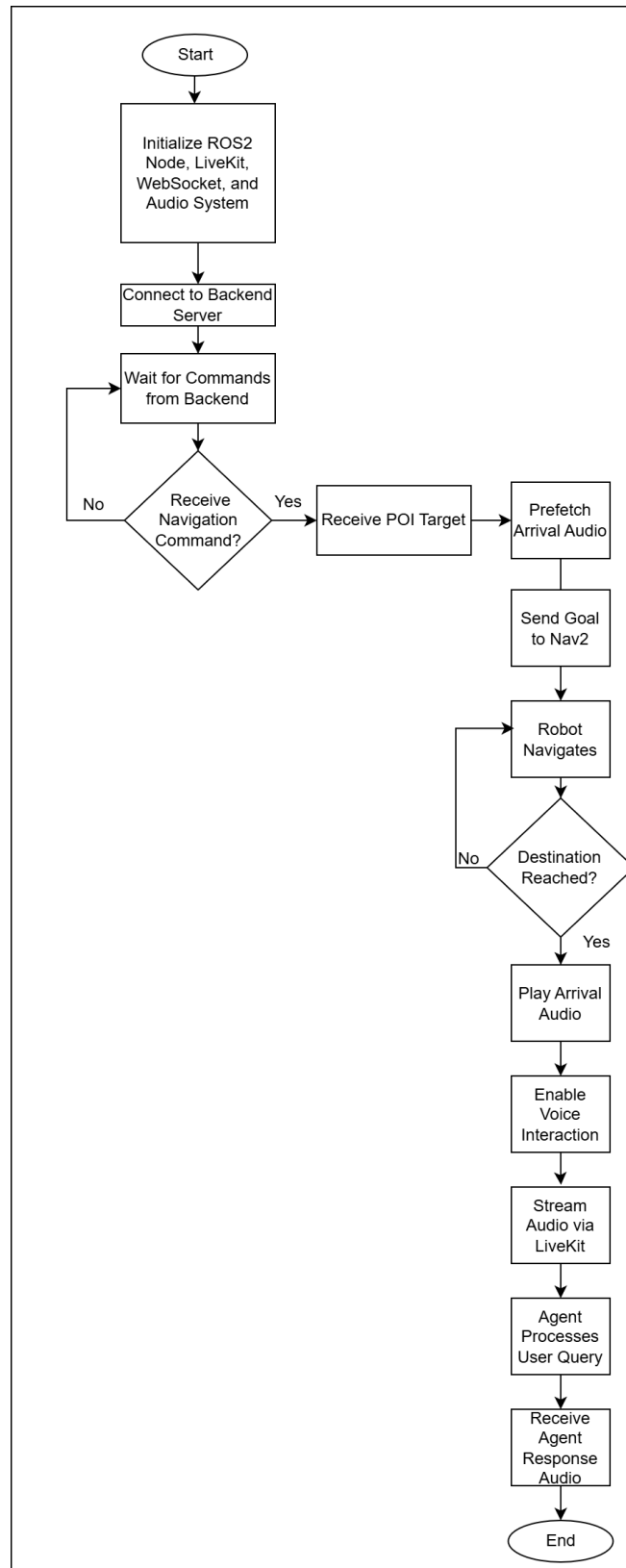
3.3.3 Lưu đồ lập kế hoạch và điều hướng Nav2



Hình 3.3.3: Lưu đồ quá trình lập kế hoạch và điều hướng của hệ thống Navigation2

Hình 3.3.3 mô tả quá trình từ lúc nhận mục tiêu di chuyển đến khi điều khiển động cơ, được quản lý toàn diện bởi bộ điều phối trung tâm (Behavior Tree). Quá trình bắt đầu khi hệ thống nhận được một mục tiêu điều hướng. Đầu tiên, bộ lập kế hoạch toàn cục xác định vị trí hiện tại của robot, đồng thời truy xuất dữ liệu từ bản đồ chi phí toàn cục. Dựa trên không gian này, thuật toán A* được áp dụng để tìm kiếm đường đi tối ưu. Nếu điểm đích bị chặn hoặc không khả thi, hệ thống sẽ kích hoạt cơ chế phục hồi. Nếu thành công, quỹ đạo toàn cục được gửi tiếp đến bộ điều khiển cục bộ. Trong quá trình di chuyển, bộ điều khiển cục bộ liên tục cập nhật bản đồ chi phí cục bộ để nhận diện các vật cản động phát sinh. Kết hợp với quỹ đạo toàn cục, thuật toán bám đường sẽ tính toán ra các lệnh vận tốc tương ứng. Cuối cùng, tín hiệu được quy đổi thành vận tốc góc cho từng bánh xe và gửi lệnh nối tiếp xuống vi điều khiển thực thi.

3.3.4 Lưu đồ giao tiếp âm thanh và hướng dẫn người dùng



Hình 3.3.4: Lưu đồ giao tiếp âm thanh và tương tác giọng nói của robot

Hình 3.3.4 mô tả quy trình giao tiếp âm thanh và tương tác giọng nói của robot trong quá trình hướng dẫn người dùng. Sau khi hệ thống nhận được lệnh điều hướng đến một điểm tham quan hoặc khu vực đích (Point of Interest - POI), robot tiến hành tải trước nội dung âm thanh giới thiệu tương ứng từ máy chủ hoặc bộ nhớ đệm cục bộ nhằm giảm độ trễ và đảm bảo nội dung có thể được phát ngay khi đến vị trí mong muốn. Tiếp theo, mục tiêu điều hướng được gửi đến hệ thống Navigation2 để robot thực hiện di chuyển tự động, trong khi hệ thống liên tục cập nhật vị trí hiện tại và trạng thái hoàn thành nhiệm vụ điều hướng. Khi robot đến đích, nội dung âm thanh giới thiệu được phát qua loa để cung cấp thông tin cho người dùng về khu vực hoặc đầu sách liên quan. Sau khi hoàn thành phần giới thiệu, robot kích hoạt chế độ hội thoại bằng giọng nói, trong đó âm thanh từ microphone được truyền theo thời gian thực qua nền tảng LiveKit đến hệ thống xử lý hội thoại. Tại đây, dữ liệu giọng nói được chuyển đổi thành văn bản, xử lý bởi mô hình ngôn ngữ lớn và chuyển ngược lại thành âm thanh phản hồi. Nội dung trả lời sau đó được gửi về robot để phát qua loa, cho phép người dùng đặt câu hỏi và nhận phản hồi bằng giọng nói theo thời gian thực, từ đó nâng cao khả năng tương tác giữa robot và người dùng trong môi trường thư viện.

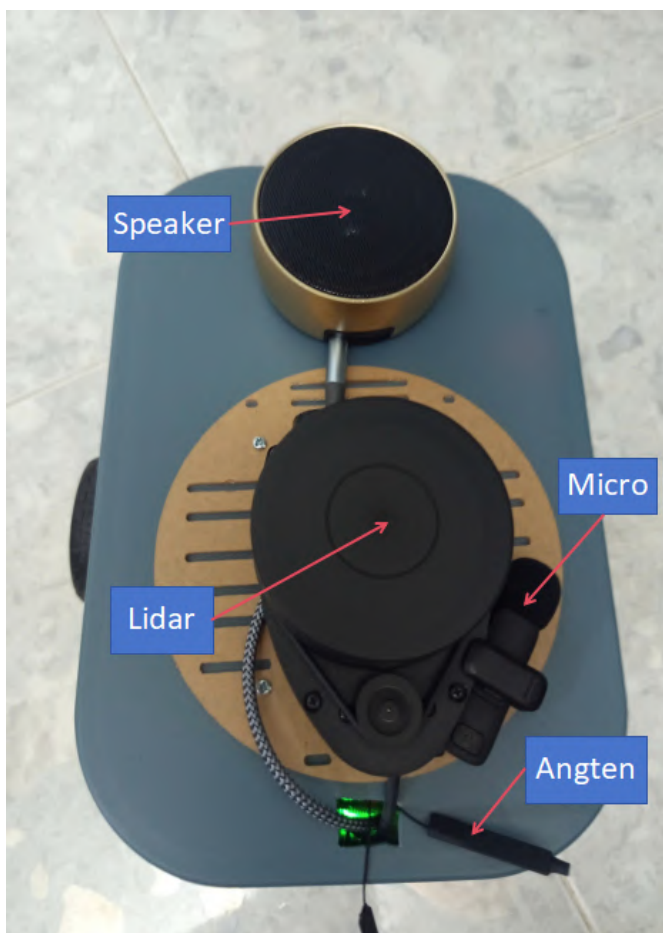
Chương 4

THI CÔNG VÀ ĐÁNH GIÁ HỆ THỐNG

4.1 THI CÔNG PHẦN CỨNG

4.1.1 LẮP RÁP ROBOT

Sau khi hoàn thiện phần khung cơ khí và hệ thống truyền động, các cảm biến và thiết bị tương tác được lắp đặt trên mặt trên của robot như Hình 4.1.1. Việc bố trí các thành phần được thực hiện nhằm đảm bảo khả năng thu nhận dữ liệu môi trường, tương tác với người dùng và duy trì tính ổn định trong quá trình vận hành.



Hình 4.1.1: Bố trí các thiết bị trên mặt trên của robot

Ở vị trí trung tâm của robot là cảm biến LiDAR 2D. Cảm biến được đặt tại khu vực có tầm nhìn bao quát nhất nhằm đảm bảo vùng quét không bị che khuất bởi các thành

phần cơ khí khác. Dữ liệu LiDAR được sử dụng cho các nhiệm vụ xây dựng bản đồ, định vị và phát hiện vật cản trong quá trình điều hướng.

Bên cạnh LiDAR là microphone thu âm. Thiết bị này có nhiệm vụ tiếp nhận các câu lệnh và câu hỏi bằng giọng nói từ người dùng. Tín hiệu âm thanh sau khi thu nhận được gửi đến server thông qua nền tảng LiveKit để thực hiện các tác vụ nhận dạng giọng nói và xử lý ngôn ngữ tự nhiên.

Phía trước robot được lắp đặt anten Wi-Fi của máy tính nhúng. Anten đảm nhiệm việc duy trì kết nối mạng không dây giữa robot và hệ thống server. Kết nối này được sử dụng để truyền dữ liệu điều hướng, trạng thái robot và luồng âm thanh thời gian thực.

Loa phát âm thanh được bố trí ở phần phía trước của mặt trên robot. Loa có chức năng phát các nội dung giới thiệu địa điểm, thông báo trạng thái hoạt động cũng như phản hồi bằng giọng nói được sinh ra từ hệ thống AI trên server.

Sau quá trình thiết kế cơ khí, tích hợp phần cứng và triển khai phần mềm, robot hoàn thiện được thể hiện trong Hình 4.1.2. Hệ thống được xây dựng trên nền tảng robot di động truyền động vi sai với hai bánh chủ động, cho phép thực hiện các chức năng di chuyển tự hành trong môi trường thư viện.



Hình 4.1.2: Robot hoàn thiện sau khi lắp ráp

4.2 KẾT QUẢ THỰC NGHIỆM

4.2.1 Thí nghiệm ở các tình huống

Sau khi hoàn thiện quá trình thiết kế, chế tạo và tích hợp phần cứng, phần mềm, hệ thống robot hướng dẫn thư viện được tiến hành thực nghiệm nhằm đánh giá khả năng đáp ứng các yêu cầu đã đề ra. Các thí nghiệm được thực hiện trong môi trường thư viện mô phỏng với các kệ sách, lối đi và các vị trí tham quan được thiết lập trước trên bản đồ.

Quá trình đánh giá tập trung vào bốn chức năng chính của hệ thống, bao gồm khả năng xây dựng bản đồ môi trường, khả năng điều hướng tự động đến vị trí được chỉ định, khả năng phát âm thanh giới thiệu tại điểm đến và khả năng tương tác hỏi đáp với người dùng thông qua giọng nói. Kết quả đánh giá được tổng hợp trong Bảng 4.2.1.

Bảng 4.2.1: Kết quả thực nghiệm các chức năng của hệ thống

STT	Test case	Điều kiện đạt	Kết quả
1	Robot xây dựng bản đồ môi trường bằng SLAM	Robot tạo được bản đồ đầy đủ của khu vực phòng Lab, các biên tường và vật cản được thể hiện chính xác, dữ liệu bản đồ được lưu thành công.	Đạt
2	Robot di chuyển đến vị trí được chỉ định trên bản đồ	Robot nhận được mục tiêu điều hướng, lập kế hoạch đường đi và đến được vị trí đích với sai số nhỏ hơn 0.25m.	Đạt
3	Robot phát âm thanh giới thiệu và trả lời câu hỏi tại vị trí đích	Sau khi đến đích, robot tự động phát nội dung giới thiệu tương ứng thông qua loa và người dùng có thể nghe rõ nội dung phát ra, đồng thời có thể đặt các câu hỏi thắc mắc về địa điểm và nhận được câu trả lời có độ trễ dưới 10 giây	Chưa đạt (độ trễ phản hồi lớn hơn 10 giây)

Test case 1: Xây dựng bản đồ môi trường bằng SLAM

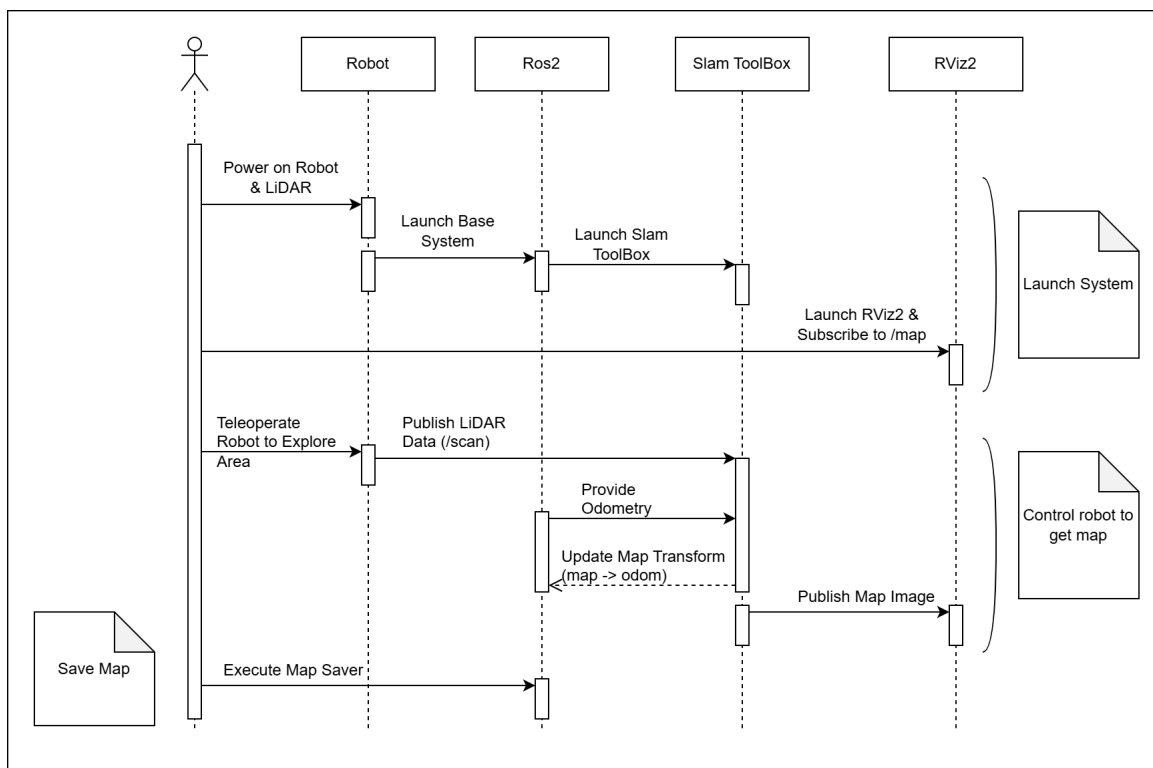
Mục tiêu kiểm thử

Đánh giá khả năng xây dựng bản đồ môi trường của hệ thống bằng thuật toán SLAM Toolbox khi robot di chuyển trong khu vực phòng Lab C406B.

Các bước kiểm thử

Hình 4.2.1 mô tả trình tự xây dựng bản đồ bằng SLAM và Bảng 4.2.2 mô tả các cấu hình khi xây dựng bản đồ. Các bước mô tả như sau:

- Khởi động robot, ROS2, SLAM Toolbox và RViz2.
- Điều khiển robot di chuyển trong khu vực khảo sát.
- Thu thập dữ liệu LiDAR và odometry trong quá trình di chuyển.
- SLAM Toolbox thực hiện đồng thời định vị và xây dựng bản đồ môi trường.
- Hiển thị bản đồ thời gian thực trên RViz2 thông qua topic */map*.
- Lưu bản đồ sau khi hoàn thành quá trình khảo sát bằng công cụ Map Saver.



Hình 4.2.1: Luồng kiểm thử quá trình xây dựng bản đồ bằng SLAM

Điều kiện Testcase đạt/trượt

Đạt:

- Robot thu thập được dữ liệu LiDAR liên tục trong suốt quá trình di chuyển.
- Hệ thống xây dựng thành công bản đồ 2D của môi trường.
- Các biên tường, lối đi và vật cản cố định được thể hiện chính xác trên bản đồ.
- Không xuất hiện hiện tượng mất bản đồ, sai lệch lớn hoặc lỗi định vị trong quá trình xây dựng bản đồ.

Trượt:

- Robot không thu thập được dữ liệu LiDAR hoặc dữ liệu bị gián đoạn trong quá trình di chuyển.
- Hệ thống không xây dựng được bản đồ hoặc quá trình xây dựng bản đồ bị dừng giữa chừng.
- Bản đồ thu được không đầy đủ, xuất hiện vùng bị thiếu, mất góc hoặc mất một phần môi trường đã khảo sát.
- Các vật cản cố định, tường hoặc lối đi không được nhận dạng chính xác hoặc không xuất hiện trên bản đồ.
- Xuất hiện hiện tượng méo bản đồ, chồng lấn bản đồ, sai lệch vị trí lớn hoặc mất khả năng định vị trong quá trình xây dựng bản đồ.
- Bản đồ cuối cùng không phản ánh đúng cấu trúc thực tế của môi trường khảo sát.

Các cấu hình sử dụng

Bảng 4.2.2: Cấu hình sử dụng trong quá trình xây dựng bản đồ

Cấu hình	Giá trị	Mô tả
Serial baudrate	115200	Tốc độ truyền dữ liệu của cảm biến RPLidar A1.
Tần số lấy mẫu	8 KHz	Số điểm dữ liệu thu thập được trong một giây (khoảng 8000 điểm/s).
Tốc độ vòng quay	10 Hz	Tốc độ quay của động cơ LiDAR, tương đương 10 vòng/giây.
Topic dữ liệu LaserScan	/scan	Topic ROS2 dùng để truyền dữ liệu tia laser tới SLAM Toolbox.

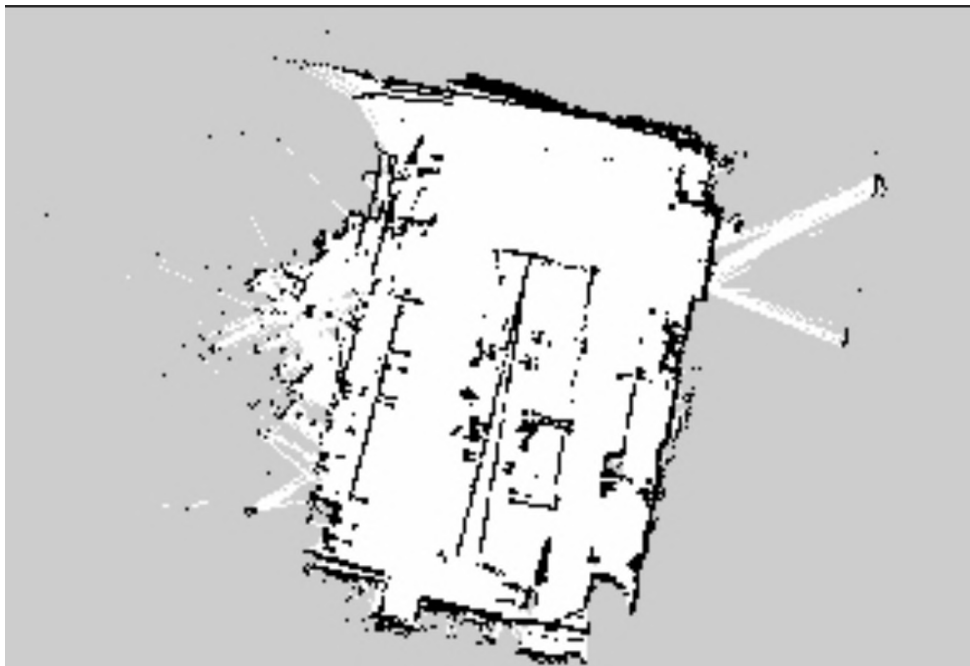
Kết quả thực tế

Hình 4.2.2 thể hiện sơ đồ bố trí thực tế của phòng Lab C406B được sử dụng trong quá trình khảo sát. Khu vực thử nghiệm bao gồm các biên tường, bàn làm việc, ghế ngồi và hai lối đi chính phục vụ cho quá trình di chuyển của robot.



Hình 4.2.2: Sơ đồ bố trí thực tế của phòng Lab C406B

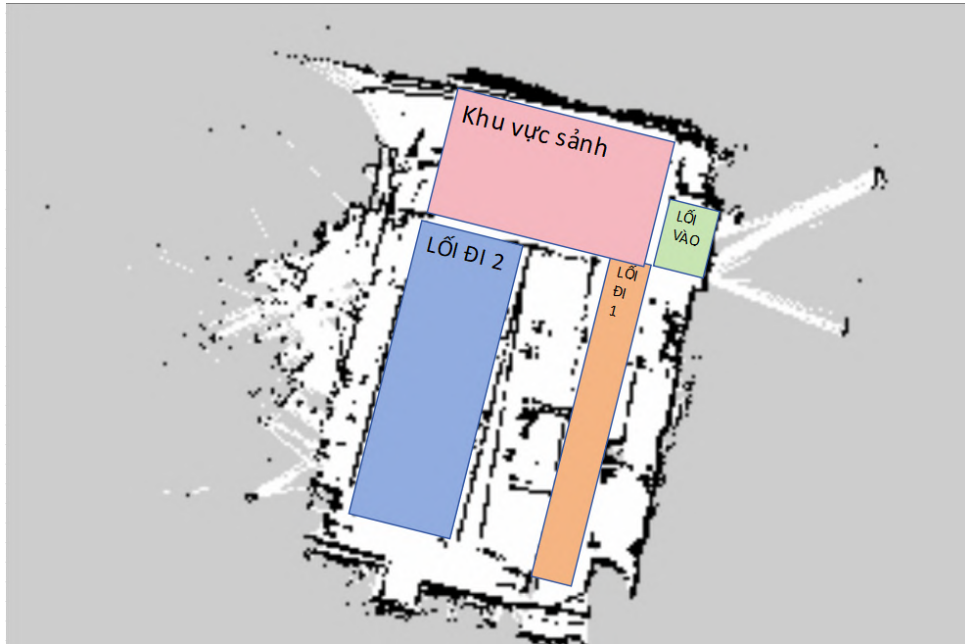
Hình 4.2.3 là bản đồ thu được sau khi thực hiện quá trình xây dựng bản đồ bằng SLAM Toolbox. Các vùng màu đen thể hiện vật cản và biên môi trường, vùng màu trắng thể hiện khu vực có thể di chuyển, trong khi các vùng màu xám là khu vực chưa được khảo sát.



Hình 4.2.3: Bản đồ môi trường được xây dựng bởi SLAM Toolbox

Trong quá trình thử nghiệm, robot di chuyển ổn định qua hai lối đi chính trong phòng và liên tục thu thập dữ liệu từ cảm biến LiDAR. Hệ thống SLAM Toolbox cập nhật bản đồ theo thời gian thực trên RViz2 mà không xuất hiện lỗi mất dữ liệu hoặc gián đoạn.

Hình 4.2.4 trình bày bản đồ môi trường sau khi được xây dựng bởi SLAM Toolbox cùng với các chú thích về những thành phần chính trong phòng Lab C406B như lối vào, sảnh, hai lối đi chính.



Hình 4.2.4: Chú thích các thành phần trên bản đồ SLAM

Kết quả cuối cùng cho thấy bản đồ thu được mô tả đầy đủ cấu trúc của phòng Lab C406B, bao gồm các biên tường, bàn ghế và lối đi chính. Các đặc trưng môi trường được tái hiện rõ ràng và nhất quán với thực tế. Hệ thống đáp ứng đầy đủ các tiêu chí Pass đã đề ra, chứng minh khả năng xây dựng bản đồ ổn định và chính xác phục vụ cho các chức năng định vị và điều hướng tự động của robot.

Test case 2: Điều hướng đến vị trí được chỉ định

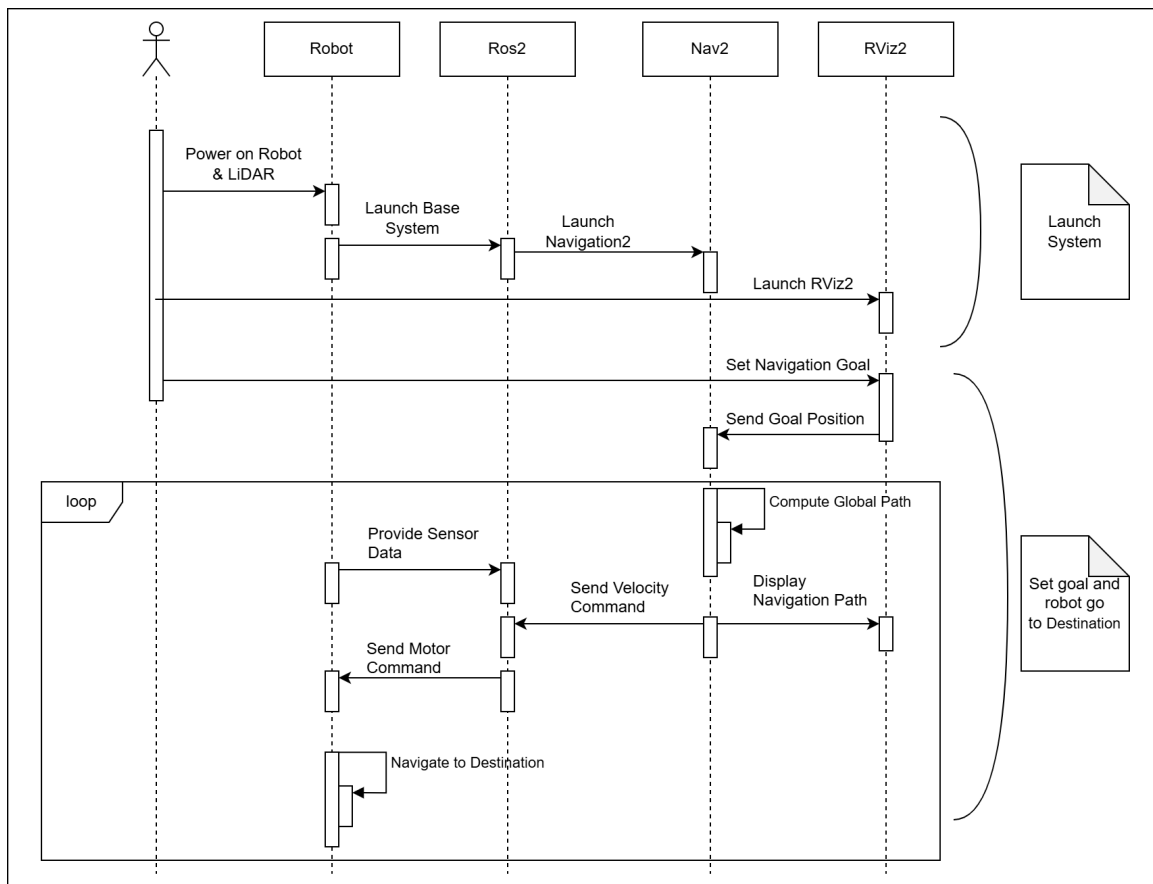
Mục tiêu kiểm thử

Đánh giá khả năng định vị và điều hướng tự động của robot trong môi trường đã được xây dựng bản đồ trước đó. Quá trình kiểm thử nhằm xác định mức độ chính xác của hệ thống Navigation2 khi dẫn robot đến các vị trí mục tiêu ở các khoảng cách khác nhau.

Các bước kiểm thử

Hình 4.2.5 mô tả trình tự điều hướng tự động của robot và Bảng 4.2.3 mô tả cấu hình khi kiểm thử. Các bước thể hiện cụ thể như sau:

- Khởi động robot và RViz2.
- Hiển thị bản đồ môi trường đã xây dựng.
- Chọn vị trí mục tiêu trên RViz2 bằng công cụ *Nav2 Goal*.
- Navigation2 lập kế hoạch đường đi và điều khiển robot di chuyển đến đích.
- Khi robot dừng tại vị trí mục tiêu, đo khoảng cách từ tâm robot đến điểm đích.
- Ghi nhận khoảng cách và lặp lại phép đo tại các khoảng cách thử nghiệm khác nhau.



Hình 4.2.5: luồng kiểm thử quá trình điều hướng tự động

Điều kiện Testcase đạt/trượt

Đạt:

- Robot nhận được mục tiêu điều hướng từ RViz2.
- Navigation2 lập kế hoạch đường đi thành công.
- Robot di chuyển liên tục và ổn định đến vị trí đích.
- Không xảy ra va chạm với các vật cản trong môi trường.

- Sai số vị trí cuối cùng nằm trong ngưỡng cho phép.

Trượt:

- Navigation2 không thể lập kế hoạch đường đi đến mục tiêu.
- Robot không thể đến được vị trí đích hoặc dừng giữa chừng.
- Robot va chạm với vật cản trong quá trình di chuyển.
- Robot mất định vị hoặc thực hiện điều hướng sai hướng.
- Trung bình khoảng cách đến đích vượt quá ngưỡng 25cm.

Các cấu hình sử dụng

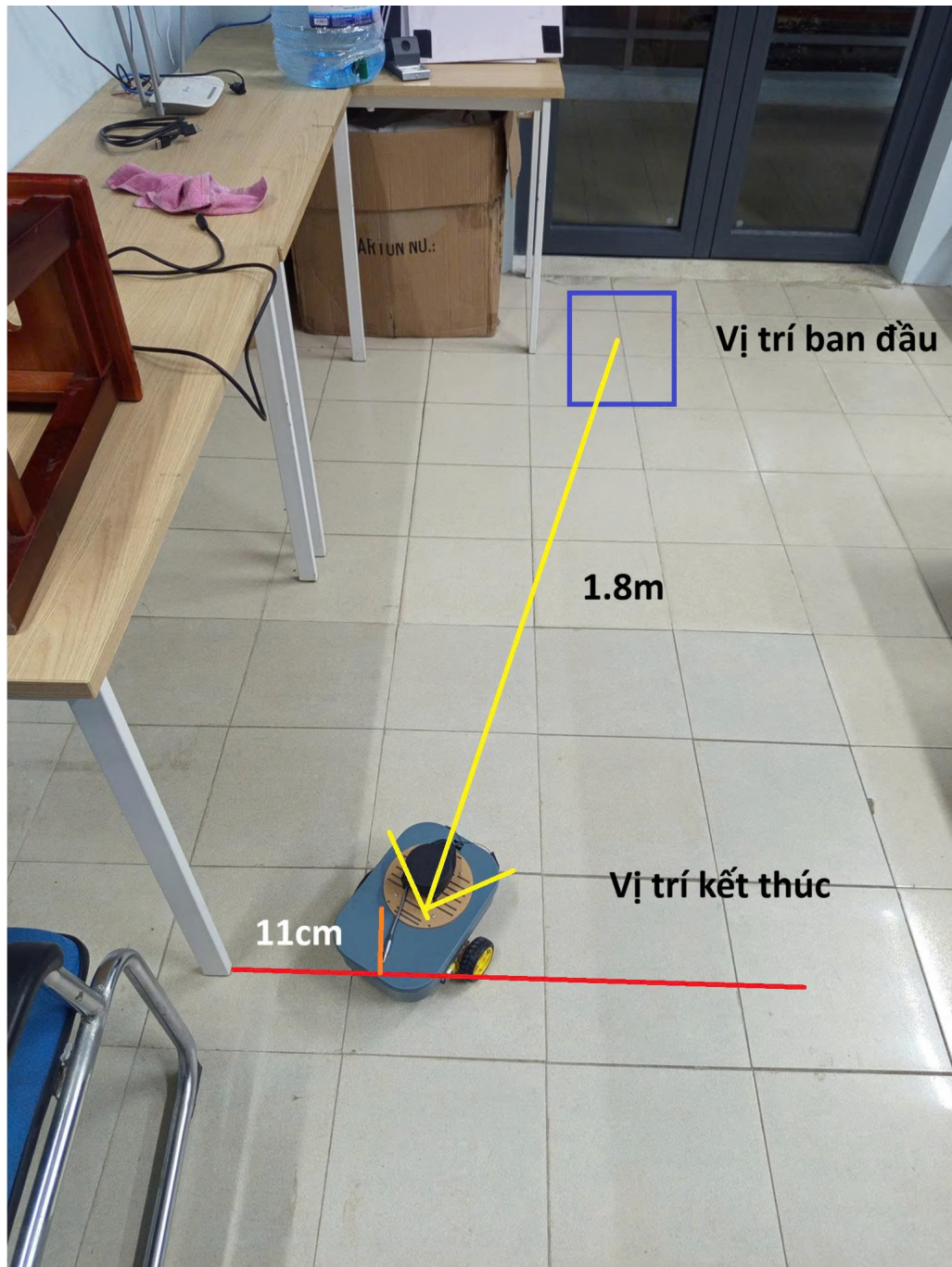
Bảng 4.2.3: Các tham số ảnh hưởng đến độ chính xác điều hướng

Cấu hình	Giá trị	Ảnh hưởng
Goal Position Tolerance	0.25 m	Ngưỡng sai số vị trí cho phép khi robot được xem là đã đến đích.
Goal Orientation Tolerance	0.25 rad	Ngưỡng sai số góc định hướng cho phép tại vị trí đích.
Lookahead Distance	0.6 m	Khoảng cách nhìn trước của bộ điều khiển Regulated Pure Pursuit.
Desired Linear Velocity	0.2 m/s	Tốc độ di chuyển của robot;
Obstacle Inflation Radius	0.40 m	Khoảng cách an toàn được thiết lập xung quanh vật cản trong quá trình lập kế hoạch đường đi.

Trong các tham số được khảo sát, *Goal Position Tolerance* là tham số ảnh hưởng trực tiếp nhất đến kết quả kiểm thử vì quy định ngưỡng sai số vị trí mà hệ thống Navigation2 chấp nhận khi xác định robot đã đến đích. Do đó, sai số vị trí trung bình thu được từ các thử nghiệm cần nhỏ hơn giá trị 0.25 m để chứng minh hệ thống điều hướng đạt độ chính xác mong muốn.

Tiến hành thực nghiệm

Hình 4.2.6 minh họa một lần thử nghiệm điều hướng ở khoảng cách 1.8 m. Trên hình thể hiện vị trí xuất phát, vị trí mục tiêu, các vật cản trong môi trường và vị trí thực tế của robot sau khi hoàn thành điều hướng. Sai số được xác định bằng khoảng cách từ tâm robot đến điểm mục tiêu.



Hình 4.2.6: Ví dụ một lần thử nghiệm điều hướng khoảng cách 1.8m

Ba vị trí mục tiêu được lựa chọn tương ứng với khoảng cách điều hướng xấp xỉ 1.8 m, 3 m và 5 m tính từ vị trí xuất phát của robot. Đối với mỗi khoảng cách, quá trình điều hướng được thực hiện 10 lần độc lập.

Sau khi robot hoàn thành điều hướng, giá trị khoảng cách đo được bằng cách xác định giữa tâm robot và điểm mục tiêu trên mặt phẳng bản đồ:

Giá trị đo được thể hiện ở Hình 4.2.7

Khoảng cách	1.8m	3m	5m
#1	11.2	12.8	13.6
#2	11.9	14.1	14.3
#3	12.5	13.7	15.2
#4	10.5	12.4	12.8
#5	9	10.8	11.7
#6	11	12.9	13
#7	12.5	14.4	14.8
#8	10.5	11.8	12.7
#9	8	10.2	10.6
#10	12	13.8	14.1
Trung bình	10.91	12.69	13.28
Kết quả (So sánh với 25cm)	OK	OK	OK

Hình 4.2.7: Giá trị khoảng cách đo được sau các lần kiểm thử tại các vị trí khác nhau

Nhận xét: Kết quả thử nghiệm cho thấy hệ thống điều hướng hoạt động ổn định tại cả ba khoảng cách 1.8 m, 3 m và 5 m. Sai số vị trí trung bình lần lượt là 10.91 cm, 12.69 cm và 13.28 cm, đều nhỏ hơn đáng kể so với ngưỡng chấp nhận 25 cm của Navigation2. Sai số có xu hướng tăng khi khoảng cách điều hướng lớn hơn, tuy nhiên mức tăng không đáng kể và vẫn nằm trong giới hạn cho phép.

Test case 3: Phát âm thanh giới thiệu và trả lời câu hỏi đúng ngữ cảnh tại vị trí đích

Mục tiêu kiểm thử

Đánh giá khả năng tương tác bằng giọng nói của robot trong quá trình tham quan, bao gồm khả năng trả lời các câu hỏi liên quan đến ngữ cảnh hiện tại, xử lý các yêu cầu chuyển sang điểm tham quan tiếp theo và phản hồi đối với các câu hỏi nằm ngoài phạm vi kiến thức được cung cấp. Đồng thời đánh giá độ trễ phản hồi của hệ thống từ thời điểm người dùng kết thúc câu hỏi đến khi robot bắt đầu phát câu trả lời.

Các bước thực hiện

- Sử dụng RViz2 để thiết lập vị trí ban đầu của robot trên bản đồ bằng chức năng *2D Pose Estimate*.
- Truy cập giao diện Web Server của robot và cấu hình các chức năng theo hướng dẫn tại Mục 2.7

- Điều hướng robot đến các điểm tham quan, thực hiện đặt câu hỏi bằng giọng nói và ghi nhận thời gian từ khi kết thúc câu hỏi đến khi robot bắt đầu phát câu trả lời.
- Đánh giá tính chính xác của câu trả lời dựa trên ngữ cảnh tương ứng tại từng điểm tham quan.

Điều kiện Testcase đạt/trượt

Đạt:

- Robot phát đúng nội dung giới thiệu tương ứng với điểm tham quan hiện tại.
- Hệ thống nhận dạng chính xác câu hỏi của người dùng.
- Các câu hỏi thuộc ngữ cảnh được trả lời đúng nội dung và phù hợp với thông tin đã cung cấp.
- Hệ thống nhận biết và xử lý đúng các lệnh chuyển sang điểm tham quan tiếp theo.
- Các câu hỏi ngoài ngữ cảnh được phản hồi phù hợp, không tạo ra thông tin sai lệch.
- Thời gian phản hồi trung bình nhỏ hơn 10 giây.

Trượt:

- Robot phát sai nội dung giới thiệu hoặc không phát được âm thanh.
- Hệ thống nhận dạng sai câu hỏi của người dùng.
- Câu trả lời không liên quan đến ngữ cảnh hiện tại hoặc chứa thông tin không chính xác.
- Hệ thống không nhận biết được lệnh chuyển sang điểm tham quan tiếp theo.
- Hệ thống đưa ra câu trả lời không phù hợp đối với các câu hỏi ngoài phạm vi kiến thức.
- Thời gian phản hồi trung bình vượt quá 10 giây.

Kết quả thực tế

Để đánh giá khả năng tương tác bằng giọng nói của robot, hệ thống được kiểm thử tại ba điểm tham quan khác nhau trên bản đồ đã xây dựng. Mỗi điểm tham quan được gắn với một ngữ cảnh giới thiệu riêng nhằm mô phỏng quá trình hướng dẫn thực tế trong phòng Lab C406B.

Các ngữ cảnh được sử dụng trong quá trình thử nghiệm bao gồm:

- **Khu vực seminar và trao đổi nghiên cứu:** Robot giới thiệu về hoạt động seminar định kỳ, quá trình hình thành ý tưởng nghiên cứu và các lĩnh vực nghiên cứu chính của phòng lab như thị giác máy tính, xe tự hành, robot hỗ trợ con người, ảnh y tế và AI đa phương thức.
- **Khu vực hệ thống tính toán hiệu năng cao và xe tự hành:** Robot giới thiệu vai trò của hệ thống tính toán trong quá trình huấn luyện mô hình AI, xử lý dữ liệu lớn và hỗ trợ nghiên cứu các bài toán nhận thức môi trường, định vị, lập kế hoạch đường đi và điều khiển xe tự hành.
- **Khu vực chế tạo nhanh và drone:** Nội dung giới thiệu tập trung vào vai trò của máy in 3D trong việc chế tạo nguyên mẫu nhanh, đồng thời trình bày các ứng dụng của drone trong thu thập dữ liệu, lập bản đồ và hỗ trợ các bài toán nghiên cứu từ góc nhìn trên cao.

Sau khi hoàn thành phần giới thiệu tại từng khu vực, người dùng có thể đặt câu hỏi bằng giọng nói. Hệ thống sử dụng mô hình ngôn ngữ để tạo câu trả lời dựa trên ngữ cảnh hiện tại và chuyển đổi kết quả thành giọng nói thông qua dịch vụ TTS. Hình 4.2.8 trình bày một số ví dụ câu hỏi và câu trả lời của hệ thống tại các điểm tham quan.

Câu hỏi	Trả lời tóm tắt
Khu vực này được sử dụng để làm gì?	Nơi các thành viên lab trình bày tiến độ nghiên cứu, chia sẻ bài báo và thảo luận hướng nghiên cứu mới.
Các buổi seminar của lab thường diễn ra vào ngày nào?	Thường được tổ chức vào mỗi Chủ nhật.
Trong các buổi seminar, mọi người thường thảo luận những chủ đề gì?	Thị giác máy tính, xe tự hành, AI đa phương thức, robot, ảnh y tế và dữ liệu đô thị.
Buổi seminar tiếp theo bắt đầu lúc mấy giờ và ai sẽ thuyết trình?	Chưa có dữ liệu xác thực để trả lời.
Thiết bị ở khu vực này là gì?	Máy chủ AI DGX GB10 phục vụ nghiên cứu và phát triển mô hình AI.
Máy chủ này được dùng để thực hiện những công việc nào?	Huấn luyện, tinh chỉnh và thử nghiệm mô hình AI; xử lý ảnh, video và dữ liệu cảm biến.
DGX GB10 ở đây có bao nhiêu GPU và hiện đang sử dụng bao nhiêu phần trăm bộ nhớ?	Chưa có dữ liệu xác thực để trả lời.
Kết quả (Trả lời đúng ngữ cảnh, không có thì trả lời chưa có dữ liệu)	OK

Hình 4.2.8: Câu hỏi và câu trả lời trong các ngữ cảnh tham quan

Để đánh giá hiệu năng phản hồi, hệ thống được kiểm thử với ba nhóm câu hỏi gồm: câu hỏi đúng ngữ cảnh, câu lệnh bỏ qua để di chuyển đến điểm tiếp theo và câu hỏi không thuộc ngữ cảnh hiện tại. Mỗi nhóm bao gồm 10 lần thử nghiệm độc lập. Thời gian phản hồi được tính từ thời điểm người dùng kết thúc câu hỏi đến khi hệ thống bắt đầu phát âm thanh trả lời.

Kết quả đo độ trễ đối với nhóm câu lệnh bỏ qua để tiếp tục hành trình tham quan được thể hiện trong Hình 4.2.9. Độ trễ trung bình đạt 2.71 giây, nhỏ hơn đáng kể so với

ngưỡng đánh giá 10 giây.

Loại câu hỏi : Bỏ qua đề đi tiếp	Độ trễ
Không	2.5
Tôi không còn câu hỏi	2.9
Hãy đi tiếp	2.6
Bạn hãy đi tiếp, tôi không còn câu hỏi	2.95
Tôi không còn thắc mắc nào nữa.	2.63
Chúng ta tiếp tục tham quan khu vực tiếp theo nhé.	2.72
Tôi đã hiểu rồi, bạn cứ giới thiệu tiếp đi.	2.81
Tôi muốn xem khu vực tiếp theo.	2.58
Không còn câu hỏi nào, chúng ta tiếp tục nhé.	2.74
Bạn hãy dẫn tôi đến điểm tham quan tiếp theo.	2.67
Trung bình	2.71
Kết quả (so sánh 10 giây)	OK

Hình 4.2.9: Kết quả đo độ trễ đối với nhóm câu lệnh bỏ qua để tiếp tục tham quan

Kết quả đo độ trễ đối với nhóm câu hỏi đúng ngữ cảnh được thể hiện trong Hình 4.2.10. Mặc dù độ trễ trung bình của nhóm câu hỏi đúng ngữ cảnh đạt 10.81 giây và cao hơn ngưỡng mong muốn 10 giây, kết quả này chủ yếu xuất phát từ đặc điểm xử lý ngôn ngữ tự nhiên tiếng Việt của hệ thống. Trong quá trình trả lời, dữ liệu âm thanh đầu vào cần trải qua nhiều giai đoạn xử lý liên tiếp bao gồm nhận dạng giọng nói, truy xuất ngữ cảnh hiện tại, suy luận bằng mô hình ngôn ngữ lớn và tổng hợp giọng nói. Đối với các câu hỏi đúng ngữ cảnh, mô hình phải phân tích nội dung câu hỏi, xác định thông tin liên quan trong ngữ cảnh tham quan hiện tại và tổng hợp câu trả lời phù hợp trước khi sinh phản hồi bằng tiếng Việt.

CHƯƠNG 4. THI CÔNG VÀ ĐÁNH GIÁ HỆ THỐNG

Loại câu hỏi : Có ngữ cảnh để trả lời	Độ trễ
Khu vực này được sử dụng để làm gì?	11.2
Các buổi seminar của lab thường diễn ra vào ngày nào?	8.2
Trong các buổi seminar, mọi người thường thảo luận những chủ đề gì?	9.5
Buổi seminar tiếp theo bắt đầu lúc mấy giờ và ai sẽ thuyết trình?	9.57
Chiếc xe có chức năng gì?	10.1
Điểm nổi bật nào giúp hệ thống phù hợp với môi trường nghiên cứu AI?	12.2
Làm thế nào hệ thống tính toán này hỗ trợ việc xử lý dữ liệu lớn và thử nghiệm mô hình AI?	13
Lab hiện đang tập trung nghiên cứu những lĩnh vực nào của AI?	11.4
Xe tự hành trong phòng lab được sử dụng cho mục đích gì?	10.8
Drone hỗ trợ nghiên cứu và thu thập dữ liệu như thế nào?	12.1
Trung bình	10.807
Kết quả (so sánh 10 giây)	NOK

Hình 4.2.10: Kết quả đo độ trễ đối với nhóm câu hỏi đúng ngữ cảnh

Kết quả đo độ trễ đối với nhóm câu hỏi không thuộc ngữ cảnh được thể hiện trong Hình 4.2.11. Độ trễ trung bình đạt 8.69 giây. Hệ thống có khả năng nhận biết các câu hỏi nằm ngoài phạm vi kiến thức được cung cấp và phản hồi phù hợp thay vì tạo ra thông tin không chính xác.

Loại câu hỏi : Không có ngữ cảnh để trả lời	Độ trễ
Chiếc xe này giá bao nhiêu?	6.4
Chiếc máy in này màu gì?	7.4
Drone nặng bao nhiêu	7.8
Chiếc drone này có giá bao nhiêu?	8.6
Máy in 3D này thuộc hãng nào?	9.4
Xe tự hành này đạt tốc độ tối đa bao nhiêu?	10
DGX này được mua vào năm nào?	9.7
Drone có thể bay liên tục trong bao lâu?	8.3
Chi phí xây dựng phòng lab này là bao nhiêu?	10.1
Ai là người thiết kế chiếc xe tự hành này?	9.2
Trung bình	8.69
Kết quả (so sánh 10 giây)	OK

Hình 4.2.11: Kết quả đo độ trễ đối với nhóm câu hỏi không thuộc ngữ cảnh

Nhìn chung, hệ thống trả lời chính xác các câu hỏi thuộc ngữ cảnh được cung cấp, hỗ trợ người dùng tiếp tục hành trình tham quan khi nhận được lệnh bỏ qua và xử lý phù hợp các câu hỏi ngoài ngữ cảnh. Đối với tiêu chí đánh giá độ trễ nhỏ hơn 10 giây, nhóm câu lệnh bỏ qua và nhóm câu hỏi ngoài ngữ cảnh đều đạt yêu cầu. Nhóm câu hỏi đúng ngữ cảnh có độ trễ trung bình 10.81 giây, cao hơn ngưỡng đặt ra do yêu cầu truy xuất và

tổng hợp lượng thông tin lớn hơn trước khi sinh câu trả lời.

4.2.2 Đánh giá tổng quan

Kết quả thực nghiệm cho thấy hệ thống robot hướng dẫn viên đã hoàn thành đầy đủ các chức năng cốt lõi được đề ra trong quá trình thiết kế. Robot có khả năng xây dựng bản đồ môi trường bằng SLAM, định vị và điều hướng tự động đến các vị trí mục tiêu với sai số trung bình nhỏ hơn đáng kể so với ngưỡng cho phép của hệ thống Navigation2.

Đối với chức năng tương tác bằng giọng nói, robot có thể tự động phát nội dung giới thiệu tương ứng với từng điểm tham quan, đồng thời trả lời các câu hỏi liên quan đến ngữ cảnh hiện tại và xử lý các yêu cầu chuyển sang điểm tham quan tiếp theo. Hệ thống cũng thể hiện khả năng nhận biết các câu hỏi nằm ngoài phạm vi kiến thức được cung cấp và đưa ra phản hồi phù hợp thay vì tạo ra thông tin không chính xác.

Mặc dù nhóm câu hỏi đúng ngữ cảnh có độ trễ phản hồi trung bình đạt 10.81 giây, cao hơn ngưỡng đánh giá đề ra, kết quả này chủ yếu xuất phát từ thời gian xử lý của chuỗi nhận dạng tiếng nói, suy luận ngôn ngữ và tổng hợp giọng nói tiếng Việt. Tuy nhiên, các câu trả lời vẫn đảm bảo tính chính xác, phù hợp với ngữ cảnh tham quan và không gây ảnh hưởng đáng kể đến trải nghiệm của người sử dụng.

Từ các kết quả thu được có thể kết luận rằng hệ thống đáp ứng tốt các yêu cầu về xây dựng bản đồ, điều hướng tự động và tương tác người–robot trong môi trường trong nhà. Đây là cơ sở để tiếp tục phát triển các chức năng nâng cao như mở rộng phạm vi tri thức, tối ưu độ trễ phản hồi và triển khai trên các môi trường thực tế có quy mô lớn hơn.

Chương 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 KẾT LUẬN

Đề tài đã trình bày quá trình thiết kế và triển khai mô hình mobile robot hướng dẫn viên tự hành tích hợp hệ thống giao tiếp giọng nói thời gian thực trong nhà. Hệ thống được xây dựng dựa trên nền tảng robot di động bánh vi sai, kết hợp các khối phần cứng chính như bộ xử lý trung tâm, vi điều khiển, cảm biến LiDAR, encoder, mạch điều khiển động cơ và khối nguồn.

Về mặt phần mềm, đề tài đã định hướng sử dụng ROS 2 cho các chức năng điều khiển, định vị, xây dựng bản đồ và điều hướng tự hành. Bên cạnh đó, hệ thống AI giao tiếp giọng nói được tích hợp nhằm hỗ trợ người dùng tương tác với robot thông qua các chức năng nhận dạng giọng nói, xử lý ngôn ngữ và phát âm thanh phản hồi.

Kết quả thực nghiệm cho thấy hệ thống đáp ứng tốt các yêu cầu cốt lõi đã đặt ra. Robot xây dựng thành công bản đồ môi trường bằng SLAM Toolbox, thực hiện điều hướng tự động đến các vị trí mục tiêu với sai số vị trí trung bình đều nhỏ hơn đáng kể so với ngưỡng cho phép. Đối với chức năng tương tác giọng nói, hệ thống trả lời chính xác các câu hỏi thuộc ngữ cảnh tham quan, xử lý phù hợp các câu hỏi ngoài phạm vi kiến thức. Vì vậy Robot có thể được phát triển thành nền tảng thử nghiệm cho các bài toán điều hướng, tương tác người-máy và ứng dụng trí tuệ nhân tạo trong robot dịch vụ.

5.2 HẠN CHẾ

Bên cạnh các kết quả đạt được, hệ thống hiện còn một số hạn chế cần được ghi nhận:

- **Độ trễ phản hồi giọng nói:** Nhóm câu hỏi đúng ngữ cảnh có độ trễ trung bình cao hơn ngưỡng đề ra. Nguyên nhân chủ yếu do chuỗi xử lý gồm nhận dạng tiếng nói, suy luận mô hình ngôn ngữ lớn và tổng hợp giọng nói tiếng Việt đều phụ thuộc vào tài nguyên server và độ trễ mạng.
- **Phụ thuộc vào kết nối mạng:** Toàn bộ pipeline xử lý âm thanh và AI hội thoại được triển khai trên server từ xa. Khi kết nối mạng không ổn định, chất lượng nhận dạng giọng nói và thời gian phản hồi bị ảnh hưởng đáng kể.
- **Giới hạn môi trường hoạt động:** Hệ thống chỉ được thử nghiệm trong môi trường trong nhà với cấu trúc tĩnh và ít người di chuyển. Robot chưa được đánh

giá trong các điều kiện có nhiều người di chuyển hoặc bề mặt sàn không bằng phẳng.

- **Nhận dạng giọng nói trong môi trường ồn:** Chất lượng nhận dạng giảm khi có tiếng ồn nền.
- **Cơ sở tri thức cố định:** Nội dung giới thiệu và ngữ cảnh tại các điểm tham quan phải được cấu hình thủ công trước, chưa có cơ chế tự động cập nhật hoặc mở rộng tri thức trong quá trình vận hành.

5.3 HƯỚNG PHÁT TRIỂN

Trong thời gian tới, hệ thống có thể được tiếp tục cải tiến theo các hướng sau:

- Nâng cao độ chính xác định vị bằng cách kết hợp thêm dữ liệu từ IMU, bổ sung thêm màn hình để có thể tương tác trực tiếp với người dùng.
- Cải thiện thời gian phản hồi của hệ thống giao tiếp giọng nói nhằm tăng tính tự nhiên khi tương tác với người dùng, có thể thông qua tối ưu mô hình hoặc triển khai một phần xử lý ngay trên thiết bị.
- Bổ sung microphone mảng đa kênh kết hợp thuật toán lọc nhiễu để cải thiện chất lượng nhận dạng giọng nói trong môi trường có tiếng ồn.
- Bổ sung cơ chế quản lý pin, giám sát trạng thái nguồn và cảnh báo khi năng lượng thấp.
- Mở rộng cơ sở dữ liệu tri thức để robot có thể cung cấp thông tin phong phú hơn trong vai trò hướng dẫn viên.
- Thử nghiệm và đánh giá hệ thống trong các môi trường thực tế có quy mô lớn hơn như bảo tàng, triển lãm hoặc không gian công cộng.

Những hướng phát triển trên sẽ giúp hệ thống hoàn thiện hơn về độ ổn định, khả năng tự hành và chất lượng tương tác, từ đó tăng tính ứng dụng của robot trong các môi trường như trường học, bảo tàng, triển lãm và bệnh viện.

Tài liệu tham khảo

- [1] SLAMTEC, “RPLIDAR A1M8-R6 Datasheet,” <https://www.unikeyic.com/media/datasheet/ae/49/4cec/ae494cec0593c6d73038de349c184489.pdf>, 2024, accessed: 2026-05-02.
- [2] Orange Pi, “Orange Pi 5 RK3588S User Manual v2.1.1,” https://orangepi.net/wp-content/uploads/2024/09/OrangePi_5_RK3588S_User-Manual_v2.1.1.pdf, 2024, accessed: 2026-05-06.
- [3] STMicroelectronics, “STM32F103x8 STM32F103xB Datasheet,” <https://www.st.com/resource/en/datasheet/stm32f103c8.pdf>, 2023, accessed: 2026-05-09.
- [4] Thế Giới IC, “Động cơ giảm tốc TT có Encoder 2 pha AB 166RPM,” <https://www.thegioiic.com/dong-co-giam-toc-tt-co-encoder-2-pha-ab-166rpm>, 2024, accessed: 2026-05-11.
- [5] SparkFun Electronics, “L298N Dual Full-Bridge Motor Driver Datasheet,” https://cdn.sparkfun.com/assets/7/1/d/6/c/Full-Bridge_Motor_Driver_Dual_-_L298N.pdf, 2023, accessed: 2026-05-13.
- [6] Lyons Computer, “USB PSU XY-3606 Single Port DC-DC Buck Converter Module,” <https://www.lyonscomputer.com.au/Electronic-Modules/USB-PSU-XY-3606-Single-Port/USB-PSU-XY-3606-Single-Port.html>, 2024, accessed: 2026-05-16.
- [7] Seeed Studio, “Build Connected Robots with NVIDIA Isaac and ROS 2,” <https://www.seeedstudio.com/blog/2022/12/23/build-connected-robots-with-nvidia-isaac-and-ros2/>, 2022, accessed: 2026-05-30.
- [8] MathWorks, “Pure Pursuit Controller,” <https://ww2.mathworks.cn/help/nav/ug/pure-pursuit-controller.html>, 2026, accessed: 2026-06-10.
- [9] B. P. E. A. Vásquez and F. Matia, “A tour-guide robot: Moving towards interaction with humans,” *Engineering Applications of Artificial Intelligence*, vol. 88, p. 103356, 2020.

- [10] Y. Zhang, Y. Peng, and L. Zhao, “Design and simulation of indoor tour guide robot based on ros,” in *International Conference on Artificial Life and Robotics*, 2021.
- [11] S. Rosa, M. Randazzo, E. Landini, S. Bernagozzi, G. Sacco, M. Piccinino, and L. Natale, “Tour guide robot: a 5G-enabled robot museum guide,” *Frontiers in Robotics and AI*, vol. 10, p. 1323675, 2024, accessed: 2026-06-08.
- [12] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. MIT Press, 2011.
- [13] Columbia University, “Differential Drive Kinematics,” <https://www.cs.columbia.edu/~allen/F17/NOTES/icckinematics.pdf>, 2017, accessed: 2026-05-20.
- [14] F. Dellaert and S. Hutchinson, “Introduction to Robotics and Perception,” <https://www.roboticsbook.org/>, 2023, online Textbook, Accessed: 2026-05-07.
- [15] ros2_control, “Wheeled Mobile Robot Kinematics,” https://control.ros.org/humble/doc/ros2_controllers/doc/mobile_robot_kinematics.html, 2022, accessed: 2026-05-12.
- [16] ROS.org, “REP 103: Standard Units of Measure and Coordinate Conventions,” <https://www.ros.org/reps/rep-0103.html>, 2022, accessed: 2026-05-03.
- [17] K. M. Lynch and F. C. Park, “Modern Robotics: Mechanics, Planning, and Control,” <https://modernrobotics.northwestern.edu/>, 2017, accessed: 2026-05-18.
- [18] ros2_control, “diff_drive_controller Documentation,” https://control.ros.org/humble/doc/ros2_controllers/diff_drive_controller/doc/userdoc.html, 2022, accessed: 2026-05-15.
- [19] Texas Instruments, “Interfacing with Quadrature Encoders,” <https://www.ti.com/>, 2020, technical Document, Accessed: 2026-05-28.
- [20] National Instruments, “Measurements Using Quadrature Encoders,” <https://www.ni.com/>, 2022, accessed: 2026-06-01.
- [21] Motion Control Tips, “How are encoders used for speed measurement?” <https://www.motioncontroltips.com/how-are-encoders-used-for-speed-measurement/>, 2022, accessed: 2026-06-03.

- [22] R. J. E. Merry, M. J. G. van de Molengraft, and M. Steinbuch, “Velocity and acceleration estimation for optical incremental encoders,” *Mechatronics*, vol. 20, no. 1, pp. 20–26, 2010.
- [23] APMonitor, “Proportional Integral Derivative – Dynamics and Control,” <https://apmonitor.com/pdc/index.php/Main/ProportionalIntegralDerivative>, 2023, accessed: 2026-05-22.
- [24] K. J. Åström and R. M. Murray, “PID Control,” <https://www.cds.caltech.edu/~murray/courses/cds101/fa02/caltech/astrom-ch6.pdf>, 2002, accessed: 2026-05-25.
- [25] E. Sundström, K. E. Häggblom, and S. Skogestad, “Reference implementation of the pid controller,” in *IFAC PID 2024*, 2024.
- [26] Open Source Robotics Foundation, “ROS 2 Humble Hawksbill Documentation,” <https://docs.ros.org/en/humble/>, 2022, accessed: 2026-05-10.
- [27] Y. Maruyama, S. Kato, and T. Azumi, “Exploring the performance of ros2,” in *Proceedings of the 13th International Conference on Embedded Software*. ACM, 2016, pp. 1–10.
- [28] Open Source Robotics Foundation, “Synchronous versus Asynchronous Service Clients in ROS 2,” <https://docs.ros.org/en/rolling/How-To-Guides/Sync-Vs-Async.html>, 2026, accessed: 2026-06-11.
- [29] Open Robotics, “Quality of Service Settings in ROS 2,” <https://docs.ros.org/en/humble/Concepts/Intermediate/About-Quality-of-Service-Settings.html>, 2022, accessed: 2026-06-19.
- [30] ROS 2 Visualization Team, “RViz2: 3D Visualization Tool for ROS 2,” <https://github.com/ros2/rviz>, 2022, accessed: 2026-05-14.
- [31] L. Joseph and J. Cacace, *Mastering ROS for Robotics Programming: Design, build, and simulate complex robots using ROS 2*, 3rd ed. Packt Publishing, 2021.
- [32] S. Macenski and I. Jambrecic, “Slam toolbox: Slam for the dynamic world,” *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021.

- [33] Open Navigation LLC, “Navigation2 (Nav2) Documentation,” <https://docs.nav2.org/>, 2026, accessed: 2026-05-17.
- [34] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.
- [35] A. Elfes, “Using occupancy grids for mobile robot perception and navigation,” *Computer*, vol. 22, no. 6, pp. 46–57, 1989.
- [36] D. Fox, “Kld-sampling: Adaptive particle filters,” in *Advances in Neural Information Processing Systems*, 2001, pp. 713–720.
- [37] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [38] S. Macenski, S. Singh, F. Martin, and J. Gines, “Regulated pure pursuit for robot path tracking,” *Autonomous Robots*, 2023.
- [39] Open Robotics, “tf2 in ROS 2,” <https://docs.ros.org/en/humble/Concepts/Intermediate/About-Tf2.html>, 2022, accessed: 2026-06-16.
- [40] World Wide Web Consortium, “WebRTC: Real-Time Communication in Browsers,” <https://www.w3.org/TR/webrtc/>, 2025, w3C Recommendation, Accessed: 2026-06-13.
- [41] A. Keranen, C. Holmberg, and J. Rosenberg, “Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal,” Internet Engineering Task Force, RFC 8445, 2018, accessed: 2026-06-15. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8445.html>
- [42] T. Reddy, A. Johnston, P. Matthews, and J. Rosenberg, “Traversal Using Relays around NAT (TURN): Relay Extensions to Session Traversal Utilities for NAT (STUN),” Internet Engineering Task Force, RFC 8656, 2020, accessed: 2026-06-17. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8656.html>
- [43] LiveKit, “LiveKit Real-Time Communication and Agents Documentation,” <https://docs.livekit.io/>, 2026, accessed: 2026-06-05.
- [44] J. Sohn, N. S. Kim, and W. Sung, “A statistical model-based voice activity detection,” *IEEE Signal Processing Letters*, vol. 6, no. 1, pp. 1–3, 1999.

- [45] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust Speech Recognition via Large-Scale Weak Supervision,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 28 492–28 518. [Online]. Available: <https://proceedings.mlr.press/v202/radford23a.html>
- [46] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: <https://papers.nips.cc/paper/7181-attention-is-all-you-need>
- [47] J. Kim, J. Kong, and J. Son, “Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech,” in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139. PMLR, 2021, pp. 5530–5540. [Online]. Available: <https://proceedings.mlr.press/v139/kim21f.html>
- [48] Open Robotics, “nav_msgs/OccupancyGrid Message Definition,” https://docs.ros.org/en/noetic/api/nav_msgs/html/msg/OccupancyGrid.html, accessed: 2026-06-04.
- [49] Open Robotics Foundation, “nav_msgs/MapMetaData Message Definition,” https://docs.ros.org/en/noetic/api/nav_msgs/html/msg/MapMetaData.html, accessed: 2026-06-07.